

Algoritmi e strutture dati - Bonomi & D'Amore

Giovanni Esposito, Danilo Di Primio, Cristian Amendola

Università degli studi La Sapienza

Ingegneria Informatica e Automatica - 2 anno

Febbraio - Maggio 2026

Indice

1	Algoritmi	5
1.1	Costo di un algoritmo	5
	Complessità temporale	5
	Complessità spaziale	5
	Istruzione dominante	5
1.2	Andamento asintotico	5
	Analisi degli algoritmi	6
	Notazioni	6
1.3	Categorie di algoritmi	7
	Costante	7
	Cicli	8
	Ricorsione	8
1.4	Definizioni importanti	8
	Upper Bound	8
	Lower Bound	8
	Definizione di $\Omega(g(x))$	8
	O-grande di una funzione	9
	Definizione di $\theta(f(x))$	9
	Definizione di Istruzione Dominante	9
	Upper Bound problema	10
	Lower Bound problema	10
	Algoritmo ottimo	10
2	Algoritmi di ordinamento	11
2.1	Insertion Sort	11
	Pseudocodice InsertionSort	12
	Analisi dell'insertion sort	12
	Costi InsertionSort	13
2.2	Invariante di ciclo	14
2.3	Divide et impera	14
2.4	Merge sort	15
	Pseudocodice MergeSort	15
	Analisi del MergeSort	16
	Equazione di ricorrenza	16
2.5	Quick sort	17
	Pseudocodice QuickSort	18
	Dimostrazione della correttezza di Quicksort	19
	Prestazioni QuickSort	19
	Tempo di esecuzione QuickSort	19
2.6	Ricorrenze	19
2.7	Impostazione di una ricorrenza	20
2.8	Alberi di ricorsione per le ricorrenze	23
2.9	Alberi di decisione	25
	Teorema su alberi di decisione	26
	Teorema su algoritmi di ordinamento	26
2.10	Counting Sort	26
2.11	Bucket sort	27
	Pseudocodice BucketSort	27
	Analisi Bucket Sort	28

2.12	Radix Sort LSD	29
2.13	Riduzione (fra problemi)	29
2.14	HeapSort	30
	Definizione Heap	30
	Max-heap e Min-heap	30
	Max-Heapify	31
	Costruire un max-heap	32
	Heap Sort	33
	Code di proprietà	33
3	Il problema della selezione e le statistiche d'ordine	36
3.1	Problema selezione	36
3.2	Minimo o Massimo	36
3.3	Minimo e Massimo	36
	Pseudocodice	37
3.4	Selezione Randomizzata	38
	Pseudocodice	39
	Analisi del costo	39
	Analisi del valore atteso del tempo di esecuzione	39
3.5	Selezione in tempo	42
	Pseudocodice	42
	Analisi	43
4	Programmazione Dinamica	44
4.1	Panoramica generale	44
4.2	Sviluppo dell'algoritmo	44
	Funzione di stato	44
	Top Down	44
	Bottom Up	45
4.3	Problema dello zaino 0/1	45
4.4	Cammini su griglia (senza ostacoli)	47
4.5	Problema LIS (List Increasing Subsequence)	47
4.6	Problema MSIS (Max Sum Increasing Subsequence)	48
4.7	Problema WIS (Weighted Interval Scheduling)	48
4.8	Problema LCS (Longest Common Subsequence)	49
4.9	Problema del Rod Cutting	50
4.10	Matrix Chain Multiplication	51
4.11	Coin Change Analysis	52
4.12	Distanza di edit (distanza di Levenshtein)	53
5	Algoritmi greedy	55
5.1	Introduzione e definizione	55
5.2	Problema della codifica di Huffman	55
5.3	Problema di Activity selection	57
	Dimostrazione dell'ottimalità	57
5.4	Confronto con altri approcci	58
6	Ridimensionamento	59
6.1	tabelle dinamiche	59
	Ridimensionamento: espansione	60
	Ridimensionamento: Contrazione	61
7	Hash	62
7.1	Dizionari	62
7.2	Tabelle ad indirizzamento diretto	62
	Operazioni su una tabella ad indirizzamento diretto	62
	Fattore di carico	63
7.3	Tabelle o tavole di Hash	63
	Collisioni	63
	Metodo del concatenamento (Liste di trabocco)	64
	Operazioni sulle liste di trabocco	64

Fattore di carico	64
Analisi	64
Teoremi metodo del concatenamento	64
Funzioni di hash	65
Hashing Statico	65
Hashing Randomizzato	66
Progettare una famiglia universale di funzioni di hash - Metodo basato sulla teoria dei numeri . .	66
7.4 Metodo dell'indirizzamento aperto	66
Ispezione lineare	68
Ispezione quadratica	68
Ispezione con doppio hashing	68
Tabella hash Adattiva	68
8 Grafi	69
Definizioni sui Grafi	69
8.1 Rappresentazione di un grafo	70
8.2 Visita in ampiezza BFS	71
Pseudocodice BFS	71
Analisi	72
Proprietà delle distanze	72
Proprietà del limite superiore	73
Proprietà della coda	73
Correttezza della BFS	74
8.3 DFS	75
Pseudocodice	75
Analisi	76
Proprietà delle parentesi	76
Teorema delle Parentesi	77
Teorema del cammino bianco	77
Tipi di archi DFS	78
Ordinamento Topologico	78
Componenti fortemente connesse	79
Grafo Trasposto G^T	79
8.4 Shortest Path (Percorsi minimi)	80
Proprietà	80
Rappresentazione dei cammini minimi	81
Rilassamento	81
Algoritmo di Bellman-Ford	82
Algoritmo di Bellman Ford versione distribuita*	83
Routing Loop	83
Algoritmo di Floyd-Warshall (APSP con pesi arbitrari)	83
Chiusura Transitiva e Riflessiva - Variante Booleana di Floyd-Warshall	84
8.5 Alberi di connessione minimi MST	85
Il problema degli alberi di connessione minimi	85
Definizioni preliminari	85
Algoritmo greedy per generare un albero di connessione minimo	85
Teorema: Caratterizzazione degli archi scuri	86
Algoritmo di Kruskal	86
Algoritmo di Prim	90
9 Alberi	93
9.1 Tipi di albero	93
9.2 Alberi binari	93
Rappresentazione degli alberi binari	93
Albero binario di ricerca BST	94
Visita di un albero binario	94
Analisi della complessità	95
Interrogazioni di un albero binario di ricerca	96
9.3 Alberi Rosso-Neri	98
Proprietà di un albero rosso-nero	99

10 Esercizi **102**

10.1 Esercizi venerdì 13/03/2026 102

10.2 Esercizi venerdì 17/04/2026 (Tabelle Hash) 105

Capitolo 1

Algoritmi

Definizione 1.0.1. Un algoritmo è una procedura di calcolo ben definita che prende un certo valore (o un insieme di valori) come input e genera, in un tempo finito, un valore (o un insieme di valori) come output.

Un algoritmo è quindi una sequenza di passi computazionali che trasforma l'input nell'output.

1.1 Costo di un algoritmo

Definizione 1.1.1. Un algoritmo A ha costo $O(g(n))$ se la quantità di tempo (spazio) sufficiente per eseguire A su un input di dimensione n nel caso peggiore è $O(g(n))$.

Complessità temporale

La complessità temporale di un algoritmo è la misura del tempo di esecuzione di un algoritmo in funzione della dimensione dei dati in input. Questa complessità cresce al crescere della dimensione in input.

Complessità spaziale

La complessità spaziale di un algoritmo è una misura delle risorse hardware utilizzate durante l'esecuzione di un algoritmo. Se un algoritmo richiede più memoria per elaborare un insieme di dati più grande, allora ha una complessità spaziale più alta.

Istruzione dominante

Quando si analizza un algoritmo in cui sono presenti molti confronti, assegnazioni ecc..., è complicato calcolare il costo, in tal caso si prende in considerazione l'istruzione dominante, ovvero quell'operazione che si ripete più volte nel programma, tipicamente quella contenuta nei cicli più interni.

1.2 Andamento asintotico

L'andamento asintotico di un algoritmo è una misura della crescita del tempo di esecuzione o dello spazio richiesto quando le dimensioni dell'input crescono. In altre parole l'andamento asintotico ci dice come si comporta l'algoritmo quando l'input diventa grande, ovvero quando cresce la sua dimensione, il numero di bit di cui si ha bisogno di rappresentarlo (se l'input è pari ad n, la sua dimensione sarà pari a $\lceil \log_2(n+1) \rceil$). Il logaritmo in base 2 di un numero rappresenta esattamente il numero di bit necessari per rappresentarlo.

L'analisi ha le seguenti caratteristiche:

- **Asintotica** $\rightarrow$ non vogliamo sapere la funzione di costo esatta, ma vogliamo sapere il suo andamento asintotico ($f(x)$ per $x \rightarrow \infty$)

Due funzioni $f(x)$ e $g(x)$ hanno lo **stesso comportamento asintotico** quando $\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = C > 0$

Algoritmo divisi in 2:

- Costo esponenziale
- Costo polinomiale

- Analisi asintotica di **caso peggiore** (worst case)
- Analisi asintotica della **dimensione in funzione dell'input**
 - Array in input con n celle, ogni cella ha k bit, dimensione input $k \cdot n$ essendo k costante mentre $n \rightarrow \infty$, k la definiamo costante moltiplicativa perciò non ci interessa.

L'algoritmo asintotico è utile perchè ci consente di fare previsioni sulle prestazioni di un algoritmo su input grandi senza dover eseguire l'algoritmo su input effettivamente grandi.

Il **comportamento asintotico** del costo di un programma si basa sull'idea di trascurare costanti moltiplicative e termini di ordine inferiore, fornendo un valore che cambia man mano che la dimensione aumenta. Ad esempio, il costo $an+b$ ha comportamento asintotico lineare, il costo $an^2 + b$ ha comportamento asintotico quadratico, il costo $a \cdot \log_b n + c$ ha comportamento asintotico logaritmico.

Per calcolare la complessità di un algoritmo si utilizza l'analisi asintotica che trasforma il tempo di esecuzione dell'algoritmo in una funzione $T(n)$ in funzione della dimensione n dei dati in input, con la quale misurare il numero di comandi eseguiti dall'algoritmo.

Analisi degli algoritmi

Analizzare un algoritmo significa prevedere le risorse che l'algoritmo richiederà, tra cui il tempo di elaborazione. Analizzando più algoritmi candidati a risolvere uno stesso problema, si può identificare quello più efficiente. Prima di analizzare un algoritmo, c'è bisogno di avere un modello della tecnologia di implementazione che sarà utilizzata e di un modo per esprimere i loro costi. Noi utilizzeremo il modello RAM (Random-Access Machine).

Notazioni

Ci sono delle notazioni principali utilizzate per esprimere l'andamento asintotico di un algoritmo:

Notazione O

La notazione O definisce un limite superiore al comportamento asintotico di una funzione. In altre parole, ci dice che una funzione non cresce più velocemente di un certo tasso di crescita, determinato dal termine di ordine superiore.

Esempio.

Data la funzione

$$7n^3 + 100n^2 - 20n + 6$$

il suo termine di ordine superiore è $7n^3$ e quindi diciamo che il tasso di crescita di questa funzione è n^3 , poichè questa funzione non cresce più velocemente di n^3 possiamo scrivere che è $O(n^3)$

Notazione Ω

La notazione Ω definisce un limite inferiore al comportamento asintotico di una funzione. In altre parole, ci dice che una funzione cresce almeno alla stessa velocità di un certo tasso di crescita.

Esempio.

Data la funzione

$$7n^3 + 100n^2 - 20n + 6$$

questa funzione è $\Omega(n^3)$, ma anche $\Omega(n^2)$ e $\Omega(n)$, più in generale è $\Omega(n^c)$ per ogni costante $c \leq 3$.

Notazione Θ

La notazione Θ definisce un limite stretto al comportamento asintotico di una funzione. Ci dice che una funzione cresce esattamente con un certo tasso di crescita.

Esempio.

Se riuscite a dimostrare che una funzione è al contempo $O(f(n))$ e $\Omega(f(n))$ per una qualche $f(n)$, allora avete dimostrato che la funzione è proprio $\Theta(f(n))$, nel nostro esempio è sia $\Omega(n^3)$ sia $O(n^3)$, allora è anche $\Theta(n^3)$.

Notazione o

Utilizziamo la notazione o per denotare un limite superiore che non è asintoticamente stretto.

Esempio.

$$2n = o(n^2), \text{ ma } 2n^2 \neq o(n^2)$$

Notazione ω

Utilizziamo la notazione ω per indicare un limite inferiore che non è asintoticamente stretto.

Esempio.

$$\frac{n^2}{2} = \omega(n), \text{ ma } \frac{n^2}{2} \neq \omega(n^2)$$

Proprietà

Proprietà transitiva

$$f(n) = \Theta(g(n)) \text{ e } g(n) = \Theta(h(n)) \text{ implicano } f(n) = \Theta(h(n))$$

$$f(n) = O(g(n)) \text{ e } g(n) = O(h(n)) \text{ implicano } f(n) = O(h(n))$$

$$f(n) = \Omega(g(n)) \text{ e } g(n) = \Omega(h(n)) \text{ implicano } f(n) = \Omega(h(n))$$

$$f(n) = o(g(n)) \text{ e } g(n) = o(h(n)) \text{ implicano } f(n) = o(h(n))$$

$$f(n) = \omega(g(n)) \text{ e } g(n) = \omega(h(n)) \text{ implicano } f(n) = \omega(h(n))$$

Proprietà riflessiva

$$f(n) = \Theta(f(n))$$

$$f(n) = O(f(n))$$

$$f(n) = \Omega(f(n))$$

Proprietà simmetrica

$$f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$$

Proprietà trasposta

$$f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \iff g(n) = \omega(f(n))$$

1.3 Categorie di algoritmi

Dividiamo in 3 categorie le istruzioni degli algoritmi

- costante
- cicli
- ricorsioni

Costante

Al variare dell'input non cambia nulla.

Esempio.

$x = 10 \rightarrow$ non dipende dall'input, costo costante

Esempio.

$y = \frac{10+20}{5} \rightarrow$ costo non cambia al variare dell'input, costo costante

Esempio.

$z = x + 20y \rightarrow$ costo costante

Esempio.

return 0 $\rightarrow$ costo costante

Cicli

Attribuiamo il costo di un ciclo in base a quante volte viene eseguito il corpo del ciclo

Esempio.

```
for (i=0; i<n; i++) {  
    for (j=0; j<n; j++) {  
        ... costante  
        ... costante  
    }  
}
```

Il ciclo viene eseguito $n \cdot m$ volte, quindi il costo sarà proprio $n \cdot m$.

Esempio.

```
for (i=0; i<n; i++) {  
    for (j=0; j<i; j++) {  
        ... costante  
        ... costante  
    }  
}
```

In questo ciclo il costo è $\frac{(n-1)n}{2}$, quindi il costo sarà quadratico.

Esempio.

Nel caso in cui abbiamo 2 cicli:

- ciclo1 $\rightarrow$ costo: n^2
- ciclo2 $\rightarrow$ costo n

Il ciclo/cicli con costo più basso può essere trascurato

Ricorsione

R

```
----- -> costo costante  
----- -> costo costante  
R  
    ----- -> costo costante  
    ...
```

Il costo di una ricorsione dipende da quanti livelli ricorsivi (numero di chiamate ricorsive) vengono eseguiti.

1.4 Definizioni importanti

Upper Bound

Definizione 1.4.1. (Upper Bound) L'upper bound è un tetto al costo dell'algoritmo.

Quando un upper bound è il più basso possibile lo chiamiamo **tight**.

Lower Bound

Definizione 1.4.2. (Lower Bound) Il lower bound è un pavimento al costo dell'algoritmo.

Definizione di $\Omega(g(x))$

Definizione 1.4.3.

$$f(x) \in \Omega(g(x)) \Leftrightarrow \exists C > 0 : \forall n > n_0 \quad f(x) > C \cdot g(x)$$

O-grande di una funzione

Definizione 1.4.4. (O-grande)

$$f(n) \in O(g(n)) \Leftrightarrow \exists C > 0 : \forall n > n_0 \quad f(n) > C \cdot g(n)$$

Definizione di $\theta(f(x))$

Definizione 1.4.5.

$$f(x) \in \theta(g(x)) \Leftrightarrow f(x) \in O(g(x)) \wedge f(x) \in \Omega(g(x))$$

Definizione di Istruzione Dominante

Definizione 1.4.6. supponiamo che:

$$A \in O(f(n))$$

istruzione D con contributo espresso da $g(n)$:

$$\exists c > 0 : c \cdot g(n) > f(n)$$

Perchè facciamo l'analisi asintotica?

Con l'analisi asintotica non solo valutiamo l'andamento asintotico del costo ma possiamo trascurare tutte le costanti moltiplicative.

Esempio.

Funzione `ordina (int primo, int ultimo);`

```
int ordina ( int primo , int ultimo ) {  
    if( ultimo > primo ) infila ( primo , ordina ( primo + 1 , ultimo ) );  
    return ultimo ;  
}
```

Il costo della funzione è definito dalla seguente:

$$C_O = C_I + C_O$$

Il costo è quindi la somma della chiamata della funzione `infila` e la chiamata ricorsiva di `ordina`. Rappresentiamo il costo esplicitando i parametri:

$$C_O(a, b) = C_I(a, b) + C_O(a + 1, b) \quad \text{se } a < b$$

Se invece $a \leq b$ la funzione esegue il `return`, che è un'istruzione a costo costante C_1 .

Ora esplicitiamo la ricorsione:

$$C_O(0, n - 1) = C_I(a, b) + C_I(a + 1, b) + C_I(a + 2, b) + \dots + C_I(a + ..., b) + C_1$$

Che possiamo riscrivere con una sommatoria:

$$C_O(0, n - 1) = \sum_{i=0}^{n-2} C_I(i, n - 1) + C_1$$

Ora vediamo il costo della funzione `infila (int i, int ultimo);`

```
static void infila ( int i , int ultimo ) {  
    int h = i + 1;  
    int k = A [ i ];  
    while ( ( h <= ultimo ) && ( A[ h ] < k ) ) {  
        A [h -1] = A [ h ];  
        h ++;  
    }  
    A [h -1] = k ;  
}
```

L'istruzione dominante è un ciclo while, quindi ci concentriamo su quello.
 Il costo di un ciclo while è il massimo numero di iterazioni che può compiere

$$\text{Numero di iterazioni : } x + 1 \dots\dots y = y + 1 - (x + 1) = \mathbf{y - x}$$

Il costo di **infil**a sarà quindi il numero di iterazioni più una costante che rappresenta il costo delle istruzioni a costo costante fuori dal ciclo.

$$C_I(x, y) = y - x + C_2$$

Avendo calcolato il costo di C_I , possiamo sostituire questo risultato al costo di **ordina** che avevamo trovato:

$$C_O(0, n-1) = \sum_{i=0}^{n-2} (\underbrace{n-1}_{y} - \underbrace{i}_{x} + C_2) + C_1$$

Per risolvere questa sommatoria applico la proprietà distributiva e divido la sommatoria in due sommatorie, dove nella prima compaiono i termini costanti e nella seconda i termini che vengono incrementati.

$$\sum_{i=0}^{n-2} (n-1-i+C_2) + C_1 = \sum_{i=0}^{n-2} (n-1+C_2) - \sum_{i=0}^{n-2} i$$

Per la prima sommatoria visto che sono tutti termini costanti vuol dire che l'espressione dentro la sommatoria si ripeterà $n-1$ volte. Per la seconda usiamo il criterio di Gauss per la somma dei numeri naturali.

$$\sum_{i=0}^{n-2} (n-1+C_2) - \sum_{i=0}^{n-2} i = (n-1)(n-1+C_2) - \frac{(n-2)(n-1)}{2}$$

Per calcolare il costo basta fare questo calcolo:

$$(n-1)(n-1+C_2) - \frac{(n-2)(n-1)}{2} = \frac{n^2 - 2n(2-C_2) + 2(1-C_2) - n^2 + 3n - 2}{2}$$

L'algoritmo ha quindi complessità ordine n^2 .

Upper Bound problema

Definizione 1.4.7. Un problema P ammette

$$\begin{aligned} &UB \ O(f(n)) \\ &\Leftrightarrow \\ &\exists \ A(\text{che risolve P}) : UB(A) \in O(f(n)) \end{aligned}$$

Lower Bound problema

Definizione 1.4.8. Un problema P ammette

$$\begin{aligned} &\Omega \ g(f(n)) \\ &\Leftrightarrow \\ &\forall \ A(\text{che risolve P}) : LB(A) \in \Omega(g(n)) \end{aligned}$$

Algoritmo ottimo

Definizione 1.4.9. Quando LB e UB coincidono, l'algoritmo è definito OTTIMO.

Capitolo 2

Algoritmi di ordinamento

Il problema dell'ordinamento è un problema molto comune e spesso è il passo base per risolvere problemi più complessi.

Esempio.

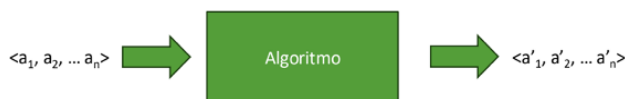
Vogliamo mettere in ordine alfabetico una lista di studenti prenotati ad un esame, perciò è necessario avere:

- un insieme di elementi
- un criterio in base a cui vogliamo ordinare

Definiamo il problema dell'ordinamento come segue:

input → una sequenza di n elementi $\langle a, b, c, \dots, d \rangle$

output → una permutazione $\langle a', b', c', \dots, d' \rangle$ della sequenza di input tale che $a' \leq b' \leq \dots \leq d'$



Definizione 2.0.1. (Istanza) L'istanza di un problema è formata dall'input richiesto per calcolare una soluzione del problema.

Definizione 2.0.2. (Algoritmo corretto) Algoritmo si dice corretto relativamente ad un problema computazionale se, per ogni istanza fornita in input esso:

1. Termina
2. Restituisce una soluzione corretta per l'istanza considerata

Algoritmi In-Place

Alcuni algoritmi di ordinamento possono ordinare una lista senza crearne una nuova. Questi sono conosciuti come algoritmi di ordinamento in loco (o in place) e richiedono uno spazio di memoria extra costante $O(1)$. InsertionSort e QuickSort sono IN-PLACE, in quanto gli elementi vengono spostati internamente all'array e non allocano memoria aggiuntiva. MergeSort non è IN PLACE, richiede memoria aggiuntiva per i diversi frammenti generati.

Definizione 2.0.3. (Definizione 1) Un algoritmo si definisce IN PLACE quando è in grado di restituire l'output utilizzando soltanto un piccolo e costante spazio di memoria extra, i dati in ingresso sono solitamente sovrascritti.

Definizione 2.0.4. (Definizione 2) Un algoritmo si dice IN PLACE se al crescere dell'input non crescono le variabili di appoggio.

2.1 Insertion Sort

Insertion Sort è un algoritmo di ordinamento molto semplice che ordina l'insieme di input come farebbe un umano per un mazzo di carte

1. inizialmente tutte le carte sono nel mazzo disordinato (input).
2. prendiamo la prima carta e la teniamo con la mano sinistra.
3. prendiamo la seconda carta, la confrontiamo con la prima e la posizioniamo alla sua destra o alla sua sinistra in base alla loro relazione d'ordine.
4. prendiamo la terza carta, la confrontiamo con quelle che abbiamo in mano e la sistemiamo nel posto giusto.
5. ripetiamo l'operazione di prendere una nuova carta e sistemarla nel posto giusto finché il mazzo non è finito.

Pesudocodice InsertionSort

:

```

INSERTION-SORT(A,n)
for i = 2 to n
    key = A[i]
    // inserisce A[i] nella sequenza ordinata A[1:i-1]
    j = i - 1
    while j > 0 and A[j] > key
        A[j + 1] = A[j]
        j = j - 1
    A[j + 1] = key

```

L'indice i identifica la carta corrente che viene inserita nella mano sinistra. All'inizio di ogni iterazione del ciclo `for`, il cui indice è i , il sotto-array formato dagli elementi $A[1 : i - 1]$ costituisce la mano di carte correntemente ordinate, mentre il sotto-array $A[i + 1 : n]$ corrisponde alla pila di carte ancora sul tavolo. In effetti, gli elementi $A[1 : i - 1]$ sono quelli che originariamente occupavano le posizioni da 1 a $i - 1$, ma che adesso sono ordinati. Definiamo formalmente queste proprietà soddisfatte da $A[i : i - 1]$ in forma di invariante di ciclo.

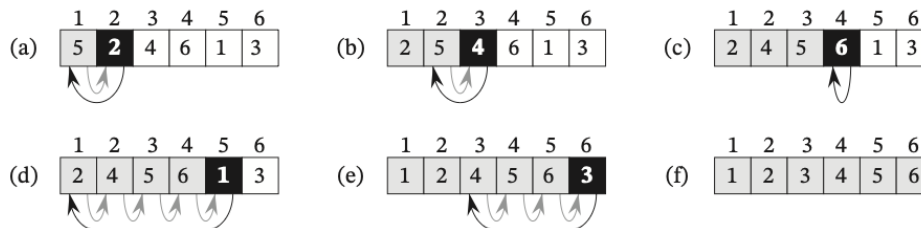


Figura 2.2 Il funzionamento della procedura $\text{INSERTION-SORT}(A, n)$ in cui originariamente A contiene la sequenza $\langle 5, 2, 4, 6, 1, 3 \rangle$ e $n = 6$. Gli indici dell'array sono indicati sopra i rettangoli; i valori memorizzati nelle posizioni dell'array sono indicati all'interno dei rettangoli. (a)-(e) Le iterazioni del ciclo `for` (righe 1-8). In ogni iterazione, il rettangolo nero contiene la chiave estratta da $A[i]$, che viene confrontata con i valori nei rettangoli grigi alla sua sinistra nel test della riga 5. Le frecce grigie mostrano i valori dell'array che vengono spostati di una posizione verso destra nella riga 6; le frecce nere indicano dove viene spostata la chiave nella riga 8. (f) L'array finale ordinato.

Analisi dell'insertion sort

Per prima cosa, dobbiamo comprendere che il tempo di esecuzione dipende dall'input, ovviamente ordinare più numeri richiederà più tempo. Per analizzare la procedura `insertion-sort`, la scriviamo qua sotto indicando a fianco di ogni istruzione il costo temporale e il numero di volte per cui viene eseguita.

$\text{INSERTION-SORT}(A, n)$	costo	numero di iterazioni
1 for $i = 2$ to n	c_1	n
2 $key = A[i]$	c_2	$n - 1$
3 // Inserisce $A[i]$ nel sottoarray // ordinato $A[1 : i - 1]$.	0	$n - 1$
4 $j = i - 1$	c_4	$n - 1$
5 while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6 $A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7 $j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8 $A[j + 1] = key$	c_8	$n - 1$

Il tempo di esecuzione dell'algoritmo è la somma dei tempi di esecuzione per ogni istruzione eseguita; un'istruzione che richiede c_k passi viene eseguita m volte contribuirà con $c_k \cdot m$ al tempo di esecuzione totale. Indicheremo il tempo di esecuzione di un algoritmo su un input di dimensione n con $T(n)$. Per calcolare $T(n)$, il tempo di esecuzione di insertion-sort con un input di n valori, sommiamo i prodotti delle colonne costo e numero di volte ottenendo:

$$T(n) = c_1n + c_2(n-1) + C_4(n-1) + c_5 \sum_{i=2}^n (t_i - 1) + c_8(n-1)$$

Per insertion-sort il caso migliore si verifica quando l'array è già ordinato. Il tempo di esecuzione nel caso migliore è

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$$

Questo tempo di esecuzione può essere espresso come $an+b$, dove a e b sono costanti che dipendono dai costi c_k delle istruzioni; quindi la funzione che esprime il tempo di esecuzione è una funzione lineare rispetto a n . Il caso peggiore si verifica quando l'array è ordinato in senso inverso, cioè in ordine decrescente. il tempo di esecuzione della procedura insertion-sort nel caso peggiore è

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) = \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + (c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8)n - (c_2 + c_4 + c_5 + c_8) \end{aligned}$$

Questo tempo di esecuzione può essere espresso come $an^2 + bn + c$, dove a , b e c sono costanti che dipendono dai costi delle istruzioni c_k ; quindi è una funzione quadratica rispetto a n .

Noi ci concentreremo solo sulla ricerca del tempo di esecuzione nel caso peggiore per tre ragioni:

1. il tempo di esecuzione nel caso peggiore di un algoritmo fornisce un limite superiore al tempo di esecuzione su un qualsiasi input.
2. per alcuni algoritmi, il caso peggiore si verifica molto spesso.
3. il "caso medio" spesso ha un costo simile a quello peggiore.

La velocità con cui cresce il tempo di esecuzione, ossia il suo tasso di crescita, è quello che effettivamente ci interessa. Di conseguenza consideriamo soltanto il termine principale di una formula, in quanto i termini di ordine inferiore sono relativamente poco significativi per grandi valori n . Ignoriamo anche il coefficiente costante del termine principale, in quanto i fattori costanti sono meno significativi del tasso di crescita nel determinare l'efficienza computazionale per grandi input. per il tempo di esecuzione del caso peggiore dell'insertion-sort, se ignoriamo i termini di ordine inferiore e il coefficiente del termine principale, resta solo il fattore n^2 che fa parte del termine principale.

Utilizzeremo la notazione $\Theta(\text{costo})$ per evidenziare l'ordine di crescita del tempo di esecuzione. L'insertion sort ha un tempo di esecuzione nel caso peggiore pari a $\Theta(n^2)$.

Un algoritmo è considerato più efficiente di un altro se il suo tempo di esecuzione nel caso peggiore ha un tasso di crescita minore rispetto all'altro.

Costi InsertionSort

Analizzando lo pseudocodice, si può notare che i cicli risultano essere le operazioni più frequenti. Dunque il ciclo `for` viene eseguito $n-1$ volte, mentre il ciclo `while`, j volte, tuttavia osservando che il valore j viene inizializzato fuori dal `while` a $i-1$, si capisce che l'ultimo valore sarà proprio $j = (n-1) - 1 = n-2$. Da ciò è possibile intuire che il costo dell'algoritmo sarà $(n-1) \cdot (n-2)$, ovvero il costo asintotico sarà pari a $O(n^2)$.

Dobbiamo fornire una **prova di correttezza**, ossia una dimostrazione formale che non considera un input specifico ma ragiona sull'algoritmo, sulle proprietà dell'input e su come viene prodotto l'output

- esistono molte tecniche per dimostrare la correttezza di un algoritmo rispetto ad un problema, come la tecnica degli invarianti di ciclo

2.2 Invariante di ciclo

È una tecnica per dimostrare che un particolare algoritmo risolve correttamente un dato problema. Ci sono tre cose che devono essere dimostrate:

1. Inizializzazione $\rightarrow$ esiste una proprietà che vale prima dell'iterazione del ciclo
2. Mantenimento $\rightarrow$ se la proprietà vale prima del ciclo, allora rimane vera anche prima dell'iterazione successiva
3. Conclusione $\rightarrow$ il ciclo termina e la proprietà che preserviamo dimostra che il risultato è corretto

Definizione 2.2.1. (Invarianza di ciclo) Una dimostrazione di invarianza di ciclo è una forma di dimostrazione per induzione matematica, dove per provare che una proprietà è valida, si prova un caso base e un passo induttivo.

Nel nostro caso, dimostrare che l'invariante è vera prima della prima iterazione corrisponde al caso base, mentre dimostrare che l'invariante resta vera da un'iterazione all'altra corrisponde al passo induttivo.

Inizializzazione

Iniziamo dimostrando che l'invariante di ciclo è vera prima della prima iterazione del ciclo, cioè quando $i = 2$. Il sottoarray $A[i : i - 1]$ è formato dal solo elemento $A[1]$, che infatti è l'elemento originale in $A[1]$. Inoltre, questo sottoarray è ordinato, il che dimostra che l'invariante di ciclo vale prima della prima iterazione del ciclo.

Mantenimento

Passiamo alla seconda proprietà: dimostrare che ogni iterazione conserva l'invariante di ciclo. Informalmente, il corpo del ciclo for esterno opera spostando $A[i - 1]$, $A[i - 2]$, $A[i - 3]$ e così via di una posizione verso destra, finché non troverà la posizione appropriata per $A[i]$, in cui inserirà il valore di $A[i]$. Il sottoarray $A[1 : i]$ quindi è ordinato ed è formato dagli stessi elementi che originariamente erano in $A[1 : i]$. Dunque, l'incremento di i per la successiva iterazione del ciclo for preserva l'invariante di ciclo.

Conclusione

Infine, esaminiamo che cosa accade quando il ciclo termina. La variabile del ciclo i viene inizializzata a 2 e aumenta di 1 a ogni iterazione. Non appena il valore di i eccede n nella linea 1, il ciclo termina. Detto in altro modo: il ciclo termina non appena i assume il valore $n + 1$. Sostituendo $n + 1$ a i nella formulazione dell'invariante del ciclo, si trova che il sotto-array $A[1 : n]$ consiste degli elementi originariamente presenti in $A[1 : n]$, ma ordinati. Pertanto l'algoritmo è corretto.

2.3 Divide et impera

Dato un problema possono esistere diversi algoritmi in grado di risolverlo. Generalmente gli algoritmi ricorsivi adottano un approccio del tipo divide et impera.

- Divide
 1. il problema viene diviso in un certo numero di sottoproblemi
 2. tali sottoproblemi sono istanze più piccole del problema originale
- Impera
 1. i sottoproblemi vengono risolti in modo ricorsivo
 2. si continua a dividere fino ad arrivare ad una istanza sufficientemente piccola da essere risolta direttamente
- Combina
 1. Le soluzioni dei sottoproblemi vengono combinate per formare la soluzione del problema originale

2.4 Merge sort

L'algoritmo merge sort segue da vicino la strategia divide et impera. A ogni passo, ordina un sotto-array $A[r : p]$, partendo dall'intero array $A[1 : n]$ e operando ricorsivamente su sotto-array sempre più piccoli.

- Divide
 1. Se la lista contiene più di un elemento spezzala in due sotto-liste grandi la metà. Per fare ciò calcola il centro q del sottoarray $A[p : r]$ e divide $A[p : r]$ nei sottoarray $A[p : q]$ e $A[q+1 : r]$.
- Impera
 1. Continua a dividere finché la lista non ha dimensione 1. Quindi ordina ciascuno dei sottoarray in modo ricorsivo utilizzando l'algoritmo merge sort
- Combina
 1. Fonde due sotto-liste ordinandole

Pseudocodice MergeSort

```
MERGE-SORT(A,p,r)
if p >= r           //zero o un solo elemento?
    return
q = (p+r)/2         //punto medio di A[p : r]
MERGE-SORT(A,p,q)   //ordina ricorsivamente A[p : q]
MERGE-SORT(A,q+1,r) //ordina ricorsivamente A[q+1 : r]
//fonde A[p : q] con A[q+1 : r] in A[p : r]
MERGE(A,p,q,r)
```

pseudocodice merge (metodo, combina, che fonde due sotto-liste ordinate):

```
MERGE(A,p,q,r)
n_l = q - p + 1      //lunghezza di A[p : q]
n_r = r - q          //lunghezza di A[q+1 : r]
crea due nuovi array L[0 : n_l - 1] e R[0 : n_r - 1]
for i = 0 to n_l - 1 //copia A[p : q] in L[0 : n_l - 1]
    L[i] = A[p + i]
for j = 0 to n_r - 1 //copia di A[q+1 : r] in R[0 : n_r - 1]
    R[j] = A[q + j + 1]
i = 0                //i è l'indice del più piccolo elemento che resta in L
j = 1                //i è l'indice del più piccolo elemento che resta in R
k = p                //k è la posizione di A in cui fare il prossimo inserimento
//finchè L e R contengono un elemento non ancora fuso,
//copia il più piccolo elemento non fuso in A[p : r]
while i < n_l
    if L[i] <= R[j]
        A[k] = L[i]
        i = i + 1
    else A[k] = R[j]
        j = j + 1
    k = k + 1
//avendo esaurito completamente uno tra L e R,
//copia il resto dell'altro nella porzione finale di A[p : r]
while i < n_l
    A[k] = L[i]
    i = i + 1
    k = k + 1
while j < n_r
    A[k] = R[j]
    j = j + 1
    k = k + 1
```

esempio:

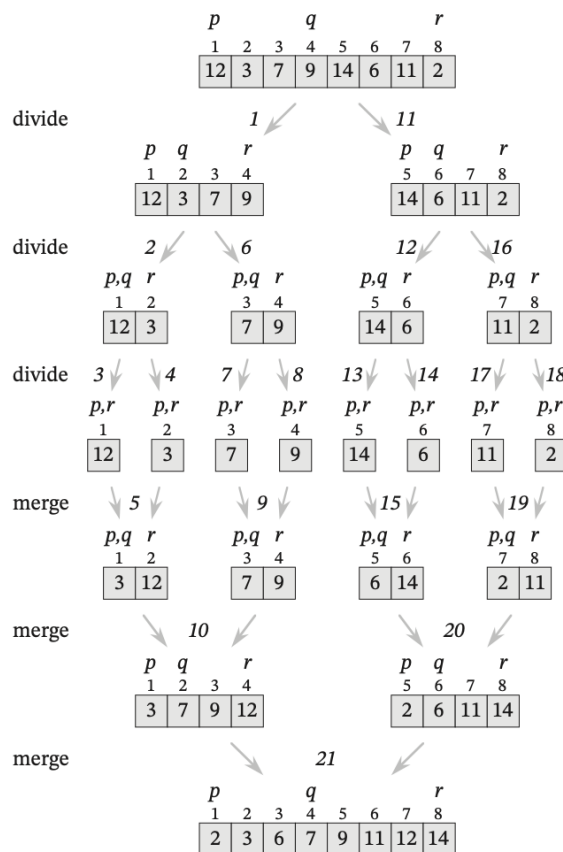


Figura 2.4

Funzionamento del merge sort su un array A di lunghezza 8 che inizialmente contiene la sequenza $\langle 12, 3, 7, 9, 14, 6, 11, 2 \rangle$. Gli indici p , q e r di ciascun sottoarray compaiono sopra i valori corrispondenti. I numeri in corsivo indicano l'ordine in cui vengono chiamate le procedure MERGE-SORT e MERGE dopo la chiamata iniziale $\text{MERGE-SORT}(A, 1, 8)$.

Analisi del MergeSort

Il MergeSort è un algoritmo efficiente la cui complessità temporale per l'ordinamento di un intero array di dimensione n è solamente $O(n \log(n))$.

L'algoritmo è conforme al paradigma Divide et Impera che viene sviluppato come segue:

1. **Divide:** divide la sequenza degli n elementi da ordinare in due sotto-sequenze di $n/2$ elementi ciascuna.
2. **Impera:** ordina le due sotto-sequenze in modo ricorsivo.
3. **Combina:** fonde le due sotto-sequenze ordinate per generare la sequenza ordinata finale.

Equazione di ricorrenza

$$\begin{cases} T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(n) & \text{passo ricorsivo} \\ T(1) = 1 & \text{passo base} \end{cases}$$

Figura 2.5

La costruzione di un albero di ricorsione per la ricorrenza

$$T(n) = 2T(n/2) + cn.$$

La parte (a) mostra $T(n)$,

che viene

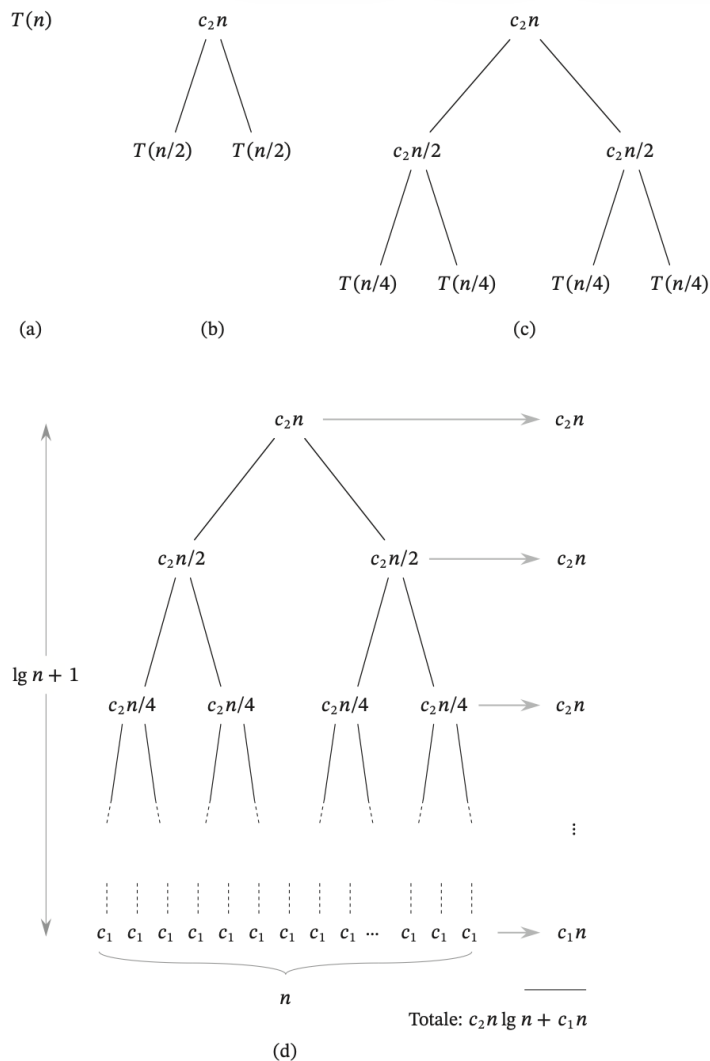
progressivamente espanso

in (b)-(d) per formare

l'albero di ricorsione.

L'albero completamente espanso nella parte (d) ha $\lg n + 1$ livelli. Ogni livello al di sopra di quello delle foglie contribuisce per un

costo complessivo di c_2n , mentre il livello delle foglie ha un contributo di c_1n . Di conseguenza, il costo totale è $c_2n \lg n + c_1n = \Theta(n \lg n)$.



2.5 Quick sort

Il quicksort è un algoritmo di ordinamento il cui tempo di esecuzione nel caso peggiore è $\Theta(n^2)$ su un array di input di n numeri.

Questi sono i tre passi del processo divide et impera per ordinare un generico sottoarray $A[p : r]$:

- **Divide**

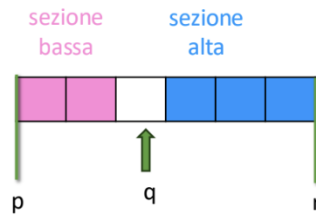
1. Fissiamo un pivot q
2. Riorganizziamo l'input in due sotto liste, $A[p : q - 1]$ (sezione bassa) e $A[q + 1 : r]$ (sezione alta)
3. Nella riorganizzazione dobbiamo garantire che ogni elemento della sezione bassa è $\leq$ di ogni elemento della sezione alta

- **Impera**

1. Ordiniamo le due sotto-liste in modo ricorsivo

- **Combina**

1. Poiché le due sotto liste sono già ordinate. Tutti gli elementi di $A[p : q - 1]$ sono in ordine e $\leq$ al pivot $A[q]$, e tutti quelli di $A[q + 1 : r]$ sono in ordine e $\geq$ al pivot $A[q]$. Ma allora l'intera lista $A[p : r]$ non può che essere ordinata.



Pseudocodice QuickSort

```

QUICKSORT(A,p,r)
1  if p < r
2      //Partiziona il sottoarray intorno al pivot, che finisce in A[q]
3      q = PARTIZIONA(A,p,r) //calcolo l'indice dell'elemento pivot con partiziona(A, p, r)
4      QUICKSORT(A,p,q-1) //ordina ricorsivamente la sezione bassa
5      QUICKSORT(A,q+1,r) //ordina ricorsivamente la sezione alta

```

Pseudocodice PARTIZIONA:

```

PARTIZIONA(A,p,r)
1  x = A[r] //il pivot
2  i = p - 1 //l'indice più alto nella sezione bassa
3  for j = p to r - 1 //tratta ogni elemento che non è il pivot
4      if A[j] <= x //questo elemento appartiene alla sezione bassa?
5          i = i + 1 //indice di nuova posizione della sezione bassa
6      scambia A[i] con A[j] //inserisce questo elemento nella nuova posizione
7  scambia A[i+1] con A[r] //il pivot si torva a destra della sezione bassa
8  return i + 1 //il nuovo indice del pivot

```

Esempio. Ordinare il seguente array:

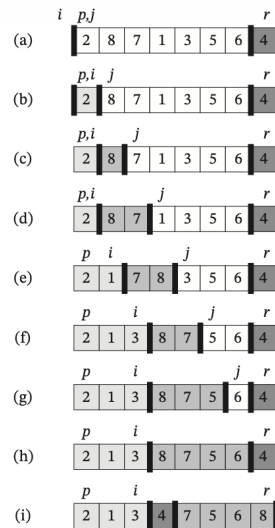


Figura 71 Il funzionamento di PARTITION su un array campione. L'elemento $A[r]$ dell'array diventa l'elemento pivot x . Gli elementi dell'array su sfondo grigio intermedio sono tutti nella sezione bassa e hanno valori non maggiori di x . Gli elementi su sfondo grigio scuro sono nella sezione alta e hanno valori maggiori di x . Gli elementi su sfondo bianco non sono stati ancora posti in una delle due sezioni; gli elementi su sfondo grigio scuro è il pivot x . (a) L'array iniziale e le impostazioni delle variabili. Nessuno degli elementi è stato posto in una delle due sezioni della partizione. (b) Il valore 2 viene "scambiato con se stesso" e posto nella sezione bassa. (c)-(d) I valori 8 e 7 vengono inseriti nella sezione alta. (e) I valori 1 e 8 vengono scambiati e la sezione bassa cresce. (f) I valori 3 e 7 vengono scambiati e la sezione bassa cresce. (g)-(h) La sezione alta della partizione acquisisce i valori 5 e 6 e il ciclo termina. (i) La riga 7 scambia il pivot in modo che cada tra le due sezioni della partizione e la riga 8 restituisce il nuovo indice del pivot.

All'inizio di ogni iterazione del ciclo for, per ogni indice k dell'array valgono le seguenti condizioni:

1. se $p \leq k \leq i$ allora $A[k] \leq x$
2. se $i + 1 \leq k \leq j - 1$ allora $A[k] \geq x$
3. se $k = r$ allora $A[k] = x$

Dimostrazione della correttezza di Quicksort

Dobbiamo dimostrare che questa invariante di ciclo è vera prima della prima iterazione, che ogni iterazione conserva l'invariante di ciclo, che il ciclo termina e, quando ciò avviene, che la correttezza segue dall'invariante.

Passo 1 - Inizializzazione

1. prima della prima iterazione del ciclo si ha $i = p - 1$ e $j = p$
2. poiché non ci sono valori fra p e i né fra $i + 1$ e $j - 1$, le prime due condizioni dell'invariante di ciclo sono banalmente soddisfatte.
3. l'assegnamento nella riga 1 soddisfa la terza condizione.

Passo 2 - Mantenimento

RECALL: Dobbiamo dimostrare che ogni iterazione mantiene l'invariante. Supponiamo quindi che sia vera prima della k -esima iterazione e mostriamo che resta vera al termine

Nella riga 11 di `partizione()` controllo se $A[k] \leq x$:

- CASO FALSE: non succede nulla e si passa alla prossima iterazione.
- CASO TRUE: viene incrementato l'indice i , vengono scambiati $A[i]$ e $A[j]$ e si passa alla prossima iterazione.

Passo 3 - Conclusione

1. il ciclo termina quando $j = r$
2. i valori dell'array si trovano ripartiti in tre insiemi: quelli minori o uguali a x , quelli maggiori di x e un insieme di un solo elemento che contiene x (il pivot).
3. viene eseguita la linea 6 del codice `PARTIZIONA(A, p, r)` e si scambiano $A[i + 1]$ e $A[r]$. L'output di `PARTIZIONA` adesso soddisfa le specifiche del passo divide. Infatti dopo la riga 3 di `QUICKSORT` $A[q]$ è strettamente minore di ogni elemento in $A[q + 1 : r]$.

Prestazioni QuickSort

il tempo di esecuzione del quicksort dipende da quanto è bilanciata ogni partizione e questo, a sua volta, dipende da quali elementi vengono usati come pivot. Se le due sezioni della partizione hanno all'incirca la stessa dimensione, cioè se è bilanciato, allora l'algoritmo ha un costo $O(n \log n)$. Se il partizionamento è sbilanciato ha un costo $O(n^2)$. Il costo in memoria dell'algoritmo invece è $O(n)$ in quanto non viene allocato nuovo spazio in memoria per i sotto-array, bensì si richiama ricorsivamente sullo stesso.

Il caso peggiore del QuickSort è quando l'array da ordinare è già ordinato o è ordinato in ordine inverso, o quando tutti gli elementi sono uguali.

Il QuickSort è di per sé un algoritmo efficiente, però può essere migliorato abbassando la probabilità di incontrare dei casi sfavorevoli, quindi evitando di incontrare casi di sbilanciamento nei due sotto-array. Per fare questo si utilizza una versione randomizzata del QuickSort, in cui la scelta del pivot avviene in modo pseudo-casuale.

Tempo di esecuzione QuickSort

$$T(n) = \begin{cases} O(1) & \text{se } n \leq 1 \text{ passo base} \\ T(k) + T(n - k - 1) + O(n) & \text{se } n > 1 \text{ passo ricorsivo} \end{cases}$$

Dove $T(k)$ rappresenta il tempo di esecuzione del QuickSort per il sotto array con k elementi, $T(n - k - 1)$ rappresenta il tempo di esecuzione del QuickSort per il sotto array con $n - k - 1$ che non includono il pivot scelto, $O(n)$ rappresenta il tempo necessario per effettuare la partizione dell'array in base al pivot, che richiede un tempo lineare proporzionale alla dimensione dell'array. Il passo base dell'equazione di ricorrenza del QuickSort è quando l'array da ordinare contiene zero o un solo elemento, in cui il tempo di esecuzione è costante.

2.6 Ricorrenze

Per analizzare gli algoritmi ricorsivi basati sul metodo divide et impera, abbiamo bisogno di alcuni strumenti matematici. Una **ricorrenza** è un'equazione che descrive una funzione in termini del suo valore su altri argomenti, in genere più piccoli. Le ricorrenze vengono usate con il metodo divide et impera perché ci danno un modo naturale per caratterizzare matematicamente i tempi di esecuzione degli algoritmi ricorsivi.

La forma generale di una ricorrenza è un'equazione o una disequazione che descrive una funzione definita sugli interi o sui reali utilizzando la funzione stessa. Essa contiene due o più casi, a seconda dell'argomento. Se un caso comporta la chiamata ricorsiva della funzione su input diversi, si tratta di un **caso ricorsivo**. Se un caso non comporta una chiamata ricorsiva, è un **caso base**.

Ricorrenze algoritmiche

Una ricorrenza $T(n)$ è di natura algoritmica se, per ogni soglia $n_0 > 0$ sufficientemente grande valgono le seguenti proprietà:

1. per ogni $n < n_0$, si ha $T(n) = \Theta(1)$
2. per ogni $n \geq n_0$, ogni cammino di ricorsione termina in un caso base entro un numero finito di chiamate ricorsive.

2.7 Impostazione di una ricorrenza

Tutte le volte che abbiamo un algoritmo ricorsivo, bisogna impostare una **ricorrenza**.

Esempio.

```
int ricercaBinariaRic(int a[], int primo, int ultimo, int k)
```

Vogliamo cercare una funzione $T(n)$ che descriva il costo della funzione, il nostro esempio ha questa ricorrenza:

$$\begin{cases} T(n) = C + T\left(\frac{n}{2}\right) & n > 1 \\ T(1) = 1 & \rightarrow \text{caso base} \end{cases}$$

Procediamo con uno sviluppo manuale per sostituzione, che chiamiamo per **Srotolamento**:

Dopo aver impostato la ricorrenza, applichiamo la manualmente la ricorsione:

$$n \rightarrow \frac{n}{2} : \quad T\left(\frac{n}{2}\right) = 1 + T\left(\frac{n}{2^2}\right)$$

$$\frac{n}{2} \rightarrow \frac{n}{4} : \quad T\left(\frac{n}{4}\right) = 1 + T\left(\frac{n}{2^3}\right)$$

Quindi riscriviamo l'espressione di $T(n)$ con le sostituzioni che abbiamo fatto sopra:

$$T(n) = 1 + \underbrace{1 + T\left(\frac{n}{2^2}\right)}_{T\left(\frac{n}{2}\right)} = 1 + 1 + \underbrace{1 + T\left(\frac{n}{2^3}\right)}_{T\left(\frac{n}{2^2}\right)} = \dots = \boxed{K + T\frac{n}{2^k}}$$

Visto che il caso base è $T(1) = 1$, dobbiamo vedere per quale valore di k si ha che $\frac{n}{2^k} = 1$. Ciò avviene per $k = \log n$, quindi riscriviamo l'espressione di $T(n)$ con questo valore di k :

$$T(n) = \log n + 1 \in \Theta(\log n)$$

Questo esempio è la dimostrazione di un algoritmo **In-Place**.

Negli algoritmi "in-place" si usa la dimensione dell'input più qualche variabile. Non tutti gli algoritmi possono essere portati "in-place" come ad esempio merge-sort.

Esempio.

```
MERGE-SORT(A,p,r)
if p >= r           //zero o un solo elemento?
    return
q=(p+r)/2           //punto medio di A[p : r]
MERGE-SORT(A,p,q)   //ordina ricorsivamente A[p : q]
MERGE-SORT(A,q+1,r) //ordina ricorsivamente A[q+1 : r]
//fonde A[p : q] con A[q+1 : r] in A[p : r]
MERGE(A,p,q,r)
```

Questo algoritmo ha questo schema di ricorrenza:

$$\begin{cases} T(n) = 2T\left(\frac{n}{2}\right) + M(n) \\ T(1) = 1 \rightarrow \text{caso base} \end{cases}$$

Dove $M(n)$ è il costo della chiamata del $\text{MERGE}(A, p, q, r)$, che ancora non conosciamo.

Per vedere quanto vale il costo guardiamo il suo pseudocodice:

```

MERGE(A, p, q, r)
nL = q - p + 1          //lunghezza di A[p : q]
nR = r - q              //lunghezza di A[q+1 : r]
//crea due nuovi array
L = [0 : nL - 1]
R = [0 : nR - 1]

for i = 0 to nL - 1      //copia A[p : q] in L[0 : nL - 1]
    L[i] = A[p + i]

for j = 0 to nR - 1      //copia di A[q+1 : r] in R[0 : nR - 1]
    R[j] = A[q + j + 1]
i = 0                    //i è l'indice del più piccolo elemento che resta in L
j = 0                    //j è l'indice del più piccolo elemento che resta in R
k = p                    //k è la posizione di A in cui fare il prossimo inserimento

//unisci i due sottoarray in ordine
while i < nL and j < nR
    if L[i] <= R[j]
        A[k] = L[i]
        i = i + 1
    else A[k] = R[j]
        j = j + 1
    k = k + 1

//avendo esaurito completamente uno tra L e R,
//copia il resto dell'altro nella porzione finale di A[p : r]
while i < nL
    A[k] = L[i]
    i = i + 1
    k = k + 1
while j < nR
    A[k] = R[j]
    j = j + 1
    k = k + 1

```

Dallo pseudocodice si evince che il costo di $M(n)$ è n , poichè quando vengono copiati gli array in due buffer temporanei, si ha un costo di $\Theta(n)$, poi quando vengono effettuati gli ultimi due cicli per svuotare i buffer, per ogni elemento "spendiamo" una costante, quindi anche in questa fase il costo è di $\Theta(n)$.

Possiamo quindi riscrivere lo schema di ricorrenza con $M(n) = n$:

$$\begin{cases} T(n) = 2T\left(\frac{n}{2}\right) + n \\ T(1) = 1 \end{cases}$$

Si procede con lo srotolamento:

$$\begin{aligned} n \rightarrow \frac{n}{2} : \quad T\left(\frac{n}{2}\right) &= 2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \\ \frac{n}{2} \rightarrow \frac{n}{4} : \quad T\left(\frac{n}{2^2}\right) &= 2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \end{aligned}$$

Quindi $T(n)$ diventa:

$$T(n) = 2T\left(\frac{n}{2}\right) + n = 2 \underbrace{\left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \right]}_{T\left(\frac{n}{2}\right)} + n = \dots = \boxed{2^k T\left(\frac{n}{2^k}\right) + k n}$$

Dato il caso base $T(1) = 1$, dobbiamo trovare il valore di k tale che $2^k = 1$, ovvero per $k = \log n$, perciò $T(n)$ diventa:

$$T(n) = 2^{\log n} T(1) + \log n \cdot n = n + n \log n \in \Theta(n \log n)$$

Esempio.

Si ha la seguente ricorrenza:

$$\begin{cases} T(n) = T(n-1) + n \\ T(1) = 1 \end{cases} \rightarrow \text{caso base}$$

Applichiamo lo srotolamento:

$$n \rightarrow n-1 : \quad T(n-1) = T(n-2) + n-1$$

$$n-1 \rightarrow n-2 : \quad T(n-2) = T(n-3) + n-2$$

Sostituendo a $T(n)$ diventa:

$$T(n) = T(n-3) + (n-2) + (n-1) + n = \dots = \boxed{T(n-k) + \sum_{i=0}^{k-1} (n-i)}$$

Semplifichiamo la sommatoria:

$$\sum_i^{k-1} (n-i) = \sum_i^{k-1} n - \sum_i^{k-1} i = k \cdot n - \frac{(k-1)(k-2)}{2}$$

dove per la seconda sommatoria abbiamo usato il criterio di Gauss.

Ora per il caso base $T(1) = 1$, troviamo che $n-k=1$ quando $k=n-1$, sostituendo si ottiene:

$$T(n) = (n-1) \cdot n - \frac{(n-2)(n-3)}{2} = 1 + \frac{2n^2 - 2n - n^2 - 3n - 2n - 6}{2} \in \Theta(n^2)$$

Esempio.

Si ha la seguente ricorrenza:

$$\begin{cases} T(n) = T\left(\frac{n}{2}\right) + \frac{n}{2} \\ T(1) = 1 \end{cases} \rightarrow \text{caso base}$$

Applichiamo lo srotolamento:

$$n \rightarrow \frac{n}{2} : \quad T\left(\frac{n}{2}\right) = T\left(\frac{n}{2^2}\right) + \frac{n}{2^2}$$

$$\frac{n}{2} \rightarrow \frac{n}{4} : \quad T\left(\frac{n}{2^2}\right) = T\left(\frac{n}{2^3}\right) + \frac{n}{2^3}$$

Sostituendo a $T(n)$ diventa:

$$T(n) = \underbrace{T\left(\frac{n}{2^2}\right) + \frac{n}{2^2} + \frac{n}{2}}_{T\left(\frac{n}{2}\right)} = \dots = \boxed{T\left(\frac{n}{2^k}\right) + n \sum_{i=1}^k \left(\frac{1}{2}\right)^i}$$

Per il caso base $\frac{n}{2^k} = 1$ per $k = \log n$, inoltre la sommatoria è una successione geometrica di ragione $q = \frac{1}{2}$, quindi la sua somma è nota. Sostituiamo e semplifichiamo:

$$T(n) = 1 + n \frac{q^{k+1} - 1}{q - 1} = 1 + 2n \left(1 - \frac{1}{2^{\log n}}\right) = 1 + 2n \left(1 - \frac{1}{n}\right)$$

2.8 Alberi di ricorsione per le ricorrenze

Oltre al metodo di sostituzione, un altro modo per analizzare il costo di un algoritmo è usare gli **alberi di ricorsione**. In questa struttura, ogni nodo rappresenta il costo di un singolo sotto-problema di una delle chiamate ricorsive della funzione. Questo metodo consiste nel sommare i costi dei nodi che si trovano sullo stesso livello per ottenere il costo per livello. Poi sommeremo il costo di ogni livello per determinare il costo totale dell'algoritmo ricorsivo.

Esempio.

Prendiamo in esame l'algoritmo Quicksort, che ha il seguente pseudo-codice:

```

1  QUICKSORT(A,p,r)
2  if p < r
3      q = PARTIZIONA(A,p,r)  //calcolo l'indice dell'elemento pivot con partiziona(A, p, r)
4      QUICKSORT(A,p,q-1)    //ordina ricorsivamente la sezione bassa
5      QUICKSORT(A,q+1,r)    //ordina ricorsivamente la sezione alta
6  end
7  PARTIZIONA(A,p,r)
8  x = A[r]                  //il pivot
9  i = p - 1                 //l'indice più alto nella sezione bassa
10 foreach j in [p : r-1] do //tratta ogni elemento che non è il pivot
11     if A[j] <= x then      //questo elemento appartiene alla sezione bassa?
12         i = i + 1          //indice di nuova posizione della sezione bassa
13         scambia A[i] con A[j] //inserisce questo elemento nella nuova posizione
14     end
15 end
16 scambia A[i+1] con A[r]    //il pivot si torva a destra della sezione bassa
17 return i + 1              //il nuovo indice del pivot

```

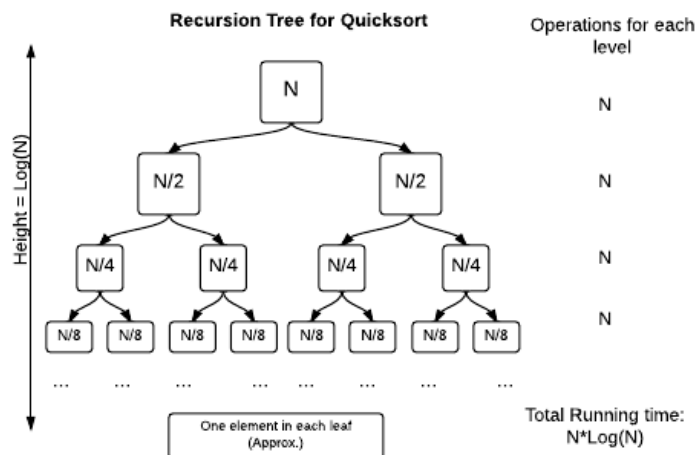
Impostiamo la ricorrenza, che terrà della chiamata ricorsiva e della chiamata alla funzione PARTIZIONA(A,p,r):

$$T(n) = 2T(n-1) + P(n)$$

Vediamo prima il costo di PARTIZIONA(A,p,r), dallo pseudo-codice si evince che sia:

$$P(n) \in \Theta(n) \longrightarrow P(n) = c \cdot n$$

Vediamo prima cosa succede nel **caso migliore**, ovvero nel caso in cui scegliamo il pivot che divide perfettamente a metà l'array da ordinare. Avremo perciò il seguente albero di ricorsione.



Quindi si ha che:

- **Radice:** Nel primo nodo più alto c'è il problema iniziale: ordinare un array di n elementi
- **Partizione:** l'algoritmo sceglie un pivot e divide l'array in due sotto-array di $\frac{n}{2}$ elementi:
 - elementi minori del pivot
 - elementi maggiori del pivot
- **Livelli successivi:** ogni sotto-array viene ordinato con un'altra chiamata a Quicksort, quindi l'albero continua:
 - livello 0 $\rightarrow n$
 - livello 1 $\rightarrow \frac{n}{2}$
 - livello 2 $\rightarrow \frac{n}{4}$
 - livello 3 $\rightarrow \frac{n}{8}$
- **Foglie dell'albero:** L'algoritmo si ferma quando i sotto-array hanno 1 elemento

Una volta impostato l'albero di ricorsione basterà quindi sommare i costi di ogni livello e successivamente sommare il costo di ogni livello:

→ Livello 0: c'è un unico nodo di costo N

→ Livello 1: ci sono due nodi di costo $\frac{N}{2}$, quindi il costo del livello è $2 \cdot \left(\frac{N}{2}\right) = N$

→ Livello 2: ci sono quattro nodi di costo $\frac{N}{4}$, quindi il costo del livello è $4 \cdot \left(\frac{N}{4}\right) = N$

⋮

→ Livello k : ci sono k nodi di costo $\frac{N}{k}$, quindi il costo del livello è $k \cdot \left(\frac{N}{k}\right) = N$

Sommando i costi di ogni livello si ottiene che il costo dell'algoritmo nel caso migliore è:

$$T(n) = N \cdot k$$

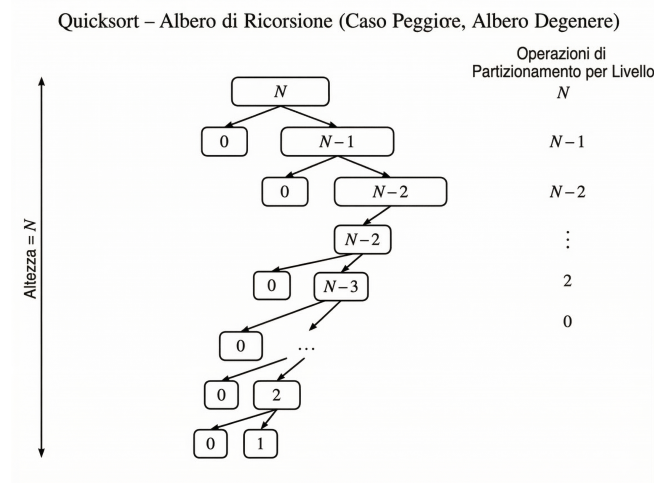
Ma quanto vale k ?

Per capire quanto vale k (ovvero quanto è alto l'albero della ricorsione) ci dobbiamo chiedere quante volte il valore N può essere diviso per due fino a raggiungere 1. Ciò equivale a dire a quale esponente si deve elevare 2 per ottenere N , e questa è proprio la definizione di logaritmo in base 2 di N .

Quindi il costo totale dell'algoritmo nel caso migliore è:

$$T(n) = N \cdot \log_2 N$$

Ora vediamo il **caso peggiore**: ciò succede quando o la lista è già ordinata, oppure quando viene scelto un pivot che sia il primo o l'ultimo elemento dell'array. Infatti l'albero di ricorsione sarà degenerare (ogni nodo ha al massimo un figlio), poichè l'array viene suddiviso in un sotto-array di dimensione $n - 1$, e un sotto-array di 1 elemento. L'albero di ricorsione perciò sarà così:



In questo caso il costo totale è evidente, in quanto l'albero di ricorsione è alto N e ad ogni livello il costo viene diminuito di 1:

$$T(n) = N \cdot \sum_{i=0}^k (N - i) \rightarrow \boxed{T(n) \in O(n^2)}$$

Il **caso medio** invece, tiene conto della probabilità con cui viene scelto il pivot. Infatti c'è sempre la probabilità del 50% di scegliere un pivot "buono" e il restante 50% di scegliere un pivot "cattivo". Nel caso medio non ci troviamo nè nel caso migliore nè nel caso peggiore, quindi l'altezza dell'albero di ricorsione sarà sempre $N \cdot \log N$, ciò che cambia rispetto al caso migliore è che la base del logaritmo non è più 2, ma dipenderà dalla scelta dei pivot. In generale, comunque, possiamo affermare che il costo nel caso medio è:

$$\boxed{T(n) = N \cdot \log N}$$

2.9 Alberi di decisione

Definizione 2.9.1. (Albero di decisione) Un albero di decisione è un albero pieno (cioè ogni nodo è una foglia oppure ha entrambi i figli), che rappresenta i confronti fra elementi che vengono effettuati da un algoritmo che agisce su un input di una data dimensione.

Osservazione. Gli alberi di decisione non sono strutture dati.

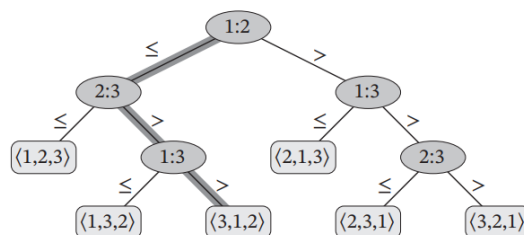
L'esecuzione dell'algoritmo di ordinamento corrisponde a tracciare un cammino semplice a partire dalla radice dell'albero di decisione fino ad arrivare ad una foglia. Ogni nodo rappresenta un confronto $a_i \leq a_j$. Il sotto-albero sinistro rappresenta i successivi confronti da fare se la condizione $a_i \leq a_j$ risulta vera, mentre il sotto-albero destro è dedicato al caso in cui $a_i \leq a_j$ non è verificata. Raggiunta una foglia l'algoritmo ha definito un certo ordinamento $a_1 \leq a_2 \leq \dots \leq a_n$.

Siccome qualsiasi algoritmo di ordinamento corretto deve essere in grado di produrre ogni permutazione del suo input, allora una **condizione necessaria** affinché un ordinamento per confronti sia corretto è che ciascuna delle $n!$ permutazioni di n elementi appaia almeno una volta in una delle foglie dell'albero di decisione. Inoltre ciascuna di queste foglie deve essere raggiungibile dalla radice, attraverso un percorso che corrisponde a una effettiva esecuzione dell'ordinamento per confronti.

Esempio.

Vogliamo ordinare un'array di 3 elementi tramite l'algoritmo di Insertion Sort.

L'albero di decisione per questo algoritmo è il seguente:



Osservazione. I numeri che sono nei nodi non sono i numeri di con cui sta avvenendo il confronto, ma sono gli indici dell'array di 3 elementi.

Una volta impostato l'albero di decisione verifichiamo la condizione necessaria, quindi controlliamo che nelle foglie ci siano tutte le possibili permutazioni dell'input: la dimensione dell'array è $n = 3$, quindi ci aspettiamo che nelle foglie ci siano tutte le $n! = 3! = 6$ combinazioni dell'array. Si nota che le 6 permutazioni sono presenti nelle foglie e che sono tutte raggiungibili a partire dalla radice, quindi la condizione necessaria è soddisfatta. Ovviamente il percorso dalla radice alla foglia più corto rappresenta il caso migliore dell'algoritmo, analogamente il percorso più lungo (che corrisponde all'altezza dell'albero) rappresenta il caso peggiore.

Teorema su alberi di decisione

Teorema 2.9.1. Dato un albero di decisione, sia

- N : il numero di nodi
- F : il numero di foglie

allora

$$N = 2F - 1$$

Esempio.

Prendiamo in considerazione l'esempio di prima, l'albero di decisione ha 6 foglie, quindi applicando la formula del teorema il numero di nodi è:

$$N = 2F - 1 = 2 \cdot 6 - 1 = \boxed{11}$$

Teorema su algoritmi di ordinamento

Teorema 2.9.2. Ogni algoritmo di ordinamento per confronti richiede $\Omega(n \log n)$ confronti nel caso peggiore.

Dimostrazione. Per stabilire il numero di confronti nel caso peggiore, è sufficiente sapere quanto è alto l'albero di decisione. Consideriamo un albero di decisione di un algoritmo che ha in input n elementi e poniamo:

- H : valore dell'altezza dell'albero
- L : numero di foglie

Poichè ciascuna delle $n!$ permutazione deve comparire almeno in una foglia (condizione necessaria), si ha che

$$n! \leq L$$

Inoltre sappiamo che un albero binario di altezza H non ha più di 2^H foglie, si ha che

$$n! \leq L \leq 2^H$$

Considero la disuguaglianza $n! \leq 2^H$ e facendo il logaritmo in base 2 da tutte e due le parti si ottiene:

$$H \geq \lg(n!) \xrightarrow{\text{Approssimazione di Stirling}} \boxed{H = \Omega(n \log n)}$$

□

Sorting non basato sul confronto

2.10 Counting Sort

L'algoritmo Counting Sort suppone che ciascuno degli n elementi di input sia un numero intero compreso tra 0 e k , per qualche intero k . Il Counting Sort per prima cosa determina, per ogni elemento di input x , il numero di elementi minori o uguali a x , e usa questa informazione per inserire l'elemento x direttamente nella sua posizione nell'array di output. Ecco lo pseudo-codice del Counting-Sort:

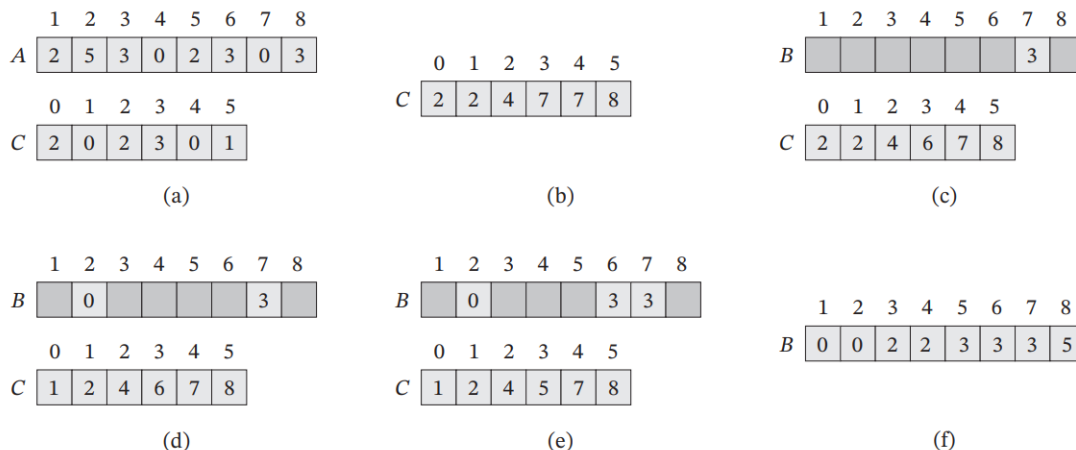
COUNTING-SORT(A, n, k)

```
1  siano B[1 : n] e C[0 : k] due nuovi array
2  for i = 0 to k
3      C[i] = 0
4  for j = 1 to n
5      C[A[j]] = C[A[j]] + 1
6  // Ora in C[i] c'è il numero di elementi uguali a i.
7  for i = 1 to k
8      C[i] = C[i] + C[i - 1]
9  // Ora in C[i] c'è il numero di elementi minori o uguali a i.
10 // Copia A in B partendo dal fondo di A.
11 for j = n downto 1
12     B[C[A[j]]] = A[j]
13     C[A[j]] = C[A[j]] - 1    // Per gestire valori duplicati
14 return B
```

L'algoritmo prende in input un'array $A[1 : n]$, la dimensione n dell'array e una soglia superiore k sui valori in A (in altre parole k è il valore massimo negli elementi dell'array), che sono interi non negativi. Restituisce l'array $B[1 : n]$ che contiene l'output ordinato e usa un array $C[0 : k]$ come memoria temporanea di lavoro.

Esempio.

Applichiamo l'algoritmo sull'array $A = [2 \ 5 \ 3 \ 0 \ 2 \ 3 \ 0 \ 3]$, il numero massimo è 5 quindi $k = 5$:



Il funzionamento della procedura Counting Sort su un array di input $A[1 : 8]$, dove ogni elemento di A è un intero non negativo non maggiore di $k = 5$.

(a) l'array A e l'array ausiliario C dopo la riga 5.

(b) l'array C dopo la riga 8

(c)-(e) l'array di output B e l'array ausiliario C dopo, rispettivamente, una, due e tre iterazioni del ciclo (righe 11-13) le caselle di colore grigio chiaro nell'array B rappresentano gli elementi che sono stati inseriti.

(f) l'array di output B è ordinato.

2.11 Bucket sort

Come il Counting Sort, il Bucket Sort è veloce perchè fa delle assunzioni sull'input. Mentre il Counting Sort suppone che l'input sia formato da interi appartenenti a un intervallo, il bucket sort suppone che l'input sia generato da un processo casuale che distribuisce gli elementi uniformemente e indipendentemente nell'intervallo $[0, 1)$.

Ciò vuol dire che l'input di array $A[1 : n]$ deve rispettare la condizione che ogni elemento $A[i]$ dell'array deve essere tale che:

$$0 \leq A[i] < 1$$

Il Bucket Sort divide l'intervallo $[0, 1)$ in n sottointervalli della stessa dimensione, detti **bucket** (cesti), e poi distribuisce gli n numeri di input nei bucket. Per produrre l'output, ordiniamo i numeri in ogni bucket (attraverso un altro algoritmo di ordinamento), che scorriamo ordinatamente, elencando gli elementi in ciascuno di essi.

Pseudocodice BucketSort

BUCKET-SORT(A, n)

1 sia $B[0 : n - 1]$ un nuovo array

2 for $i = 0$ to $n - 1$

3 crea la lista vuota $B[i]$

4 for $i = 1$ to n

5 inserisci $A[i]$ nella lista $B[n \cdot A[i]]$

6 for $i = 0$ to $n - 1$

7 ordina la lista $B[i]$ con l'insertion sort

8 concatena ordinatamente le liste $B[0], B[1], \dots, B[n - 1]$

9 return la lista concatenata

La supposizione che $0 \leq A[i] < 1$ però può essere limitante per problemi reali dove sono presenti numeri più grandi. Quindi, quando abbiamo valori che sono maggiori di 1, possiamo comunque usare il Bucket Sort ma bisogna **normalizzare** l'elemento dell'array prima di metterlo nei bucket. La normalizzazione avviene così:

$$\text{Valore normalizzato} = \frac{\text{Valore attuale} - \text{Valore minimo}}{\text{Valore massimo} - \text{Valore minimo}}$$

Ad esempio, se stiamo ordinando un array di valori tra 10 e 100 e stiamo analizzando il l'elemento 55, il valore normalizzato sarà:

$$\text{Valore normalizzato} = \frac{55 - 10}{100 - 10} = 0,5$$

E in base a cosa organizziamo gli elementi dell'input nei bucket?

Gli elementi vengono organizzati nei bucket calcolando in quale sottointervallo ricade il loro valore. Il numero totale di bucket da creare si sceglie spesso in base alla dimensione dell'input (n), ma la regola fondamentale è che tutti i bucket devono coprire un intervallo di valori della stessa identica ampiezza.

Infatti il Bucket Sort è particolarmente efficace quando i dati sono distribuiti in modo uniforme su un intervallo.

Riassumendo, gli step principali del Bucket Sort sono questi:

1. **Distribuzione:** gli elementi vengono distribuiti in diversi bucket in base al loro valore.
2. **Ordinamento locale:** ogni bucket viene ordinato separatamente (con un altro algoritmo di ordinamento).
3. **Concatenazione:** i bucket ordinati vengono concatenati per ottenere l'output finale.

Analisi Bucket Sort

Sia n la dimensione dell'input e k il costo per produrre l'output.

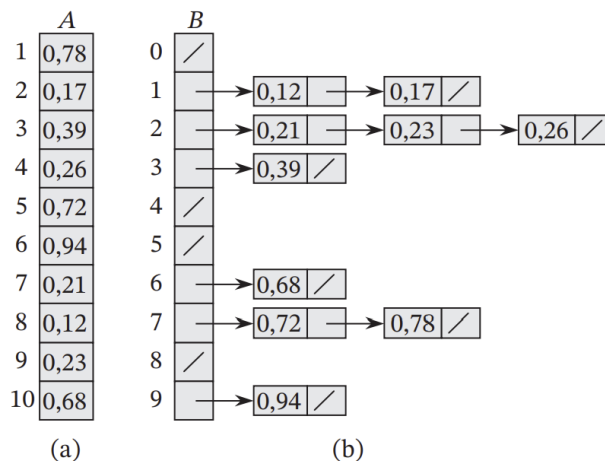
In base al caso in cui ci troviamo avremo costi temporali e spaziali differenti:

- **Caso peggiore:** avviene quando tutti gli elementi dell'input vengono messi nello stesso bucket.
 - Costo temporale: $O(n^2)$
 - Costo spaziale: $O(n + k)$
- **Caso medio:** avviene quando gli elementi dell'input si distribuiscono in maniera non perfettamente uniforme nei bucket.
 - Costo temporale: $O(n)$
 - Costo spaziale: $O(n + k)$
- **Caso ottimale:** avviene quando gli elementi dell'input vengono distribuiti in maniera perfettamente uniforme nei bucket, ovvero ogni bucket ha lo stesso numero di elementi.
 - Costo temporale: $O(n)$
 - Costo spaziale: $O(n + k)$

Esempio.

Ecco un esempio facendo riferimento allo pseudocodice di Bucket Sort.

Si vuole ordinare l'array $A = [0.78 \ 0.17 \ 0.39 \ 0.26 \ 0.72 \ 0.94 \ 0.21 \ 0.12 \ 0.23 \ 0.68]$



Il funzionamento di Bucket Sort con $n = 10$ e tutti gli elementi compresi tra 0 e 1, quindi non c'è bisogno della normalizzazione.

- (a) L'array di input $A[1 : 10]$
- (b) L'array $B[0 : 9]$ formato da liste ordinate (i bucket) dopo l'esecuzione della riga 7 dell'algoritmo, le barre "/" indicano la fine di ciascun bucket. L' i -esimo bucket contiene i valori che cadono nell'intervallo semiaperto $\left[\frac{i}{10}, \frac{(i+1)}{10}\right)$. L'output ordinato è formato dalla concatenazione ordinata delle liste $B[0], B[1], \dots, B[9]$.

2.12 Radix Sort LSD

Il Radix Sort LSD (Least Significant Decimal) è un algoritmo di ordinamento per valori numerici interi. Questo algoritmo risolve il problema dell'ordinamento dei numeri in maniera controintuitiva, ordinando prima i numeri in base alla cifra meno significativa. Concretamente, vogliamo ordinare dei numeri aventi d cifre, mettiamo in colonna i numeri e prendiamo in considerazione la colonna delle cifre meno significative (quindi la colonna tutta a destra) e ordiniamo i numeri a seconda dell'ordinamento delle cifre in quella colonna.

Vediamo un esempio numerico:

Esempio.

Dobbiamo ordinare i seguenti numeri: 329 457 657 839 436 720 355

329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839

Il codice per il Radix Sort è semplice, l'algoritmo suppone che ogni elemento nell'array $A[1 : n]$ abbia d cifre, dove la cifra 1 è quella di ordine più basso e la cifra d è quella di ordine più alto. Ecco il codice:

```
for i = 1 to d
    usa un ordinamento stabile per ordinare l'array A[1:n]
    rispetto alla cifra i
```

È essenziale che gli ordinamenti delle cifre in questo algoritmo siano **stabili**.

Ma cosa vuol dire che un algoritmo di ordinamento deve essere stabile?

Definizione 2.12.1. (Algoritmo stabile) Un algoritmo di ordinamento è stabile se preserva le relazioni delle posizioni fornite come input.

Per capire meglio facciamo un esempio:

Esempio.

Devo ordinare i seguenti numeri: 3 1* 2 1

Nell'array ci sono due 1, quindi se l'algoritmo è stabile allora viene preservato l'ordine degli 1 e l'output sarà:

1* 1 2 3

Altrimenti l'algoritmo è considerato non stabile.

2.13 Riduzione (fra problemi)

Definizione 2.13.1. (Riduzione fra problemi) Nel campo degli algoritmi, ridurre un Problema A a un Problema B significa trasformare un'istanza del Problema A in un'istanza del Problema B , in modo da poter usare un algoritmo che risolve B per risolvere anche A .

2.14 HeapSort

Definizione Heap

Un Heap è una struttura dati, basata su alberi binari, ovvero alberi che possono avere al massimo due figli. Queste strutture possono essere rappresentate mediante array i cui elementi sono i nodi dell'albero.

proprietà

- ogni elemento dell'albero corrisponde ad un elemento dell'array
- tutti i livelli dell'albero sono completamente riempiti salvo l'ultimo che può essere parzialmente riempito da sinistra fino ad un certo punto.

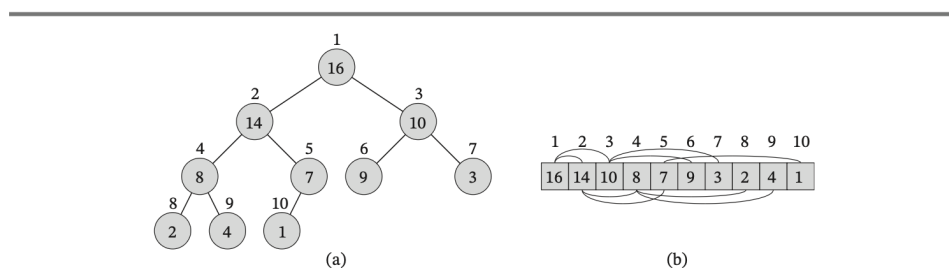


Figura 6.1 Un max-heap visto come (a) un albero binario e (b) un array. Il numero all'interno del cerchio di un nodo dell'albero è il valore memorizzato in quel nodo. Il numero sopra un nodo è il corrispondente indice dell'array. Sopra e sotto l'array ci sono delle linee che rappresentano le relazioni padre-figlio; i padri sono sempre a sinistra dei loro figli. L'albero ha altezza 3; il nodo, con indice 4 e valore 8, ha altezza 1.

Un heap è rappresentato da un oggetto che contiene

- un array di elementi $A[1..n]$
- un intero $size$ (tale che $0 \leq size \leq n$)

Un array A che rappresenta un heap ha due attributi specifici: **A.length** che indica il numero di elementi dell'array e **A.heapSize** che indica il numero di elementi dell'array che appartengono all'heap. In generale si ha che $0 \leq A.heapSize \leq A.length$.

Per ogni elemento i dell'array, ad eccezione della radice che è l'elemento $A[1]$, è possibile determinare il padre e i suoi due figli, in particolare:

$$PARENT(i) = \lfloor \frac{i}{2} \rfloor$$

$$LEFT(i) = 2i$$

$$RIGHT(i) = 2i+1$$

Max-heap e Min-heap

Esistono due tipi particolari di heap chiamati max-heap e min-heap.

Un **max-heap** è un heap in cui, per ogni nodo i diverso dalla radice, abbiamo che $A[PARENT(i)] \geq A[i]$, ovvero il padre è maggiore del figlio.

Un **min-heap** è un heap in cui, per ogni nodo i diverso dalla radice, abbiamo che $A[PARENT(i)] \leq A[i]$, ovvero i figli sono maggiori del padre.

Osservazione

In un max-heap la radice rappresenta il massimo.

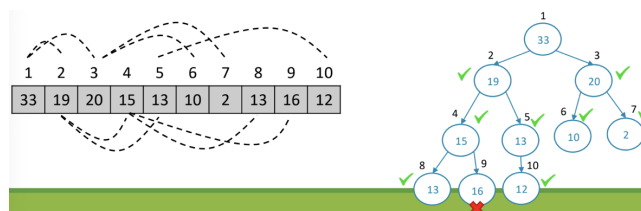
In un min-heap la radice rappresenta il minimo.

Esempio:

Dato l'array

33	19	20	15	13	10	2	13	16	12
----	----	----	----	----	----	---	----	----	----

determinare il max-heap:



Max-Heapify

L'heap memorizza elementi e può essere modificato. Per non perdere proprietà durante le modifiche della strutture dati utilizziamo l'algoritmo MAX-HEAPIFY(A, i) che si occupa di costruire un max-heap con radice nel nodo i .

pseudocodice MAX-HEAPIFY(A, i)

```

MAX-HEAPIFY( $A, i$ )
1   $l = \text{left}(i)$ 
2   $r = \text{right}(i)$ 
3  if  $i \leq A.\text{heapSize}$  AND  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{shapeSize}$  AND  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      scambia  $A[i]$  con  $A[\text{largest}]$ 
10 MAX-HEAPIFY( $A, \text{largest}$ )

```

Funzionamento

Quando viene chiamata questa funzione, si assume che i sottoalberi con radici $\text{left}(i)$ e $\text{right}(i)$ siano max-heap, mentre non è certo che il padre i sia effettivamente più grande dei figli. Vengono salvati gli indici del figlio sinistro e del figlio destro, dopodiché si procede a verificare innanzitutto se questi indici sono nei limiti dell'array e se viene rispettata la condizione $A[i] > A[l]$ e $A[r]$, se così non fosse verrebbe salvato nella variabile **largest** l'indice del figlio maggiore e verrebbero scambiati $A[\text{largest}]$ e il padre.

Così facendo la condizione di un max-heap sarebbe verificata per la terna appena considerata, tuttavia potrebbe non essere verificata nel sottoalbero che ha come radice il nodo appena scambiato, allora si procede a richiamare ricorsivamente la funzione sull'indice di largest, ovvero il nodo appena scambiato. Dunque, in breve, si fa scendere il valore $A[i]$ nel max-heap in modo che il sottoalbero con radice di indice i diventi un max-heap.

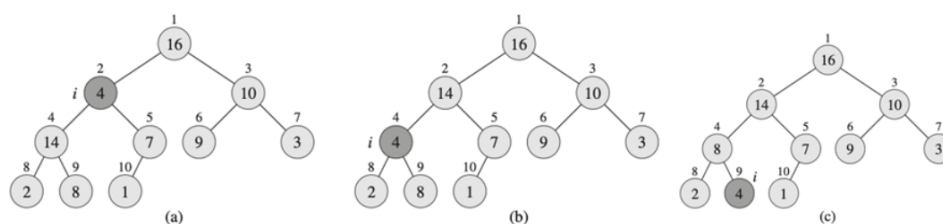


Figura 6.2 Il funzionamento di MAX-HEAPIFY($A, 2$), dove $A.\text{heap-size} = 10$. (a) La configurazione iniziale, con $A[2]$ nel nodo $i = 2$, che viola la proprietà di max-heap, in quanto non è più grande di entrambi i figli. In (b) la proprietà di max-heap viene ripristinata nel nodo 2 scambiando $A[2]$ con $A[4]$; ma questo distrugge la proprietà di max-heap nel nodo 4. La chiamata ricorsiva MAX-HEAPIFY($A, 4$) adesso ha $i = 4$. Dopo aver scambiato $A[4]$ con $A[9]$, come mostrato in (c), il nodo 4 è sistemato e la chiamata ricorsiva MAX-HEAPIFY($A, 9$) non apporta ulteriori modifiche alla struttura dati.

costo

dalla riga a 9 faccio solo confronti perciò ha costo $O(1)$, mentre la chiamata ricorsiva fatta al più h volte, dove h è l'altezza del sottoalbero ($< \log n$). Quindi ha costo **$O(h)$** .

Costruire un max-heap

La procedura Build-Max-Heap eseguita in un tempo lineare, genera da un array di input non ordinato un max-heap;

```
BUILD-MAX-HEAP(A,n)
1  A.heapSize = n
2  for i = [n/2] down to 1
3      MAX-HEAPIFY(A,i)
```

Funzionamento

La funzione Max-Heapify può essere utilizzata in modo alternativo, ovvero dal basso verso l'alto, per convertire un array di input con $n = A.length$ in un max-heap. E' possibile utilizzare la funzione Max-Heapify poiché gli ultimi elementi di ciascun sottoalbero possono essere considerati foglie dell'albero, dunque si parte da esse per salire verso la radice. Build-Max-Heap si limita a richiamare iterativamente Max-Heapify su A passando l'indice i che varia dalla metà dell'array decrementato fino ad 1, così facendo fa salire nell'albero i valori più grandi e fa scendere quelli più bassi.

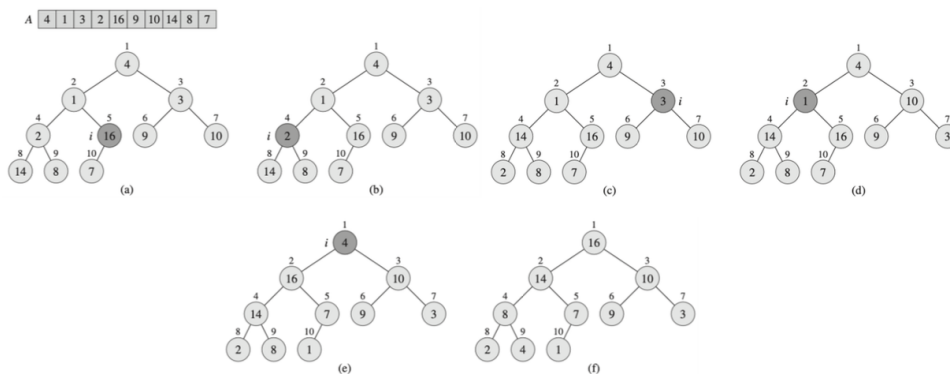


Figura 6.3 Il funzionamento di BUILD-MAX-HEAP, illustrato mostrando la struttura dati prima della chiamata della procedura MAX-HEAPIFY nella riga 3 di BUILD-MAX-HEAP. **(a)** Un array di input di 10 elementi e l'albero binario che esso rappresenta. La figura mostra che l'indice i del ciclo fa riferimento al nodo 5 prima della chiamata della procedura MAX-HEAPIFY(A, i). **(b)** La struttura dati risultante. L'indice i del ciclo per la successiva iterazione fa riferimento al nodo 4. **(c)-(e)** Le successive iterazioni del ciclo **for** in BUILD-MAX-HEAP. Notate che, ogni volta che viene chiamata la procedura MAX-HEAPIFY su un nodo, i due sottoalberi del nodo sono entrambi max-heap. **(f)** Il max-heap alla fine della procedura BUILD-MAX-HEAP.

costo

Upper bound $O(n \log n)$ perché MAX-HEAPIFY mi costa $\log n$ e lo ripeto al più n volte

Questo limite superiore sebbene corretto, non è asintoticamente stretto.

Possiamo ottenere un limite più stretto osservando che il tempo per eseguire Max-Heapify in un nodo varia con l'altezza del nodo nell'albero, e le altezze della maggior parte di essi sono più piccole. L'analisi più rigorosa si basa sulle proprietà che un heap di n elementi ha altezza $\lfloor \log n \rfloor$ e, per ogni h , ha al più $\lceil n/2^{h+1} \rceil$ nodi di altezza h . Il tempo richiesto dalla procedura Max-Heapify quando viene chiamata per un nodo di altezza h è $O(h)$.

Possiamo dire che il costo totale di build-Max-Heap è limitato superiormente da

$$\sum_{h=0}^{\lfloor \log n \rfloor} \lceil \frac{n}{2^{h+1}} \rceil O(h) = O(n \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h})$$

$$\text{con } \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h} \rightarrow \sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = 2$$

Perciò ho $O(n)$.

Correttezza di Build-Max-Heap

Per verificare che Build-Max-Heap funziona correttamente, utilizziamo la seguente invariante di ciclo.

All'inizio di ogni iterazione del ciclo **for**, righe 2-3, ognuno dei nodi $i+1, i+2, \dots, n$ è la radice di un max-heap

Inizializzazione

Prima della prima iterazione del ciclo, $i = \lfloor n/2 \rfloor$

. Ognuno dei nodi $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$ è una foglia e, quindi, è la radice di un max-heap con un solo nodo.

Mantenimento

per verificare che ogni iterazione conserva l'invariante di ciclo, notiamo che i figli del nodo i hanno una numerazione più alta di i . Per l'invariante di ciclo, quindi, essi sono entrambi radici di un max-heap. Questa è esattamente la condizione richiesta affinché la chiamata $\text{Max-Heapify}(A,i)$ renda il nodo i la radice di un max-heap. Inoltre, la chiamata di Max-Heapify preserva la proprietà che tutti i nodi $i+1, i+2, \dots, n$ siano radici di un max-heap. Il fatto che i venga decrementato di un'unità a ogni iterazione del ciclo for ristabilisce l'invariante di ciclo per la successiva iterazione.

Conclusione

Il ciclo esegue esattamente $\lfloor n/2 \rfloor$ iterazioni, e quindi termina. Alla fine del ciclo si ha $i = 0$. Per l'invariante di ciclo, ognuno dei nodi $1, 2, \dots, n$ è la radice di un max-heap; in particolare, lo è il nodo 1.

Heap Sort

Che eseguita in un tempo $O(n \log n)$, ordina un array.

$\text{HeapSort}(A, n)$

```

1  BUILD-MAX-HEAP(A, n)
2  for i = n down to 2
3      scambia A[1] con A[i]
4      A.heapSize = A.heapSize - 1
5      MAX-HEAPIFY(A, 1)
```

Funzionamento

La funzione HeapSort chiama inizialmente $\text{Build-Max-Heap}(A, n)$ per trasformare A in un max-heap. Dunque, se per definizione l'elemento più grande dell'array è memorizzato nella radice $A[1]$, per ordinare in modo crescente l'albero, basta scambiare la radice con l'elemento in posizione finale, ovvero $A[n]$. Dopodiché si decrementa $A.\text{heapSize}$, in modo tale da non considerare più l'elemento in ultima posizione, in quanto si trova nella sua posizione definitiva. Per ottenere un max-heap basta chiamare $\text{Max-Heapify}(A, 1)$ per far tornare il max alla radice. Questo ciclo viene ripetuto $n-1$ volte. Build-Max-Heap impiega $O(n)$, mentre Max-Heapify $O(\log n)$, in totale il costo di HeapSort ammonta a $O(n \log n)$.

Code di proprietà

Una delle applicazioni più efficienti e diffuse dell'heap è la coda di priorità. Una coda di priorità è una struttura dati che ci consente di mantenere un insieme S di elementi, ciascuno con un valore associato detto chiave che ne rappresenta la priorità. Una coda di max-priorità supporta le seguenti operazioni:

- **INSERT(S, x, k)** inserisce l'elemento x con chiave k nell'insieme S
- **MAXIMUM(S)** restituisce l'elemento di S con la chiave più grande
- **EXTRACT-MAX(S)** rimuove e restituisce l'elemento di S con la chiave più grande
- **INCREASE-KEY(S, x, k)** aumenta il valore della chiave di x al nuovo valore k
- **CHANGE-KEY(S, x, k)** aggiorna la priorità di x

Per rappresentare una coda di priorità si utilizza un heap, in questo caso si ha bisogno di due array: un array di chiavi che definisce le priorità e un array di oggetti che sono gli elementi di S . In particolare, ad ogni elemento in posizione i dell'array di oggetti è associata la rispettiva chiave in posizione i nell'array delle chiavi.

	i	1	2	3	4	5	6	7	8	9	10
A	x	33	19	20	16	13	10	2	14	15	12
	key	10	9	8	7	6	5	4	3	2	1

MAX-HEAP-MAXIMUM

$\text{MAXIMUM}(A)$

```

1  if A.heapSize < 1
2      error "underflow dell'heap"
3  return A[1]
```

Costo: $O(1)$

MAX-HEAP-EXTRACT-MAX

```
EXTRACT-MAXIMUM(A)
1  max = MAX-HEAP-MAXIMUM(A)
2  A[1] = A[A.heapSize]
3  A.heapSize = A.heapSize-1
4  MAX-HEAPIFY(A, 1)
5  return max
```

Costo:

Istruzioni 1, 2, 3 costo $O(1)$

Istruzione 4 costo $O(\log n)$

Costo complessivo: $O(\log n)$

INCREASE-KEY

```
INCREASE-KEY(A, x, k)
1  if k < x.key
2      error "nuova chiave più piccola"
3  x.key = k
4  cerca l'indice nell'array dove si trova x
5  while i > 1 and A[parent(i)].key < A[i].key
6      scambia A[i] e A[parent] e
7  aggiorna le corrispondenze tra gli oggetti e gli indici dell'array
8  i = parent(i)
```

Innanzitutto, verifica che la nuova chiave k causi il decremento di quella dell'oggetto x , e se non ci sono problemi assegna a x il suo nuovo valore. Quindi la procedura cerca l'indice i dell'array che corrisponde all'oggetto x , cioè tale che $A[i] = x$. Poiché l'incremento della chiave di $A[i]$ potrebbe violare la proprietà di max-heap, la procedura, segue un cammino semplice da questo nodo verso la radice, per trovare un posto appropriato alla nuova chiave. Durante questo attraversamento, INCREASE-KEY confronta ripetutamente la chiave di un elemento con quella del padre, e se la chiave dell'elemento è più grande, scambia i puntatori. In caso contrario la procedura termina, perchè in questo caso la proprietà di max-heap è soddisfatta.

Costo: $\log n$

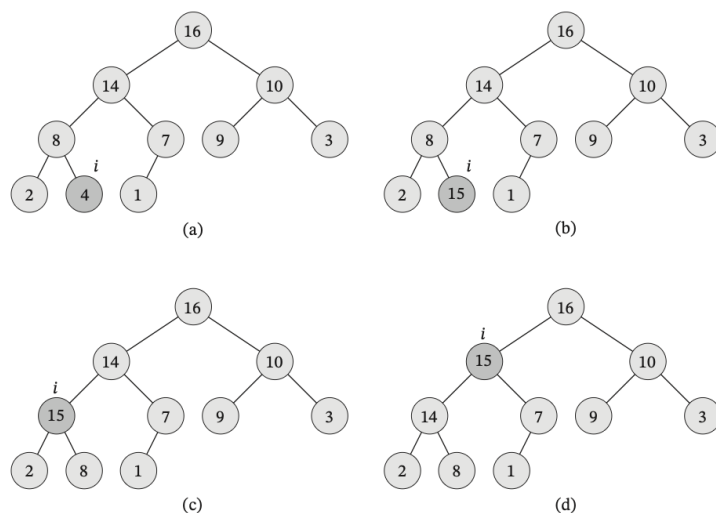


Figura 6.5 Il funzionamento di MAX-HEAP-INCREASE-KEY. Sono mostrate solo le chiavi di ogni elemento della coda di priorità. Il nodo di indice i a ogni iterazione è mostrato più scuro. **(a)** Il max-heap della Figura 6.4(a) con i che indica il nodo la cui chiave sta per essere incrementata. **(b)** La chiave di questo nodo viene aumentata a 15. **(c)** Dopo un'iterazione del ciclo **while**, righe 5-7, il nodo e suo padre hanno le chiavi scambiate, e l'indice i passa al padre. **(d)** Il max-heap dopo un'altra iterazione del ciclo **while**. A questo punto, $A[\text{PARENT}(i)] \geq A[i]$. La proprietà di max-heap adesso è soddisfatta e la procedura termina.

INSERT-KEY

```
INSERT-KEY(A, x, n)
1  if A.heap-size == n
```

```
2      error "overflow dell'Heap"
3  A.heap-size = A.heap-size + 1
4  x.key = k
5  x.key = - (infinito)
6  A[A.heap-size] = x
7  INCREASE-KEY(A,x,k)
```

Costo: $O(\log n)$, più il costo della corrispondenza tra gli oggetti della coda di priorità e gli indici.

Capitolo 3

Il problema della selezione e le statistiche d'ordine

Definizione 3.0.1. Statistica d'ordine

Dato un insieme A di n numeri (distinti), la k -esima statistica d'ordine è il k -esimo elemento più piccolo di A .

- il minimo di un insieme A è la prima statistica d'ordine.
- il massimo di un insieme A è l' n -esima statistica d'ordine.
- la mediana di un insieme A è l'elemento di mezzo.
 - se dispari una mediana $\lfloor (n+1)/2 \rfloor$.
 - se pari due mediana una inferiore e una superiore $\lceil (n+1)/2 \rceil$.

3.1 Problema selezione

Dato un insieme di elementi A il problema della selezione consiste nel selezionare la k -esima statistica d'ordine. definiamo il **problema della selezione** come segue: **input:** un insieme A di n numeri (distinti) ed un intero k , con $1 \leq k \leq n$. **output:** $x \in A$ tale che x è il k -esimo in ordine di grandezza.

osservazione:

dato un elemento $x \in A$, x è il k -esimo in ordine di grandezza se gli altri $n-1$ elementi si possono ripartire in 2 gruppi tali che:

- un gruppo contiene $k-1$ elementi tutti minori o uguali di x ed
- un gruppo contiene $n-k$ elementi tutti maggiori o uguali di x

3.2 Minimo o Massimo

Trovare il minimo e il massimo di un insieme sono problemi molto comuni e per cui già conosciamo un algoritmo semplice:

```
1  Minium(a,n)
2      min=A[1]
3      for i=2 to n
4          if min > A[i]
5              min = A[i]
6      return min
```

osservazioni $n-1$ confronti è un upper bound per il problema del minimo/massimo
L'algoritmo proposto è ottimo rispetto al numero di confronti e la complessità è $\Theta(n)$

3.3 Minimo e Massimo

Come sappiamo per cercare il min o il max siamo abituati a svolgere n confronti, ovvero confrontare ciascun elemento dell'array con il min e max.

Per trovare sia il minimo che il massimo, eseguo in sequenza Minium e il duale Maxium (soluzione Naif) e ottengo un totale di $2(n-1)$ confronti. Il costo asintotico nel caso peggiore per la ricerca del min e del max è $\Theta(n)$, non è possibile trovare un algoritmo che faccia meglio. Nonostante ciò, è comunque possibile scansionarli a coppie così che il numero di cicli passi da n a $\frac{n}{2}$ e con al più $3\lfloor n/2 \rfloor$.

Psudocodice

```

1  Min-Max(A,n)
2  if (n è dispari) then
3      max=A[1];
4      min=A[1];
5      i=2;
6  else
7      if(A[1]<A[2]) then
8          max=A[2];
9          min=A[1];
10         i=3;
11     else
12         max=A[1];
13         min=A[2];
14         i=3;
15     end
16 end
17 while(i<n) do
18     if(A[i]<A[i+1]) then
19         if (A[i]<min) then
20             min=A[i];
21         end
22         if (A[i+1]>max) then
23             max=A[i+1];
24         end
25     else
26         if(A[i+1]<min) then
27             min=A[i+1];
28         end
29         if(A[i]>max) then
30             max=A[i];
31         end
32     end
33     i=i+2;
34 end
35 return min,max

```

Prima confrontiamo due elementi di input l'uno con l'altro e poi il più piccolo dei due con il minimo corrente, e il più grande dei due con il massimo corrente, con un costo di 3 confronti ogni 2 elementi.

Come si debba impostare i valori iniziali del minimo e del massimo dipende dal fatto che n sia pari o dispari. Se n è dispari, assegniamo al minimo e al massimo il valore del primo elemento, e poi esaminiamo i restanti elementi a coppie. Se n è pari, effettuiamo un confronto fra i primi 2 elementi per determinare i valori iniziali del minimo e del massimo; poi esaminiamo i restanti elementi a coppie, come nel caso di n dispari.

Nelle iterazioni del ciclo while vengono effettuati sempre 3 confronti, allora è possibile dire che il numero di confronti se n è pari o dispari è:

$$n \text{ dispari: } 3 \cdot \left(\frac{n-1}{2}\right) \quad n \text{ pari: } 3 \cdot \left(\frac{n-2}{2}\right) + 1$$

Nel caso di n dispari viene assegnato ad i il valore 2, questo significa che il while farà $\frac{n-1}{2}$ cicli; infatti, bisogna notare come i venga incrementato di 2 ogni volta, dimezzando così il numero di cicli. Il tutto è moltiplicato per 3 in quanto come abbiamo visto il numero di confronti per il ciclo è 3. Nel caso n pari dobbiamo considerare il costo costante del confronto (+1) e notare in seguito come ad i venga assegnato il valore 3. Questo significa che, ragionando come prima, il numero di cicli che effettuerà il while sarà $\frac{n-2}{2}$, il tutto moltiplicato per 3 dato il numero di confronti per ciclo.

Esempio.

$A = \langle 5, 8, 23, 1, 7, 9 \rangle$ con $n = 6$

Passo 1:

n è pari allora prendo i primi due valori dell'array e li confronto e assegno $min = 5$ e $max = 8$ e pongo $i = 3$

Passo 2:**Iterazione 1:**

1. prendo la coppia $A[i] = 23$ e $A[i + 1] = 1$:
2. $A[i] < A[i + 1]$? No, allora:
3. $A[i + 1] < min = 5$? Si, allora $min = 1$
4. $A[i] > max = 8$? Si, allora $max = 23$
5. $i = 3 + 2 = 5$

Iterazione 2:

1. prendo la coppia $A[i] = 7$ e $A[i+1] = 9$:
2. $A[i] < A[i + 1]$? Si, allora:
3. $A[i] < min = 1$? No
4. $A[i + 1] > max = 23$? No
5. $i = 5 + 2 = 7$
6. esce dal ciclo e il programma termina con $min = 1$ e $max = 23$.

Analisi esempio.

	n dispari	n pari
Init	1 confronto → 0 confronti	2 confronti → 1 confronto
Iterazione 1	confronti	3 confronti
Iterazione 2	3 confronti	3 confronti

3.4 Selezione Randomizzata

Il problema generale della selezione e il problema del minimo o del massimo hanno lo stesso tempo di esecuzione asintotico: $\Theta(n)$. Sfruttiamo la proprietà della funzione partizione usata nel QuickSort e continuo a cercare solo se il pivot non è la statistica d'ordine che cercavo.
in dettaglio:

- usiamo la versione modificata della procedura usata in QuickSort in cui il pivot viene scelto in modo casuale
- partizioniamo l'input come facciamo in QuickSort
- controllo in che posizione cade il pivot dopo la partizione
 - Se il pivot è nella posizione i corrispondente alla statistica d'ordine che cercavo ho finito
 - Altrimenti faccio una chiamata ricorsiva su una sola delle due partizioni
 - Quale partizione scegliere per fare la chiamata ricorsiva dipende dalla relazione tra il pivot trovato e la mia statistica d'ordine cercata

Pseudocodice

```

1 Randomized_Select(A,p,r,i)
2 if(p == r) then
3     return A[p]
4 end
5 q = Randomized_Partition(A,p,r)
6 k = q - p + 1
7 if(i == k)
8     return A[q]
9 else
10     if (i < k) then
11         return Randomized_Select(A,p,q-1,i);
12     else
13         return Randomized_Select(A,q+1,r,i-k);
14     end
15 end

16 Randomized_Partition(A,p,r)
17 i = random(p,r);
18 scambia(A[r],A[i]);
19 return partiziona(A,p,r) //stessa procedura usata in QuickSort

```

Esempio.

		p	r	i	partizionamento	utile?																															
$A^{(0)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>6</td><td>19</td><td>4</td><td>12</td><td>14</td><td>9</td><td>15</td><td>7</td><td>8</td><td>11</td><td>3</td><td>13</td><td>2</td><td>5</td><td>10</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	6	19	4	12	14	9	15	7	8	11	3	13	2	5	10	1	15	5			
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
6	19	4	12	14	9	15	7	8	11	3	13	2	5	10																							
$A^{(1)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>6</td><td>4</td><td>12</td><td>10</td><td>9</td><td>7</td><td>8</td><td>11</td><td>3</td><td>13</td><td>2</td><td>5</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	6	4	12	10	9	7	8	11	3	13	2	5	14	19	15	1	12	5	1	no	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
6	4	12	10	9	7	8	11	3	13	2	5	14	19	15																							
$A^{(2)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>10</td><td>9</td><td>7</td><td>8</td><td>11</td><td>6</td><td>13</td><td>5</td><td>12</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	10	9	7	8	11	6	13	5	12	14	19	15	4	12	2	2	sì	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
3	2	4	10	9	7	8	11	6	13	5	12	14	19	15																							
$A^{(3)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>10</td><td>9</td><td>7</td><td>8</td><td>11</td><td>6</td><td>12</td><td>5</td><td>13</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	10	9	7	8	11	6	12	5	13	14	19	15	4	11	2	3	no	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
3	2	4	10	9	7	8	11	6	12	5	13	14	19	15																							
$A^{(4)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>11</td><td>9</td><td>12</td><td>10</td><td>13</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	5	6	7	8	11	9	12	10	13	14	19	15	4	5	2	4	sì	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
3	2	4	5	6	7	8	11	9	12	10	13	14	19	15																							
$A^{(5)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>11</td><td>9</td><td>12</td><td>10</td><td>13</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	5	6	7	8	11	9	12	10	13	14	19	15	5	5	1	5	sì	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
3	2	4	5	6	7	8	11	9	12	10	13	14	19	15																							

Figura 9.1 Il funzionamento della procedura RANDOMIZED-SELECT, rappresentata tramite successive partizioni che riducono la dimensione del sottoarray $A[p : r]$, mostrando i valori dei parametri p , r e i a ogni chiamata ricorsiva. A ogni passo della ricorsione, il sottoarray $A[p : r]$ è mostrato in grigio chiaro, mentre l'elemento più scuro è il pivot individuato per il partizionamento successivo. Gli elementi in grigio intermedio sono al di fuori di $A[p : r]$. La risposta è l'elemento più scuro dell'array più in basso, con $p = r = 5$ e $i = 1$. I nomi $A^{(0)}, A^{(1)}, \dots, A^{(5)}$, con cui sono designati i vari array, i numeri di partizionamenti e il fatto che il partizionamento sia utile o no, sono spiegati nel testo.

Figura 9.1 Il funzionamento della procedura RANDOMIZED-SELECT, rappresentata tramite successive partizioni che riducono la dimensione del sottoarray $A[p:r]$, mostrando i valori dei parametri p , r e i a ogni chiamata ricorsiva. A ogni passo della ricorsione, il sottoarray $A[p:r]$ è mostrato in grigio chiaro, mentre l'elemento più scuro è il pivot individuato per il partizionamento successivo. Gli elementi in grigio intermedio sono al di fuori di $A[p:r]$. La risposta è l'elemento più scuro dell'array più in basso, con $p = r = 5$ e $i = 1$. I nomi $A^{(0)}, A^{(1)}, \dots, A^{(5)}$, con cui sono designati i vari array, i numeri di partizionamento e il fatto che il partizionamento sia utile o no, sono spiegati nel testo.

Analisi del costo

Questo algoritmo ha un costo di caso peggiore $\Theta(n^2)$. poichè il partizionamento richiede un tempo $\Theta(n)$, la ricorrenza per il tempo di esecuzione nel caso peggiore è la stessa di quella per QuickSort, ovvero $T(n) = T(n-1) + \Theta(n)$, che ha soluzione $T(n) = \Theta(n^2)$. Tuttavia la randomizzazione mi aiuta nel caso medio in cui il tempo atteso di esecuzione è lineare.

Analisi del valore atteso del tempo di esecuzione



Supponiamo che ogni volta che l'algoritmo seleziona casualmente un pivot, questo si trovi da qualche parte tra il secondo e il terzo quartile (parte centrale, quella in verde) degli elementi rimanenti, se presi in ordine. Se l' i -esimo elemento più piccolo è minore del pivot, tutti gli elementi maggiori del pivot vengono ignorati in tutte le future chiamate ricorsive. Tra questi elementi che verranno sempre ignorati, ci sono senz'altro quelli del quartile superiore, e possibilmente anche qualcun altro. Allo stesso modo, se l' i -esimo elemento più piccolo è maggiore del pivot, tutti gli elementi minori del pivot, cioè almeno tutti quelli nel primo quartile, vengono ignorati in tutte le future chiamate ricorsive.

In entrambi i casi almeno $\frac{1}{4}$ degli elementi rimanenti viene ignorato in tutte le future chiamate ricorsive, lasciando in gioco al massimo $\frac{3}{4}$ degli elementi rimanenti, che rimarranno nell'array $A[p : r]$. Poiché Randomized-Partition richiede un tempo $\Theta(n)$ su un sottoarray di n elementi, la ricorrenza per il tempo di esecuzione nel caso peggiore è $T(n) = T(\frac{3n}{4}) + \Theta(n)$; questa ricorrenza ha soluzione $T(n) = \Theta(n)$.

naturalmente il pivot non cade necessariamente ogni volta nella metà centrale. Poiché il pivot è scelto a caso, la probabilità che cada nella metà centrale è circa $\frac{1}{2}$ ogni volta.

Ci aspettiamo che metà delle partizioni riduca il numero di elementi ancora in gioco di un numero che è almeno $\frac{1}{4}$ di quelli in gioco al passo precedente, mentre che l'altra metà delle partizioni non sia altrettanto utile. Di conseguenza il numero previsto di partizioni al massimo raddoppia, rispetto al caso in cui il pivot cade sempre nella metà centrale. Il costo di ogni partizione aggiuntiva è inferiore a quello della precedente, per cui il tempo di esecuzione atteso è ancora $\Theta(n)$.

Lemma 3.4.1. Una partizione è utile con probabilità almeno $\frac{1}{2}$.

(da correggere la frazione con la parte intera superiore)

Dimostrazione. Definiamo più precisamente la metà centrale di un array di n elementi. Questa corrisponde a tutti gli elementi tranne gli $\lceil \frac{n}{4} \rceil - 1$ più piccoli e gli $\lceil \frac{n}{4} \rceil - 1$ più grandi.

Indipendentemente dal punto in cui cade il pivot, o tutti gli elementi maggiori o tutti quelli minori, compreso stesso, non saranno più in gioco dopo la partizione. Se il pivot cade nella metà centrale, quindi, almeno gli $\lceil \frac{n}{4} \rceil - 1$ elementi minori o gli $\lceil \frac{n}{4} \rceil - 1$ elementi maggiori del pivot, esso compreso, non saranno più in gioco. Il numero di elementi che rimangono in gioco sarà al massimo $n - \lceil \frac{n}{4} \rceil$ che è uguale a $\lfloor \frac{3n}{4} \rfloor$. Poiché $\lfloor \frac{3n}{4} \rfloor \leq \frac{3n}{4}$, la partizione è utile.

Per determinare un limite inferiore alla probabilità che un pivot scelto a caso cada nella metà centrale, determiniamo un limite superiore alla probabilità che non ci cada. Tale probabilità è

$$\begin{aligned} \frac{2(\lceil n/4 \rceil - 1)}{n} &\leq \frac{2(\lceil n/4 + 1 \rceil - 1)}{n} \\ &= \frac{n/2}{n} \\ &= \frac{1}{2} \end{aligned}$$

pertanto il pivot ha probabilità almeno $\frac{1}{2}$ di cadere nella metà centrale, e quindi la probabilità che una partizione sia utile è almeno $\frac{1}{2}$. $\square$

Teorema 3.4.1. La procedura Randomized-Select ha un tempo di esecuzione atteso $\Theta(n)$ su un array di input di n valori distinti

Dimostrazione. Poiché non tutte le partizioni non sono necessariamente utili, assegniamo a ciascuna un indice a partire da 0 e indichiamo con $\langle h_0, h_1, h_2, \dots, h_m \rangle$ la sequenza di partizioni utili, in modo che la h_k -esima partizione sia utile per $k = 0, 1, 2, \dots, m$. Assumiamo convenzionalmente che la partizione fittizia di indice 0 è utile, così che $h_0 = 0$. Indichiamo $|A^{(h_k)}|$ con n_k , dove $n_0 = |A^{(0)}|$ è la dimensione del problema originale. Poiché la h_k -esima partizione è utile e le cardinalità degli insiemi $|A^{(j)}|$ decrescono strettamente, si ha $n_k = |A^{(h_k)}| \leq (3/4)|A^{(h_{k-1})}| = (3/4)n_{k-1}$ per $k = 1, 2, \dots, m$. Iterando la disuguaglianza $n_k \leq (3/4)n_{k-1}$, si ha $n_k \leq (3/4)^k n_0$ per $k = 0, 1, 2, \dots, m$.

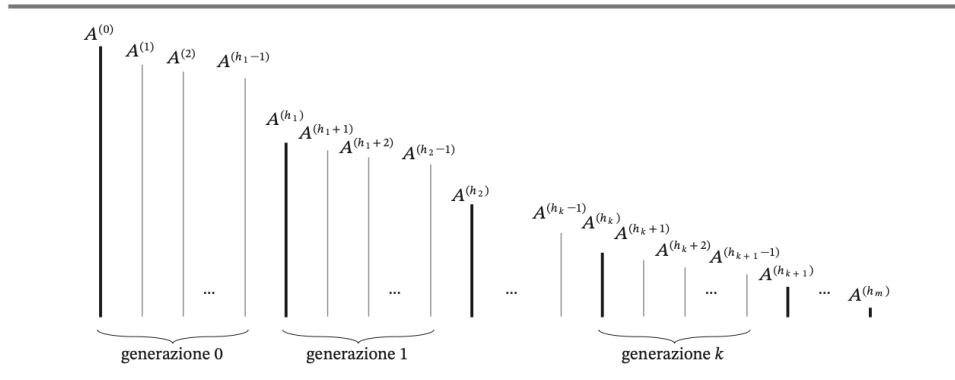


Figura 9.2 Gli insiemi all'interno di ogni generazione nella dimostrazione del Teorema 9.2. Le linee verticali rappresentano gli insiemi, e l'altezza di ogni linea rappresenta la cardinalità dell'insieme, che equivale al numero di elementi in gioco. Ogni generazione inizia con un insieme $A^{(h_k)}$, che è il risultato di una partizione utile. Questi insiemi sono rappresentati da una linea più spessa e hanno al massimo $3/4$ degli elementi degli insiemi alla loro immediata sinistra. Gli insiemi rappresentati da una linea più sottile non sono i primi di una generazione. Una generazione può anche contenere un solo insieme. Gli insiemi della generazione k sono $A^{(h_k)}, A^{(h_k+1)}, \dots, A^{(h_k+X_k-1)}$. Gli insiemi $A^{(h_k)}$ sono definiti in modo che $|A^{(h_k)}| \leq (3/4)|A^{(h_{k-1})}|$. Se la sequenza di partizioni arriva fino alla generazione m , l'insieme $A^{(h_m)}$ ha al più un solo elemento in gioco.

Ora definiamo la variabile casuale

$$X_k = h_{k+1} - h_k$$

per $k=0,1,2,\dots,m-1$. Quindi, X_k è il numero di insiemi nella k -esima generazione, di modo che gli insiemi nella k -esima generazione sono $A^{(h_k)}, A^{(h_k+1)}, \dots, A^{(h_k+X_k-1)}$.

Per il lemma precedente, la probabilità che una partizione sia utile è almeno $\frac{1}{2}$. La probabilità è in realtà maggiore, poiché una partizione è utile anche se il pivot non cade nella metà centrale, ma l' i -esimo elemento più piccolo si torva nella sezione più piccola della partizione. Utilizzeremo comunque la stima inferiore di $\frac{1}{2}$, quindi dall'equazione segue $E[X_k] \leq 2$ per $k = 0, 1, 2, \dots, m-1$.

Ne segue che l'equazione di ricorrenza calcolata in precedenza viene al massimo raddoppiata non andando ad impattare sul costo asintotico atteso.

Stimiamo superiormente il numero totale di confronti durante tutte le partizioni. Alla j -esima partizione si effettuano meno di $|A^{(j-1)}|$ confronti.

Raggruppando per generazioni, otteniamo:

$$\sum_{k=0}^{m-1} \sum_{j=h_k}^{h_{k+1}-1} |A^{(j)}| \leq \sum_{k=0}^{m-1} X_k |A^{(h_k)}|.$$

Poiché la dimensione del problema diminuisce geometricamente:

$$|A^{(h_k)}| \leq \left(\frac{3}{4}\right)^k n_0,$$

segue:

$$\sum_{k=0}^{m-1} X_k |A^{(h_k)}| \leq \sum_{k=0}^{m-1} X_k \left(\frac{3}{4}\right)^k n_0.$$

Prendendo il valore atteso e usando la linearità:

$$\mathbb{E} \left[\sum_{k=0}^{m-1} X_k \left(\frac{3}{4}\right)^k n_0 \right] = n_0 \sum_{k=0}^{m-1} \left(\frac{3}{4}\right)^k \mathbb{E}[X_k] \leq 2n_0 \sum_{k=0}^{m-1} \left(\frac{3}{4}\right)^k.$$

Estendendo la somma a infinito:

$$< 2n_0 \sum_{k=0}^{\infty} \left(\frac{3}{4}\right)^k = 8n_0.$$

Quindi il numero atteso di confronti è $O(n)$; inoltre, essendo necessario esaminare tutti gli elementi almeno una volta, si ha anche un limite inferiore $\Omega(n)$. Pertanto, il tempo di esecuzione atteso di RANDOMIZED-SELECT è:

$$\Theta(n).$$

□

3.5 Selezione in tempo

Esamiamo ora un algoritmo di selezione interessante dal punto di vista teorico, il cui tempo di esecuzione è $\Theta(n)$ nel caso peggiore.

Come **RANDOMIZED-SELECT**, che ha un tempo di esecuzione lineare nel caso peggiore, trova l'elemento desiderato partizionando ricorsivamente l'array di input. A differenza di **RANDOMIZED-SELECT**, però, **SELECT** garantisce che la partizione sia buona scegliendo, durante la partizione dell'array, un pivot di cui si può provare la bontà. Pertanto ci sono due chiamate di **SELECT**: una per trovare un buon pivot e una seconda per trovare ricorsivamente la statistica d'ordine.

Pseudocodice

prende in input un array $A[p : r]$ di $n = r - p + 1$ elementi e un intero i tale che $1 \leq i \leq n$

```
1  Select (A,p,r,i)
2  while(r-p+1) mod5 != 0 // operazione di modulo ovvero il resto
3      for j = p+1 to r do
4          if(A[p] > A[j]) then
5              scambia(A[p],A[j]);
6          end
7      end
8      if (i == 1) then
9          return A[p]
10     end
11     p = p+1;
12     i = i-1;
13 end
14 g=(r-p+1)/5;
15 for j = p to p + g - 1 do
16     ordina_sul_posto(A[j],A[j+g],A[j+2g],A[j+3g],A[j+4g]);
17 end
18 // Cerca ricorsivamente il pivot x come mediana delle mediane
19 x = select(A,p+2g,p+3g-1,(g+1)/2);
20 q = partition_around(A,p,r,x) //partiziona intorno al pivot
21 k = q-p+1
22 if(i==k) then
23     return A[q];
24 else
25     if(i<k) then
26         return Select(A,p,q-1,i);
27     else
28         return Select(A,q+1,r,i-k);
29     end
30 end
```

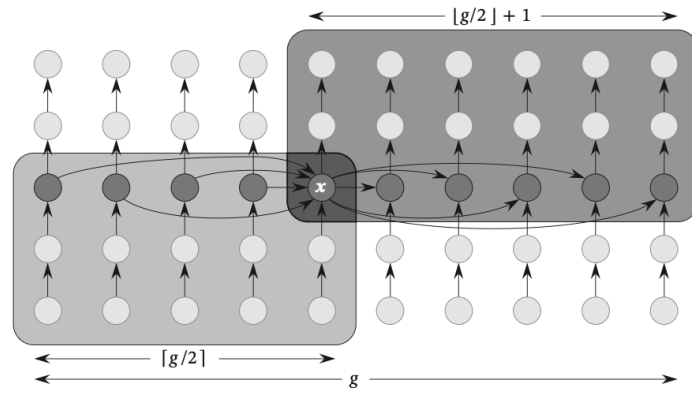


Figura 9.3 Le relazioni tra gli elementi di A (rappresentati da cerchi) subito dopo la riga 17 dell'algoritmo di selezione SELECT. Ci sono $g = (r - p + 1)/5$ cinquine di elementi, ognuna delle quali occupa una colonna. Per esempio, la colonna più a sinistra contiene gli elementi $A[p], A[p + g], A[p + 2g], A[p + 3g], A[p + 4g]$ e la colonna successiva contiene $A[p + 1], A[p + g + 1], A[p + 2g + 1], A[p + 3g + 1], A[p + 4g + 1]$. Le mediane dei gruppi sono i cerchi scuri e il pivot x è indicato con una etichetta. Le frecce vanno dagli elementi più piccoli a quelli più grandi. Gli elementi su sfondo grigio in basso a sinistra sono tutti minori o uguali a x e non possono cadere nella sezione alta della partizione fatta intorno a x . Gli elementi su sfondo grigio in alto a destra sono maggiori o uguali a x e non possono cadere nella sezione bassa della partizione fatta intorno a x . Il pivot x appartiene a entrambe le regioni con sfondo grigio ed è mostrato su uno sfondo più scuro. Gli elementi su sfondo bianco potrebbero trovarsi in entrambe le sezioni della partizione.

Le cinquine sono costruite verticalmente, in modo che gli elementi che le costruiscono siano vicini tra loro. Di ciascuna si seleziona la mediana, con la caratteristica di essere maggiore degli elementi precedenti e minore dei successivi della cinquina. In questo modo si è messo in evidenza l'insieme delle mediane. Con l'istruzione $x = \text{select}(A, p + 2g, p + 3g - 1, (g + 1)/2)$; si cerca la mediana delle mediane all'interno dell'insieme, questo coincide con l'elemento al centro dell'array complessivo. Una volta chiamata la funzione partiziona questa sposta tutti gli elementi minori di x nel sottoarray di sinistra e quelli maggiori nel sottoarray di destra. Se $i == k$ si è trovata la k -esima statistica d'ordine, altrimenti si chiama ricorsivamente la funzione select sul sottoarray di sinistra se $i < k$, o su quello di destra.

Capitolo 4

Programmazione Dinamica

4.1 Panoramica generale

La programmazione dinamica, come il metodo Divide et Impera, risolve i problemi combinando le soluzioni dei sottoproblemi. La programmazione dinamica può essere applicata quando i sottoproblemi non sono indipendenti, ovvero quando i sottoproblemi hanno in comune dei sottoproblemi.

A differenza del metodo Divide et Impera, la programmazione dinamica risolve ciascun sotto sottoproblema **una sola volta** e salva la sua soluzione in una tabella, evitando così di ricalcolare la soluzione ogni volta che si presenta il sotto sottoproblema.

La programmazione dinamica, tipicamente, si applica ai **problemi di ottimizzazione**. Per questi problemi ci possono essere molte soluzioni possibili, ogni soluzione ha un valore e vogliamo trovare una soluzione con il valore ottimo.

4.2 Sviluppo dell'algoritmo

Per sviluppare un algoritmo che faccia uso della programmazione dinamica, seguiamo una strategia formata dalle seguenti quattro fasi:

1. Identificare i sottoproblemi.
2. Definire la funzione di stato.
3. Scegliere tra un approccio TOP DOWN o BOTTOM UP.
4. Recuperare la soluzione (se necessario).

Vediamo nel dettaglio i vari passaggi.

Funzione di stato

Dopo aver identificato i sottoproblemi dobbiamo definire la **funzione di stato**.

Definizione 4.2.1. (Funzione di Stato) La funzione di stato è la funzione che descrive la soluzione di un sottoproblema.

Ogni sottoproblema è identificato da una o più variabili di stato, cioè i parametri che descrivono in quale punto del problema ci si trova. La funzione di stato permette di tradurre il problema in una forma ricorsiva.

Ora vediamo i due approcci TOP DOWN e BOTTOM UP:

Top Down

Con l'approccio **Top Down** con **memoizzazione** scriviamo la soluzione ricorsiva, salvando il risultato di ciascun sottoproblema (di solito in un array o in una tabella hash). La procedura prima verifica se ha risolto precedentemente questo sottoproblema. In caso affermativo, restituisce il valore salvato, risparmiando gli ulteriori calcoli a quel livello; altrimenti, la procedura calcola il valore nel modo usuale e lo memoizza. Diciamo che la procedura ricorsiva è stata dotata di un blocco note (memo) e quindi memoizzata, nel senso che “annota” quali risultati sono stati calcolati in precedenza. Una cosa importante dell'approccio Top Down è che l'array o la tabella dove vengono salvati i risultati, possono non essere riempiti completamente, e questo è un gran vantaggio per l'utilizzo della memoria.

Bottom Up

Con l'approccio **Bottom Up** Risolviamo i problemi in ordine di grandezza, partendo dai più piccoli, e tenendo da parte la soluzione di ogni sottoproblema la prima volta che viene incontrato. Così facendo, quando si risolve un certo sottoproblema, le soluzioni di tutti quelli più piccoli da cui dipende sono già state salvate. Dobbiamo risolvere un sottoproblema una sola volta e la prima volta in cui ci imbattiamo in esso abbiamo già risolto tutti i sottoproblemi da cui dipende la sua soluzione.

Esempio.

Prendiamo in considerazione il problema della successione di **Fibonacci**.

Per prima cosa vediamo un codice che non segue le regole della programmazione dinamica:

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
// Complessità:  $O(2^n)$ 
```

Questa soluzione segue il metodo Divide et Impera, ma è molto inefficiente poichè sottoproblemi uguali vengono calcolati più volte, senza alcuna memorizzazione.

Ora utilizziamo la programmazione dinamica.

Vediamo prima l'approccio TOP DOWN con memoizzazione:

```
#define MAX 100
int memo[MAX];
int fib_memo(int n) {
    if (memo[n] != -1) return memo[n];
    if (n <= 1) return n;
    memo[n] = fib_memo(n-1) + fib_memo(n-2);
    return memo[n];
}
// In main(): inizializzare memo con -1
```

In questa soluzione ogni valore delle chiamate ricorsive viene salvato in `memo[n]`, in questo modo non servirà ricalcolare sottoproblemi già calcolati in precedenza.

Infine vediamo l'approccio BOTTOM UP:

```
int fib_tab(int n) {
    int dp[MAX];
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

Qui vengono prima calcolati i primi due sottoproblemi (`dp[0]` `dp[1]`), che poi verranno usati nel ciclo `for` e non dovranno essere ricalcolati.

4.3 Problema dello zaino 0/1

Immaginiamo di possedere uno zaino che può sopportare un peso massimo W . Abbiamo a disposizione n oggetti, ogni oggetto ha:

- Un **peso** (`peso[i]`)
- Un **valore** (`valore[i]`)

L'**obiettivo** è massimizzare il valore totale degli oggetti che mettiamo nello zaino, senza che il loro peso totale superi il valore di peso massimo w . Si chiama (0/1) perchè ogni oggetto o lo prendiamo intero (1) o lo lasciamo (0), non è possibile quindi prendere frazioni di oggetti.

Questo è il codice che risolve il problema dello zaino 0/1:

```
int knapsack(int n, int W, int peso[], int valore[]) {
    int dp[n+1][W+1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (peso[i-1] <= w)
                dp[i][w] = fmax(dp[i-1][w], dp[i-1][w - peso[i-1]] + valore[i-1]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }
    return dp[n][W];
}
```

Ecco la spiegazione della soluzione proposta:

1. `dp[n+1][W+1];`
Viene introdotta la **funzione di stato**, che rappresenta il massimo valore con n oggetti e capacità massima W .
2. `for (int i = 0; i <= n; i++)`
Inizia il ciclo esterno, iteriamo sul numero di oggetti a nostra disposizione.
3. `for (int w = 0; w <= W; w++)`
Inizia il ciclo interno, per ogni numero di oggetti i , proviamo a riempire zaini di capacità crescente, da 0 fino alla capacità massima W .
4. `if (i == 0 || w == 0) dp[i][w] = 0;`
Con questa condizione viene implementato il **caso base**, se stiamo considerando 0 oggetti o abbiamo una capacità massima nulla, impostiamo il valore a 0.
5. `else if (peso[i-1] <= w);`
Con questa condizione si verifica se l'oggetto corrente ($i-1$) può entrare nello zaino senza superare il peso disponibile w . Se l'oggetto non può entrare perchè troppo pesante, allora `dp[i][w] = dp[i-1][w];`, ovvero copiamo semplicemente il valore che avevamo ottenuto senza quell'oggetto.
6. `dp[i][w] = fmax (dp[i-1][w], dp[i-1][w-peso[i-1]] + valore[i-1]);`
Se l'oggetto può entrare, devo fare una scelta che massimizzi il mio guadagno. Usiamo la funzione `fmax` per decidere se conviene di più:
 - Lasciare l'oggetto (`dp[i-1][w]`)
 - Prendere l'oggetto (`dp[i-1][w-peso[i-1]] + valore[i-1]`)

Qui l'uso degli indici può sembrare difficile da capire, perciò vediamo nel dettaglio cosa vogliono dire quelle due espressioni. Se lasciamo l'oggetto corrente, impostiamo che `d[i][w] = dp[i-1][w]`, quindi stiamo attribuendo a `d[i][w]` un valore che già era stato calcolato in precedenza e che è stato ritenuto il valore massimo dalla funzione `fmax` rispetto al valore che avremmo ottenuto prendendo l'oggetto. Siccome non prendiamo l'oggetto l'indice w rimane invariato, in quanto il peso corrente non cambia.

Invece se prendiamo l'oggetto, impostiamo che `d[i][w] = dp[i-1][w-peso[i-1]] + valore[i-1]`. Siccome stiamo prendendo l'oggetto, dobbiamo considerare due cose: la prima è aggiungere il valore dell'oggetto che prendiamo, e questo lo facciamo sommando `valore[i-1]`, la seconda è che ora il peso disponibile diminuisce visto che abbiamo preso l'oggetto e quindi dobbiamo sottrarre alla capacità corrente (w) il peso dell'oggetto che prendiamo (`peso[i-1]`), quindi scriviamo `w-peso[i-1]`.

7. `return dp[n][W];`
Restituisco il risultato finale: il valore massimo ottenibile considerando tutti gli n oggetti e sfruttando tutta la capacità W dello zaino.

4.4 Cammini su griglia (senza ostacoli)

Data una griglia rettangolare $m \times n$ si ha che:

- **Partenza:** si parte alle coordinate $0, 0$
- **Arrivo:** si deve raggiungere la cella in basso a destra, alle coordinate $(m - 1, n - 1)$
- **Vincoli:** ci si può muovere solo verso destra oppure verso il basso

L'**obiettivo** è quello di contare quanti percorsi diversi (distinti) esistono per andare dalla partenza all'arrivo. Seguiamo la logica della programmazione dinamica e spezziamo il problema in sottoproblemi. Quindi piuttosto che cercare di calcolare tutti i percorsi fino all'arrivo in un colpo solo, cerchiamo di capire quanti percorsi ci sono per arrivare a una singola cella intermedia.

Questo è il codice che risolve il problema:

```
int unique_paths(int m, int n) {
    int dp[m][n];
    for (int i = 0; i < m; i++)
        dp[i][0] = 1;
    for (int j = 0; j < n; j++)
        dp[0][j] = 1;
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
    return dp[m-1][n-1];
}
```

La **funzione di stato** $dp[i][j]$ rappresenta il numero totale di cammini distinti per raggiungere la cella alle coordinate (i, j) . Per capire come calcolare questo numero, basta pensare ai vincoli di movimento: in una qualsiasi cella (i, j) , ci si può arrivare solamente da due posizioni:

1. Dalla cella esattamente **sopra**: $(i - 1, j)$, scendendo di un passo.
2. Dalla cella esattamente a **sinistra**: $(i, j - 1)$, spostandosi verso destra di un passo.

Quindi, il numero totale di modi per arrivare ad una cella è la somma dei modi in cui si può arrivare dalla cella di sopra, più i modi in cui si può arrivare dalla cella di sinistra, perciò l'espressione diventa:

$$dp[i][j] = dp[i-1][j] + dp[i][j-1]$$

I primi due `if` si occupano dei due casi base, ovvero il percorso che percorre la prima riga e il percorso che percorre la prima colonna. I due `for` annidati permettono di riempire il resto della matrice.

4.5 Problema LIS (List Increasing Subsequence)

Dato un array di numeri interi $A[0 \dots n - 1]$, l'**obiettivo** è determinare la lunghezza della sottosequenza strettamente crescente più lunga contenuta in A . La sottosequenza deve rispettare l'ordine dell'array in input A . Ecco il codice che risolve il problema:

```
int LIS(int A[], int n) {
    int dp[n];
    int maxLIS = 1;
    // Inizializzazione: ogni elemento da solo è una LIS di lunghezza 1 */
    for (int i = 0; i < n; i++)
        dp[i] = 1;
    // Transizione dinamica
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++)
            if (A[j] < A[i] && dp[j] + 1 > dp[i])
                dp[i] = dp[j] + 1;
        if (dp[i] > maxLIS)
            maxLIS = dp[i];
    }
    return maxLIS;
}
```

La **funzione di stato** $dp[i]$ rappresenta la lunghezza della sottosequenza crescente più lunga che termina esattamente nell'elemento $A[i]$. Per sapere quanto può essere lunga la sequenza che termina nell'elemento attuale $A[i]$, bisogna guardare tutti gli elementi $A[j]$ che precedono l'elemento attuale in cui termina la sequenza (cioè per ogni $j < i$). Se si trova un elemento $A[j] < A[i]$, vuol dire che ci si può 'attaccare' alla sequenza che finiva in $A[j]$ allungandola di 1.

4.6 Problema MSIS (Max Sum Increasing Subsequence)

Dato un array A di n interi positivi, l'**obiettivo** è trovare una sottosequenza crescente dell'array in input tale che la somma dei suoi elementi sia massima. Ecco lo pseudocodice che risolve il problema:

```

Algoritmo MSIS(A):
    n = lunghezza(A)
    se n == 0 ritorna 0

    crea array D di dimensione n

    per i da 0 a n - 1
        D[i] = A[i]

    per i da 1 a n - 1
        per j da 0 a i - 1
            se A[j] < A[i]
                D[i] = max(D[i], D[j] + A[i])

    ritorna il valore massimo in D

```

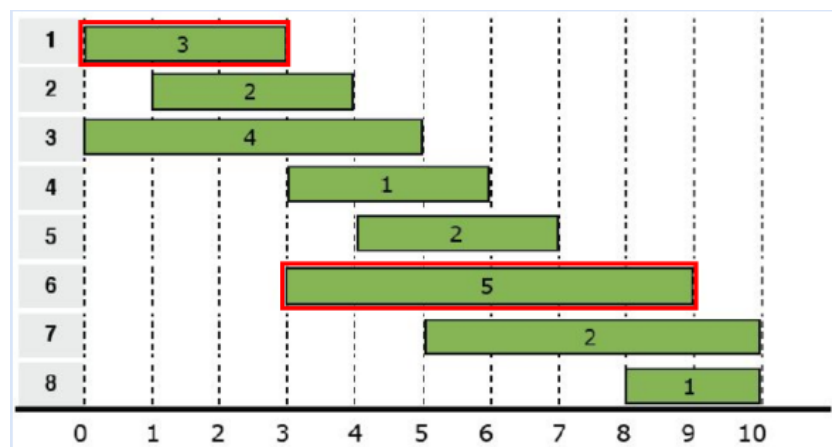
La **funzione di stato** $D[i]$ rappresenta la somma massima di una sottosequenza crescente. Il **caso base** è il caso in cui la lunghezza dell'array in input è 0. Per ottenere il risultato finale viene usato un approccio Bottom Up, ovvero vengono confrontati ogni numero dell'array e, se la condizione di crescita ($A[j] < A[i]$) è vera, allora decidiamo di prendere il massimo tra $D[i]$ e $D[j] + A[i]$. Il **costo temporale** di questa soluzione del problema MSIS è $\Theta(n^2)$, poichè vengono effettuati due cicli che eseguono $n - 1$ iterazioni.

4.7 Problema WIS (Weighted Interval Scheduling)

Sono date n attività, ogni attività i è caratterizzata da:

- un **inizio** $start[i]$.
- una **fine** $end[i]$.
- un **profitto** $profit[i]$.

L'**obiettivo** è trovare il massimo profitto totale selezionando un sottoinsieme di attività non sovrapposte. Ad esempio:



Questo problema è leggermente più complesso rispetto agli altri, perciò vediamo che strategia viene usata nel codice:

1. Ordinare le attività in base al loro tempo di fine (`end[i]`), in questo modo si crea una linea temporale facilmente analizzabile da sinistra verso destra.
2. Per ogni attività i , trovare l'attività $j < i$ non sovrapposta, per questo viene creato l'array `pre[i]`, che indica qual è l'ultima attività di i che si è conclusa in tempo affinché l'attività i possa iniziare, e viene quindi definita così:

$$\text{pre}[i] = \max \{ j < i \mid \text{end}[j] \leq \text{start}[i] \}$$

Se non ci sono eventi che non si sovrappongono allora il valore di `pre[i]` viene impostato a -1.

3. Definire la **funzione di stato** `dp[i]` come il massimo profitto ottenibile considerando solo le prime i attività, ed è definita così:

$$\text{dp}[i] = \max(\text{dp}[i-1], \text{profit}[i] + \text{dp}[\text{pre}[i]])$$

È una relazione binaria (prendo o lascio):

- **Lascio l'attività i :** il profitto rimane quello che avevo calcolato per le prime $i-1$ attività (`dp[i-1]`).
- **Prendo l'attività i :** guadagno il suo profitto (`profit[i]`). Ma visto che la sto prendendo, non posso prendere attività che si sovrappongono, quindi a questo profitto devo sommare il miglior guadagno che avevo ottenuto fino all'ultima attività compatibile, ovvero `pre[i]`.

Ecco il codice che risolve il problema:

```
typedef struct {
    int start, end, profit;
}

// Caso base: il primo intervallo può essere incluso da solo
dp[0] = intervals[0].profit;
decision[0] = 1;
// Costruzione tabella dp: applicazione della ricorrenza
// dp[i] = max(dp[i-1], profit[i] + dp[pre[i]])
for (int i = 1; i < n; ++i) {
    int incl = intervals[i].profit;
    // Se esiste un intervallo compatibile, aggiungine il profitto
    if (pre[i] != -1) incl += dp[pre[i]];
    // Scelta migliore: includere o escludere l'intervallo i
    if (incl > dp[i - 1]) {
        dp[i] = incl;
        decision[i] = 1; // includiamo i
    } else {
        dp[i] = dp[i - 1];
        decision[i] = 0; // escludiamo i
    }
}
```

Il risultato finale si troverà in `dp[n-1]`.

4.8 Problema LCS (Longest Common Subsequence)

Date due stringhe X e Y , l'**obiettivo** è trovare la sequenza di caratteri più lunga che compare in entrambe le stringhe. Attenzione: i caratteri delle sottosequenze devono apparire nello stesso ordine relativo, ma non devono essere per forza contigui (attaccati). Ad esempio:

$$X = \text{A}\underline{\text{B}}\underline{\text{C}}\underline{\text{B}}\underline{\text{A}}\underline{\text{B}} \quad Y = \underline{\text{B}}\underline{\text{D}}\underline{\text{C}}\underline{\text{A}}\underline{\text{B}}$$

La sottosequenza comune più lunga è BCAB, che ha lunghezza 4.

Ecco il codice che risolve il problema:

```
int lcs(const char* A, const char* B, char** lcs_str_out) {
    int m = strlen(A);
    int n = strlen(B);
    int dp[m+1][n+1];
    // Costruzione tabella dp
    for (int i = 0; i <= m; i++)
        for (int j = 0; j <= n; j++)
            if (i == 0 || j == 0)
                dp[i][j] = 0;
            else if (A[i-1] == B[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
}
```

La **funzione di stato** $dp[i][j]$ viene riempita in questo modo:

- **Caso base** ($i == 0$ || $j == 0$): se una delle due stringhe è vuota, non possono avere nessuna lettera in comune, perciò la prima riga e la prima colonna della matrice saranno tutte a 0.
- **I caratteri sono uguali** ($A[i-1] == B[j-1]$): la LCS si allunga di 1. Il suo valore sarà $1 +$ la lunghezza della LCS che avevamo prima di considerare queste due lettere.
- **I caratteri sono diversi** ($A[i-1] != B[j-1]$): non possiamo allungare la LCS di 1, però bisogna decidere se scartare l'ultima lettera della stringa A o scartare l'ultima lettera della stringa B , quindi scegliamo quello che ci garantisce il valore maggiore ($dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$).

Alla fine $dp[m][n]$ conterrà la lunghezza della LCS.

4.9 Problema del Rod Cutting

Data un'asta di lunghezza n e un'array di prezzi `price[]`, dove `price[i]` è il prezzo di un pezzo di asta di lunghezza $i + 1$. L'**obiettivo** è massimizzare il profitto tagliando o non tagliando l'asta.

La **funzione di stato** $dp[i]$ rappresenta il massimo profitto ottenibile vendendo un'asta (o frammenti di essa) di lunghezza totale i . Il massimo profitto verrà calcolato in questo modo: il prezzo del pezzo che ho appena tagliato (`price[j-1]`) sommato al massimo profitto che posso ottenere dall'asta rimanente, che ora è lunga $i - j$ e questo valore è proprio $dp[i-j]$. Quindi la funzione di stato è:

$$dp[i] = price[j-i] + dp[i-j]$$

Ecco quindi il codice che risolve il problema:

```
int rodCutting(int price[], int n) {
    int dp[n + 1];
    int cuts[n + 1]; // array per memorizzare il primo
    // taglio ottimale
    dp[0] = 0;
    for (int i = 1; i <= n; i++) {
        int max_val = INT_MIN;
        int best_cut = 0;
        for (int j = 1; j <= i; j++)
            if (price[j - 1] + dp[i - j] > max_val) {
                max_val = price[j - 1] + dp[i - j];
                best_cut = j;
            }
        dp[i] = max_val;
        cuts[i] = best_cut;
    }
    // ricostruzione
    while (length > 0) {

```

```

        printf("%d ", cuts[length]);
        length -= cuts[length];
    }
    return dp[n]; // restituisce il massimo profitto
}

```

Il risultato (massimo profitto) sarà in `dp[n]`.

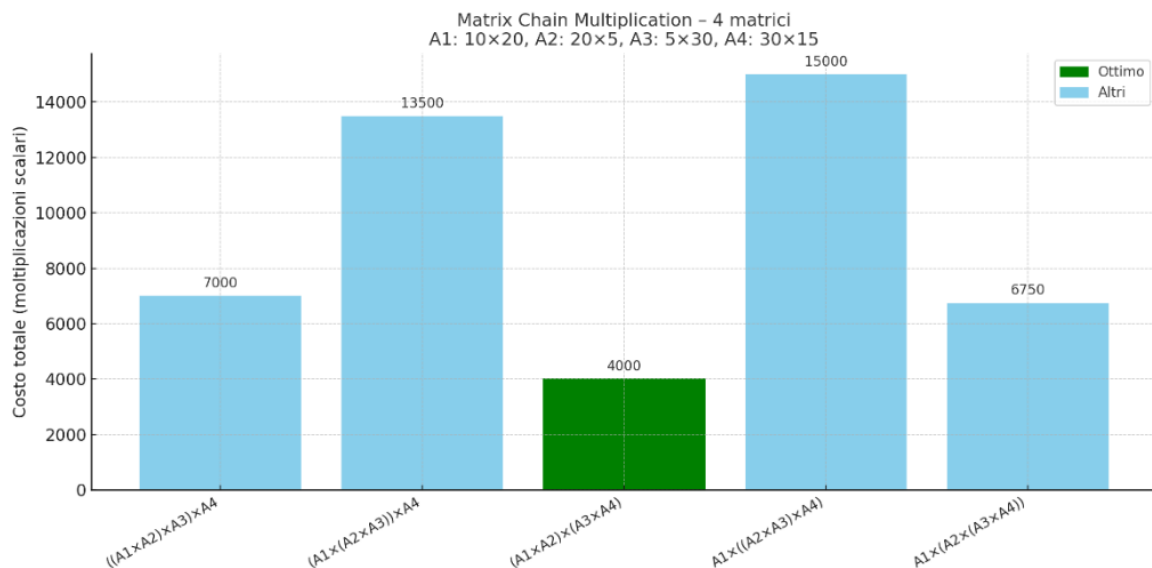
4.10 Matrix Chain Multiplication

Il problema del Matrix Chain Multiplication (moltiplicazione di una catena di matrici) ha come **obiettivo** quello di determinare una parentizzazione ottimale della sequenza (catena) di prodotti matriciali $A_1, A_2, \dots, A_n$, ma che cazzo vuol dire?

Il prodotto tra matrici è **associativa**, ma non è commutativa, quindi l'ordine dei moltiplicandi non può essere cambiato, ma solo le parentesi. Se ad esempio devo fare il prodotto tra le matrici A, B, C , posso fare:

$$(A \times B) \times C \quad \text{oppure} \quad A \times (B \times C)$$

L'algoritmo Matrix Chain Multiplication si occupa di scegliere in quale modo svolgere questo prodotto in modo tale che vengano effettuate il minor numero di moltiplicazioni possibili. A seconda di quale parentizzazione viene scelta, il costo computazionale può variare molto, basti vedere il seguente esempio:

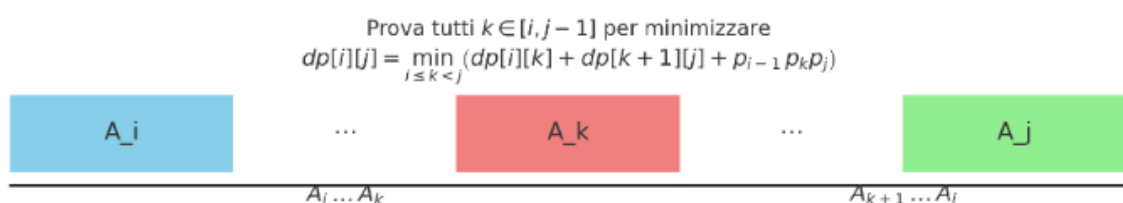


Ora che abbiamo compreso di cosa si occupa questo problema, vediamo in che modo viene risolto.

Sia $A_1, A_2, \dots, A_n$ una sequenza di n matrici, sia $p = (p_0, p_1, p_2, \dots, p_n)$ una sequenza di dimensioni. Ovvero che ogni matrice A_i ha dimensioni $p_{i-1} \times p_i$. Definiamo la **funzione di stato** $dp[i][j]$ che rappresenta il costo minimo per moltiplicare le matrici $A_i A_{i+1} \dots A_j$, ricordando che il costo per moltiplicare una matrice $l \times m$ per una matrice $m \times n$ è lmn :

$$dp[i][j] = \begin{cases} 0 & \text{se } i = j \\ \min_{i \leq k < j} (dp[i][k] + dp[k+1][j] + p_{i-1} \cdot p_k \cdot p_j) & \text{se } i < j \end{cases}$$

Dove k , che è compreso tra i e j , è l'indice della matrice dopo la quale viene divisa l'espressione con le parentesi. Per chiarezza ecco l'immagine:



Allora, ecco il codice che risolve il problema Matrix Chain Multiplication:

```
/* dp[] inizializzato a -1 */
int matrixChain(int p[], int i, int j) {
    if (i == j)
        return 0;
    if (dp[i][j] != -1)
        return dp[i][j];
    int min = INT_MAX;
    for (int k = i; k < j; k++) {
        int cost = matrixChain(p, i, k) + matrixChain(p, k+1, j) + p[i-1]*p[k]*p[j];
        if (cost < min) min = cost;
    }
    dp[i][j] = min;
    return min;
}
```

In questa soluzione la funzione di stato che avevamo definita è stata implementata con delle chiamate **ricorsive**, e il risultato ottimale è nella variabile `min`.

4.11 Coin Change Analysis

Il problema Coin Change Analysis è molto simile al problema knapsack. Dato un set di n tagli $C = \{c_1, c_2, \dots, c_n\}$ e un target V , l'**obiettivo** è trovare il numero minimo di monete tale che:

$$\sum_{i=1}^n x_i c_i = V$$

Prima di tutto specifichiamo che l'approccio **greedy** (prendi sempre la monete più grande), non funziona sempre, infatti esso fallisce se il sistema monetario non è canonico (canonico vuol dire che l'uso del taglio massimo fa sempre parte della soluzione). Dimostriamo questo fatto con un esempio:

Sia $C = \{1, 3, 4\}$ e il target $V = 6$

- **Greedy**: $4 + 1 + 1$ (3 monete)
- **Ottimo**: $3 + 3$ (2 monete)

Ora definiamo la **funzione di stato** `dp[v]` come il numero minimo di monete per il valore v .

$$dp[v] = \begin{cases} 0 & \text{se } v = 0 \\ \min_{j: c_j \leq v} \{dp[v - c_j + 1]\} & \text{se } v > 0 \end{cases}$$

Ovviamente se il valore da coprire è 0, serviranno 0 monete, e questo è il **caso base**. Altrimenti, se possiamo prendere la moneta $c_j < v$ aggiungiamo 1 (la moneta c_j che prendiamo) al numero di monete che servono per coprire il valore precedente ($v - c_j$) che avremmo già calcolato e non servirà ricalcolarlo (infatti il valore sarà in `dp[v - c_j]`). Questo è il codice che risolve Coin Change Analysis:

```
int* get_coin_set(int coins[], int n, int V, int* result_size) {
    int* dp = malloc((V + 1) * sizeof(int));
    int* parent = malloc((V + 1) * sizeof(int));
    dp[0] = 0;
    for (int i = 1; i <= V; i++) {
        dp[i] = INT_MAX;
        parent[i] = -1;
    }
    for (int v = 1; v <= V; v++)
        for (int j = 0; j < n; j++)
            if (coins[j] <= v && dp[v - coins[j]] != INT_MAX)
                if (dp[v - coins[j]] + 1 < dp[v]) {
                    dp[v] = dp[v - coins[j]] + 1;
                    parent[v] = coins[j];
                }
}
```

```

    if (dp[V] == INT_MAX) { free(dp); free(parent); return NULL; }
    // Ricostruzione nell'array di output
    *result_size = dp[V];
    int* sol = malloc((*result_size) * sizeof(int));
    for (int curr = V, i = 0; curr > 0; i++) {
        sol[i] = parent[curr];
        curr -= parent[curr];
    }
    free(dp); free(parent);
    return sol;
}

```

Quindi prima troviamo quante monete sono necessarie nel caso ottimo ($dp[V]$) e successivamente viene effettuata la ricostruzione nell'array `sol[]`.

4.12 Distanza di edit (distanza di Levenshtein)

Date due stringhe s_1, s_2 aventi rispettivamente lunghezza n e lunghezza m , l'**obiettivo** è calcolare il numero minimo di operazioni per trasformare s_1 in s_2 . Sono ammesse 3 operazioni a costo unitario:

- **Inserimento:** aggiunta di un carattere a s_1
- **Cancellazione:** rimozione di un carattere da s_1
- **Sostituzione:** cambio di un carattere di s_1 in uno di s_2

Definiamo la **funzione di stato** $dp[i][j]$ che rappresenta il costo minimo per trasformare i primi i caratteri di s_1 nei primi j caratteri di s_2 . Prima di vedere quale operazione usare, bisogna stabilire i **casi base**:

1. $dp[i][0] = i$: caso in cui si vuole trasformare una stringa di lunghezza i in una stringa vuota (lunghezza 0), quindi l'unica operazione da fare è **cancellare** tutte le lettere i . Costo unitario della cancellazione per i -volte, quindi il costo è i .
2. $dp[0][j] = j$: caso in cui si vuole trasformare una stringa vuota in una stringa di lunghezza j , quindi l'unica operazione da fare è **inserire** tutte le lettere j . Costo unitario dell'inserimento per j -volte, quindi il costo è j .

Ora vediamo cosa succede al di fuori dei casi base. Ci sono due scenari:

- $s_1[i] = s_2[j]$: i caratteri sono uguali, quindi non dobbiamo effettuare alcuna operazione. Basterà copiare il costo minimo che avevamo calcolato prima di questo carattere: $dp[i][j] = dp[i-1][j-1]$.
- $s_1[i] \neq s_2[j]$: i caratteri non sono uguali, quindi dobbiamo effettuare una delle tre operazioni consentite, per fare ciò l'algoritmo simulerà tutte e tre le operazioni e alla fine verrà scelta quella che ha costo **minimo**:
 - **cancellazione:** $dp[i-1][j]$
 - **inserimento:** $dp[i][j-1]$
 - **sostituzione:** $dp[i-1][j-1]$

Detto questo siamo pronti a vedere il codice che risolve il problema:

```

int edit_distance(char* s1, int n, char* s2, int m) {
    int dp[n + 1][m + 1];
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                int del = dp[i - 1][j];
                int ins = dp[i][j - 1];
                int sub = dp[i - 1][j - 1];
                dp[i][j] = 1 + fmin(del, fmin(ins, sub));
            }
        }
    }
}

```

```

        }
    }
}
return dp[n][m];
}

```

Un'osservazione da fare è che in C la funzione `fmin` supporta solamente 2 parametri, per questo motivo quando troviamo il minimo, al posto di scrivere `fmin(del, ins, sub)`, scriviamo `fmin(del, fmin(ins,sub))`.

Capitolo 5

Algoritmi greedy

5.1 Introduzione e definizione

Gli **algoritmi greedy** (golosi), sono una classe di algoritmi che prende sempre una soluzione ottima localmente, nella speranza che tale soluzione porterà ad una ottimalità globale. Infatti questo tipo di algoritmo non funziona sempre, ma se riusciamo ad usarlo è preferibile per motivi di semplicità ed efficienza. Vediamo degli esempi:

Abbiamo già visto nel capitolo precedente (4.11) il problema del *Coin Change Analysis*. Questo problema è un esempio perfetto per capire che un approccio **greedy** può funzionare (rendendo il problema nettamente più semplice ed efficiente), o non funzionare.

Infatti avevamo visto che se il sistema monetario non è canonico (canonico vuol dire che l'uso del taglio massimo fa sempre parte della soluzione), non era possibile utilizzare un algoritmo greedy, ma dovevamo utilizzare la programmazione dinamica che rendeva l'algoritmo molto più dispendioso e complesso.

Se invece il sistema monetario è canonico, possiamo utilizzare un approccio greedy, prendendo sempre il taglio di moneta più grande disponibile fino a raggiungere il target (vd 4.11).

La **strategia** che caratterizza un algoritmo greedy è:

- Si valutano l'insieme di opzioni disponibili.
- Si sceglie quella che sembra migliore nel breve termine (la scelta "greedy"), che porterà ad un ottimo locale.
- Si procede senza ripensare alle decisioni fatte (che è esattamente il contrario di quello che si fa con la programmazione dinamica).

Un problema si può affrontare con un approccio greedy se possiede queste due caratteristiche:

- **Greedy choice property**: una scelta locale ottimale porta ad una soluzione globale ottimale.
- **Optimal substructure**: la soluzione ottimale del problema contiene le soluzioni ottimali ai suoi sotto-problemi.

Nelle prossime sezioni vedremo problemi risolti con degli algoritmi greedy.

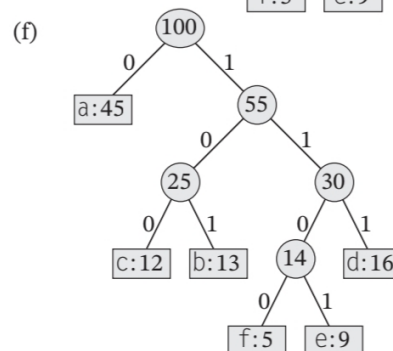
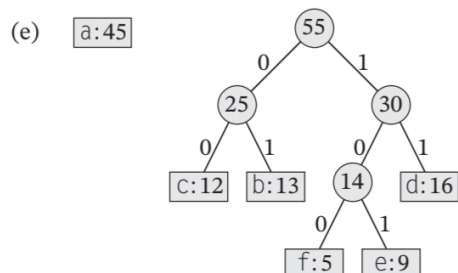
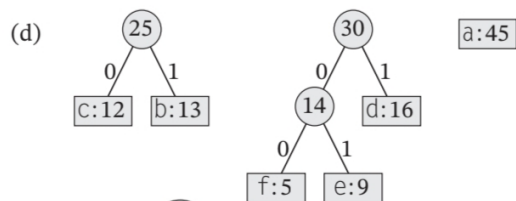
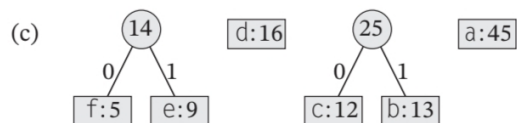
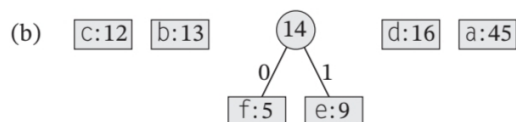
5.2 Problema della codifica di Huffman

Dato $S = \{s_1, s_2, \dots, s_n\}$ un insieme di n caratteri e che ogni carattere $c \in C$ sia presente con una certa frequenza f_i . L'**obiettivo** è:

1. Assegnare ad ogni simbolo un codice binario (0 o 1), creando un prefisso (detto *codice di Huffman*) che identificherà univocamente il simbolo.
2. Minimizzare la lunghezza della media pesata $\left(\sum^i f_i \times l_i(\text{lunghezza codice di } s_i)\right)$

Visto che a parole può risultare poco chiaro, vediamo un esempio e successivamente spieghiamo la strategia di risoluzione.

(a) f:5 e:9 c:12 b:13 d:16 a:45



Questi sono i passi dell'algoritmo di Huffman $((a) \rightarrow (b) \cdots \rightarrow (f))$, la **strategia** è la seguente:

All'inizio abbiamo i dati di input, ovvero i caratteri con la rispettiva frequenza. Consideriamo ogni dato come radice di un albero binario. A ogni passo i due alberi con le frequenze più piccole vengono fusi, quindi questi due alberi diventano figli di un nuovo nodo rappresentante la somma delle frequenze degli alberi che sono stati fusi. Ad ogni fusione vengono assegnati i codici binari, in questo caso la regola è: un arco che collega un nodo interno con i suoi figli ha l'etichetta 0 se è un arco che congiunge il figlio sinistro, ha l'etichetta 1 se è un arco che congiunge un figlio destro.

Ripetendo questo procedimento fino ad ottenere un unico albero binario abbiamo ottenuto che ogni carattere ha il proprio codice che lo rappresenta, formato dalla sequenza di codici binari (0 o 1) a partire dalla radice dell'albero completo fino alla foglia che rappresenta il carattere. In questo esempio guardando l'ultimo step (f), si ha che:

- f:5 $\rightarrow$ 1100
- e:9 $\rightarrow$ 1101
- c:12 $\rightarrow$ 100
- b:13 $\rightarrow$ 101
- d:16 $\rightarrow$ 111
- a:45 $\rightarrow$ 0

Il codice per creare questo albero è:

```
while (heap->size > 1) {
    left = extractMin(heap);
    right = extractMin(heap);
    top = createNode('$', left->freq+right->freq);
    top->left = left;
    top->right = right;
    insertMinHeap(heap, top);
}
```

Come è ben visibile dall'esempio, i simboli con le frequenze minime f_i si trovano alla massima profondità dell'albero. Visto che la **lunghezza media pesata** è calcolata come $\sum_i (f_i \times l_i)$, e che più la frequenza è alta, minore è il codice binario associato al carattere che ha tale frequenza, la lunghezza media è **minima**.

5.3 Problema di Activity selection

Il problema della selezione di attività consente di selezionare il più grande insieme di attività compatibili. Più formalmente diciamo che:

Dato $A = \{a_1, a_2, \dots, a_n\}$ un insieme di attività, dove ciascuna attività a_i ha un tempo di inizio $s_i \in \mathbb{R}$ e un tempo di fine $f_i \in \mathbb{R}$ con $s_i < f_i$, e dato il vincolo di compatibilità: a_i e a_j sono compatibili se non si sovrappongono nel tempo ($[s_i, f_i] \cap [s_j, f_j] = \emptyset$), l'**obiettivo** di trovare un sottoinsieme $A' \subseteq A$ tale che:

- Tutte le attività in A' sono compatibili.
- $|A'|$ è il massimo possibile, ovvero il numero di attività del sottoinsieme soluzione è massimo.

La **strategia** per risolvere questo problema con un approccio greedy è la seguente:

1. Ordina le attività di A per tempo di fine crescente, ovvero $f_1 \leq f_2 \leq \dots \leq f_n$.
2. Inizializza $G \leftarrow \emptyset$ e $t = 0$, dove $G = \{g_1, g_2, \dots, g_k\}$ è l'insieme delle attività selezionate dall'algoritmo greedy, all'inizio ovviamente sarà inizializzato come un insieme vuoto.
3. Realizza il ciclo che ad ogni iterazione aggiorna le variabili sotto certe condizioni, ovvero se $s_i \geq t$ aggiungo a_i a G e pongo $t \leftarrow f_i$, in questo modo scarto le attività che si sovrappongono e in G andranno solo quelle che rispettano il vincolo di compatibilità.
4. Restituisce G

Il codice quindi sarà (supponendo che gli eventi in A siano già ordinati):

```
GREEDY-ACTIVITY-SELECTOR(s, f, n, A) {  
    G = {};  
    t = 0;  
    for i = 1 to n  
        if s[i] >= t  
            G.add(A[i]);  
            t = f[i];  
    return G;  
}
```

Dimostrazione dell'ottimalità

Vogliamo dimostrare che l'approccio greedy ci dà effettivamente un risultato ottimale. Sia $G = \{g_1, g_2, \dots, g_k\}$ l'insieme delle attività selezionate dall'algoritmo greedy, sia $O = \{o_1, o_2, \dots, o_m\}$ un sottoinsieme ottimo, cioè un insieme di attività compatibili con $m \geq k$. **Lobiettivo** è dimostrare che $k = m$, cioè che l'algoritmo greedy seleziona tante attività quante ne ha una soluzione ottima. Procediamo per **induzione**:

Costruiamo un altro insieme O' tale che tutte le sue attività sono compatibili (non si sovrappongono le attività) e che $|O| = |O'|$ (soluzione ottima).

- **Passo base:**

- confrontiamo g_1 con o_1 , e avremo che $f_{g_1} \leq f_{o_1}$ per costruzione, poichè in G ci sono le attività scelte dall'algoritmo Greedy, il quale sceglie le attività che finiscono prima.
- sostituiamo o_1 con g_1 in O
- dopo la sostituzione otteniamo un insieme O' che ha tutti gli elementi di O , ma con la prima attività che è diventata g_1 , ovvero $O' = \{g_1, o_2, o_3, \dots, o_m\}$.
- Notiamo che l'insieme O' è ancora ottimo, perchè l'attività g_1 che abbiamo inserito è compatibile con tutte le altre attività dentro O' , visto che finisce prima.

- **Passo induttivo**

- supponiamo di aver già sostituito le prime r attività in O con le prime r in G , ottenendo un insieme O' ancora ottimo, dobbiamo mostrare che possiamo sostituire anche g_{r+1} a o_{r+1} , mantenendo una soluzione ottima.
- o_{r+1} essendo compatibile con g_r , deve iniziare dopo f_{g_r} , l'algoritmo greedy sceglie l'attività che finisce prima, quindi sceglie g_{r+1} .
- sostituiamo anche o_{r+1} con g_{r+1} , siccome $f_{g_{r+1}} \leq f_{o_{r+1}}$.

Alla fine di tutti i confronti abbiamo che $O' = G$, che è la soluzione ottima scelta dall'algoritmo greedy. In questo procedimento non abbiamo scartato alcuna attività, ma abbiamo solo sostituito facendo rimanere la soluzione ottima, quindi abbiamo dimostrato che $k = m$.

5.4 Confronto con altri approcci

caratteristica	greedy	programmazione dinamica	divide et impera
strategia principale	scelte locali ottimali	risoluzione e memorizzazione di sottoproblemi	divisione in sottoproblemi indipendenti
approccio	iterativo	iterativo o ricorsivo con memoization	ricorsivo
ottimalità	solo se problema ha proprietà greedy	sempre (se struttura ottimale esiste)	non garantita direttamente
dipendenza tra sottoproblemi	non considera sottoproblemi	i sottoproblemi si sovrappongono	i sottoproblemi sono indipendenti
efficienza	molto veloce ($O(n)$ spesso)	più lento ($O(n^2)$ o $O(n^3)$ tipico)	dipende dalla profondità della ricorsione
uso della memoria	basso	alto (per salvare soluzioni parziali)	variabile
esempi classici	cammini minimi (Dijkstra), Kruskal	Zaino 0/1, Fibonacci, allineamento di sequenze	MergeSort, QuickSort, Ricerca binaria
punti di forza	semplicità, velocità	precisione, correttezza garantita	suddivisione naturale di problemi complessi
punti deboli	rischio di soluzioni subottimali	maggior consumo di tempo e memoria	richiede problemi naturalmente divisibili

Capitolo 6

Ridimensionamento

A volte possiamo avere la necessità di gestire strutture dati dinamiche ossia che siano in grado di adattare la loro dimensione a fronte di inserimenti e cancellazioni

- può capitare di aver allocato una certa quantità di memoria per poi accorgersi, durante l'esecuzione del programma, che non ci basta.
- E' anche opportuno evitare spechi di memoria ossia, dopo aver rimosso molti elementi, ridurre la memoria allocandone una più piccola in cui memorizzare elementi rimasti

Strategia per la gestione dinamica

- Alloca una nuova struttura più grande/più piccola
- Copia tutti gli elementi della vecchia struttura dati nella nuova

Osservazioni

- La strategia appena descritta ha un costo proporzionale all'allocazione della nuova struttura + il costo della copia + il costo dell'inserimento/cancellazione
- Nella maggior parte delle strutture dati, il costo dominante è quello della copia
- Se il ridimensionamento avviene una tantum il suo costo può essere ammortizzato e distribuito sulle operazioni di inserimento successive
- in questo caso parleremo quindi di analisi ammortizzata del costo
- Consideriamo una struttura dati tabellare e vediamo
 - come sia possibile aggiungere/togliere un elemento in tempo ammortizzato costante $O(1)$, benché tali operazioni abbiano un costo unitario maggiore
 - come sia possibile ridimensionare garantendo che la memoria inutilizzata sia sempre inferiore ad una frazione costante della memoria allocata

6.1 tabelle dinamiche

Una tabella è una generica struttura dati che viene rappresentata in una locazione di memoria e che è divisa in celle.

Una tabella **T** abbia i seguenti attributi:

- un puntatore **T.table** alla memoria allocata;
- **T.size** che rappresenta la dimensione della tabella
- un intero **T.num** che rappresenta il numero di elementi presenti nella tabella

Le operazioni che possiamo eseguire su una tabella sono:

- **Table(T)** che crea una nuova tabella vuota T
- **Insert(T,x)** che inserisce l'oggetto x nella tabella T
- **Delete(T,x)** che rimuove l'oggetto x dalla tabella T

Ridimensionamento: espansione

Consideriamo dapprima il caso in cui vengano eseguite soltanto inserimenti di nuovi elementi e nessuna rimozione. Definiamo il fattore di carico della tabella come il rapporto $\alpha = num/size$.

Quando $\alpha = 1$ la tabella è piena ed occorre espanderla allocando nuova memoria.

Un tipico metodo euristico consiste nell'allocare una nuova tabella che ha il doppio di celle della vecchia il che garantisce un fattore di carico $\alpha > 1/2$.

L'operazione di inserimento dovrà quindi essere realizzata come segue

```
INSERT(T, x)
1 If T.size == 0
2   Alloca una nuova tabella di dimensione 1
3 If T.size == T.num
4   Alloca T' di dimensione 2*T.size e assegna al puntatore p
5   Copia tutti gli elementi di T in T'
6   Rilascia la memoria occupata da T
7   Aggiorna puntatore di T a p
8   T.table=T'
9   T.size=2*T.size
10  inserisci x in T'
11  T.num = T.num+1
```

Costo:

Assumiamo che l'allocazione e il rilascio di memoria abbiano costo costante mentre la copia dipende dalla dimensione originale della struttura dati.

La procedura di INSERT costerà quindi:

- $O(1)$ se non devo espandere la tabella
- $O(T.size)$ se devo procedere con l'espansione

Analisi del costo ammortizzato

Consideriamo il costo di una sequenza di n **Insert** eseguite a partire da una tabella vuota

- Il costo della k-esima Insert è 1 se non vi è espansione ed è k se vi è espansione

Siccome T.size è sempre una potenza di 2 e l'espansione si ha quando T.num=k-1 è uguale a T.size, il costo della k-esima Insert è:

$$\begin{cases} k & \text{se } k-1 \text{ è una potenza di } 2 \\ 1 & \text{negli altri casi} \end{cases}$$

e il costo totale di n Insert è:

$$C(n) = \sum_{k=1}^n c_k = n + \sum_{j=0}^{\lfloor \log n \rfloor} 2^j < n + 2n = 3n$$

se adesso dividiamo il costo totale di n operazioni per il numero di operazioni otteniamo il costo ammortizzato $O(3n)/n = O(1)$

Ridimensionamento: Contrazione

Quando cancelliamo elementi, la struttura dati usa sempre più memoria di quanta in realtà gliene serve. Per evitare uno spreco eccessivo di memoria è opportuno contrarre la tabella quando il fattore di carico $\alpha = \text{num}/\text{size}$ diventa troppo piccolo.

La contrazione della tabella è simile all'espansione: quando il numero di elementi in essa diventa troppo piccolo, allochiamo una nuova tabella più piccola e poi copia gli elementi della vecchia in quella nuova. Possiamo rilasciare lo spazio in memoria per la vecchia tabella restituendo al sistema di gestione della memoria.

Strategia naive

Dimezzare la memoria quando il fattore di carico α diventa $\leq 1/2$

Questo ci assicura un fattore di carico $\alpha > 1/2$ anche in presenza di **Delete**

Purtroppo con questa strategia il costo ammortizzato delle operazioni non è più costante.

Consideriamo una successione costituita da 2^k **Insert** seguite da 2^{k-2} gruppi di quattro operazioni

Insert - Delete - Delete - Insert

per un totale di $n = 2^{k+1}$ operazioni.

In ogni gruppo la prima Insert comporta una espansione di costo 2^k e la seconda Delete una contrazione di costo 2^k .

Quindi ogni gruppo ha costo $\geq 2^{k+1}$

e in totale i 2^{k-2} gruppi hanno costo:

$$\geq 2^{k-2} 2^{k+1} = \frac{n^2}{8} = \Omega(n^2)$$

e il costo ammortizzato è $\Omega(n^2)/n = \Omega(n)$.

Utilizziamo come criterio per ridimensionare $\alpha = 1/2$ le sequenze di contrazioni non mi consentono di ammortizzare i costi degli inserimenti come vorrei.

Per le contrazioni quindi ho bisogno di svuotare a sufficienza la tabella per bilanciare il costo degli inserimenti occorre quindi aspettare che α diventa $1/4$.

Capitolo 7

Hash

7.1 Dizionari

I dizionari sono strutture dati fondamentali che consentono di rappresentare una relazione tra un insieme di chiavi D , il dominio, e un insieme di valori C , il codominio. Questa associazione chiave-valore è comunemente utilizzata in diverse applicazioni come le rubriche telefoniche, dove ogni numero di telefono (chiave) è associato a un nome (valore). Le operazioni principali che devono essere supportate efficientemente da un dizionario sono:

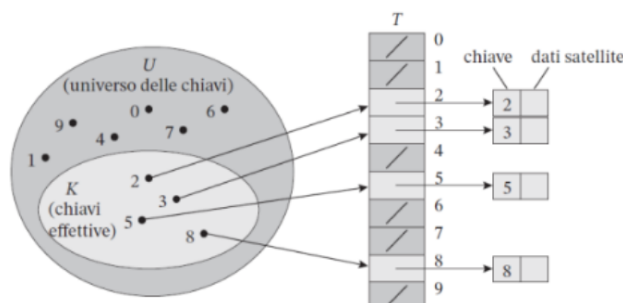
- **Insert:** aggiunge un nuovo valore
- **Search:** restituisce il valore associato a una chiave
- **Delete:** elimina un valore dalla struttura

Esistono diversi modi per implementare un dizionario, tra cui:

- tabelle ad indirizzamento diretto
- tabelle di hash

7.2 Tabelle ad indirizzamento diretto

L'indirizzamento diretto è una tecnica efficace quando U , l'universo delle chiavi, è ragionevolmente piccolo, ovvero dato $U = 0, 1, \dots, m - 1$, con m non troppo grande. Le tabelle ad indirizzamento diretto utilizzano un array per gestire le chiavi e rappresentare l'insieme dinamico in cui ogni elemento ha associata una chiave estratta dall'universo U . Supponendo che due elementi non possano avere la stessa chiave, si può rappresentare l'insieme dinamico utilizzando un array o tavola a indirizzamento diretto indicata con $T[0 \dots m - 1]$, dove ogni posizione o cella dell'array è associata a una chiave specifica, e il valore associato ad essa è memorizzato nell'array. La cella k punta a un elemento dell'insieme con chiave k . Se nessun elemento è associato ad una specifica chiave k , allora $T[k] = NIL$.



Operazioni su una tabella ad indirizzamento diretto

Le operazioni su una tabella ad indirizzamento diretto sono le seguenti:

- **Search(T, k):** restituisce $T[k]$;
- **Insert(T, x):** imposta $T[x.key] = x$;

- **Delete(T, x):** imposta $T[x.key] = nil$

Analisi

I principali **vantaggi** delle tabelle a indirizzamento diretto includono operazioni di inserimento, ricerca e cancellazione molto efficienti in tempo $O(1)$.

Bisogna notare quanto sia vantaggiosa la possibilità di evitare di memorizzare la chiave e i dati satelliti di un elemento in un oggetto esterno puntato da una cella dell'array T , al contrario è possibile memorizzare l'oggetto nella cella stessa, risparmiando spazio e usando l'indice come chiave.

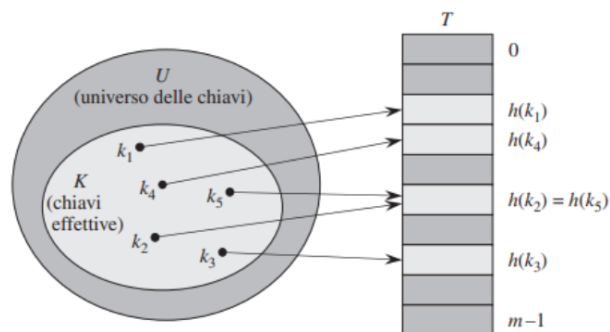
Tuttavia, presentano anche significativi **svantaggi**, infatti con l'indirizzamento diretto si è "costretti" ad allocare memoria per una tabella T di dimensione $|U|$, ovvero pari alla cardinalità dell'insieme dell'universo delle chiavi. Se l'universo è troppo grande può essere impraticabile allocare così tanta memoria, inoltre potrebbe succedere che l'insieme K delle chiavi effettivamente memorizzate sia così piccolo rispetto a U che la maggior parte dello spazio allocato per la tabella T sarebbe sprecato.

Fattore di carico

Il fattore di carica α misura il grado di riempimento di una tabella ed è definito come $\alpha = n/m$ dove $n = |K|$ è il numero di chiavi effettivamente utilizzate e $m = |U|$ la dimensione della tabella.

7.3 Tabelle o tavole di Hash

Le tabelle hash rappresentano un'evoluzione delle tabelle ad indirizzamento diretto, utilizzando una funzione di hash per mappare le chiavi alle posizioni dell'array. Se con l'indirizzamento diretto un elemento con chiave k è memorizzato nella cella k , con l'hashing, questo elemento è memorizzato nella cella $h(k)$, ovvero si utilizza una funzione di hash h per calcolare la cella della chiave k . Dunque la funzione h associa ogni chiave k a una posizione $h(k)$ nell'array.



La funzione hash $h : U \rightarrow 0, \dots, m-1$ associa l'universo U delle possibili chiavi alle celle di una tabella di hash $T[0 \dots m-1]$ dove la dimensione m della tabella è generalmente più piccola di $|U|$, in questo modo la dimensione dell'array viene ridotta e sarà proporzionale al numero massimo di chiavi presenti nel dizionario.

Collisioni

Tuttavia, essendo $|U| > m$, si verificano collisioni quando due chiavi diverse, $k_1 \neq k_2$, vengono mappate alla stessa posizione $h(k_1) = h(k_2)$.

Il principio della PICCIONAIA

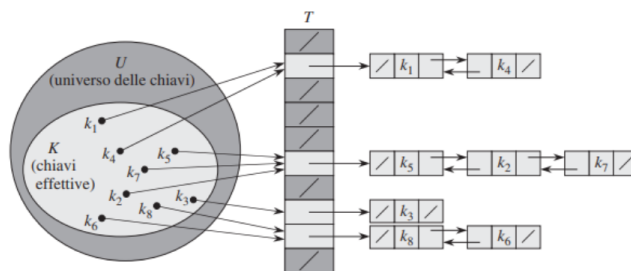
Se p piccioni devono trovare posto in c caselle e ci sono più piccioni che caselle allora in qualche casella entreranno almeno due piccioni.

Gestione delle collisioni

La gestione delle collisioni è cruciale per l'efficienza delle tabelle hash. Esistono due metodi principali:

- Metodo del concatenamento
- Metodo dell'indirizzamento aperto

Metodo del concatenamento (Liste di trabocco)



L'insieme di input di n elementi viene diviso casualmente in m sottoinsiemi, ciascuno di cardinalità n/m . Una funzione di hash determina a quale sottoinsieme appartiene un elemento, ovvero associa ad ogni elemento una chiave, per gestire i vari sottoinsiemi (chiavi) si realizzano delle liste collegate. Immaginiamo di avere più chiavi $k_1 \neq k_2 \neq k_3$ mappate su una stessa cella j dell'array T , questa ha un puntatore alla testa della lista concatenata che conterrà tutti gli elementi memorizzati in j , se non ci sono elementi, la cella j contiene la costante NIL . Per concatenare le liste associate a chiavi mappate alla stessa posizione si usufruisce di un doppio puntatore.

Operazioni sulle liste di trabocco

Le operazioni su una tabella con liste di trabocco sono le seguenti:

- **Search(T, k):** cerca nella lista $T[h(k)]$ un elemento x tale che $x.key == k$ e lo restituisce
- **Insert(T, x):** aggiunge x nella lista $T[h(x.key)]$
- **Delete(T, x):** toglie x dalla lista $T[h(x.key)]$

Fattore di carico

Il concetto di fattore di carico α per il metodo del concatenamento può essere ridefinito come segue: data una tavola hash T con m celle dove sono memorizzati n elementi, si definisce fattore di carico α come il rapporto n/m , ossia il numero medio di elementi memorizzati in una lista.

Analisi

L'analisi sarà fatta in funzione del fattore di carico α , che può essere minore, uguale o maggiore di 1. Il caso peggiore si presenta quando tutte le n chiavi sono associate alla stessa cella, così creando una lista di lunghezza n con operazioni che richiedono il seguente tempo di esecuzione:

- **Search(T, k):** $\Theta(n)$ + tempo di calcolo della funzione di hash
- **Insert(T, x):** $O(1)$ con l'assunzione che x non sia già nel dizionario, se questa assunzione non valore allora si paga il costo di una precedente search
- **Delete(T, x):** $O(1)$ se le liste sono doppiamente concatenate, ovvero quando ogni nodo della lista ha un campo *next* che punta al nodo successivo e un campo *prev* che punta al nodo precedente, altrimenti il costo di una search

Le prestazioni dell'hashing nel caso medio dipendono dal modo in cui la funzione hash h distribuisce l'insieme delle chiavi da memorizzare tra le m celle.

Supponiamo che qualsiasi elemento abbia la stessa probabilità di essere mandato in una qualsiasi delle m celle, indipendentemente dalle celle in cui sono mandati gli altri elementi, si tratta così l'ipotesi di **hash uniforme indipendente**.

Teoremi metodo del concatenamento

Teorema 1

Teorema 7.3.1. In una tabella hash le cui collisioni sono risolte tramite la tecnica del concatenamento, una ricerca senza successo richiede in media un tempo $\Theta(1 + \alpha)$, nell'ipotesi di hashing uniforme e indipendente.

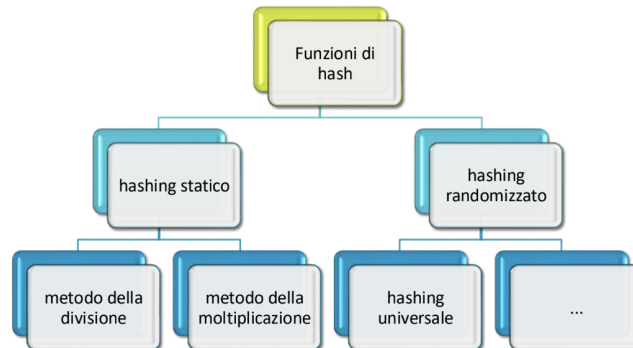
Teorema 2

Teorema 7.3.2. In una tabella hash le cui collisioni sono risolte tramite la tecnica del concatenamento, una ricerca con successo richiede in media un tempo $\Theta(1 + \alpha)$, nell'ipotesi di hashing uniforme e indipendente.

Corollario

Corollario 1. Utilizzando l'hashing universale e risolvendo le collisioni tramite concatenamento in una tabella inizialmente vuota di m celle, il tempo atteso per gestire qualsiasi sequenza di s operazioni Insert, Search e Delete contenente $n = O(m)$ operazioni INSERT è $\Theta(s)$

Funzioni di hash



Esistono degli schemi per poter progettare e realizzare delle buone funzioni di hash, due sono di tipo euristico, ovvero l'hashing per divisione e quello per moltiplicazione, mentre il terzo usa la randomizzazione ed è denominato hashing universale. Una buona funzione di hash soddisfa l'ipotesi dell'hashing uniforme indipendente: ogni chiave ha la stessa probabilità $1/m$ di essere mandata in una qualsiasi delle m celle, indipendentemente dalle chiavi inserite precedentemente.

Se le chiavi sono rappresentate da numeri reali x estratti casualmente e indipendentemente da una distribuzione di probabilità uniforme nell'intervallo $0 \leq x < 1$, la funzione di hash $h(x) = \lfloor m \cdot x \rfloor$ soddisfa l'ipotesi di hash uniforme indipendente.

Poiché questa ipotesi dipende dalle probabilità con cui vengono estratti gli elementi da inserire e, questa probabilità in genere non è nota, si assume che l'universo delle chiavi sia l'insieme dei numeri naturali interi non negativi. Questa assunzione non è restrittiva, in quanto ogni tipo di chiave è rappresentato nel calcolatore con una sequenza di bit e ognuna di esse si può interpretare come un intero non negativo.

Hashing Statico

Metodo divisione

Associa ad una chiave k una delle m celle prendendo il resto della divisione intera tra k e m , ovvero

$$h(k) = k \mod m$$

Per esempio, se la tavola hash ha dimensione $m = 12$ e la chiave è $k = 100$, allora $h(k) = 4$. Quando si utilizza questo metodo vengono evitati certi valori di m , ad esempio m non dovrebbe essere una potenza di 2, perché se $m = 2 \cdot p$, allora $h(k)$ rappresenta proprio i p bit meno significativi di k . Una buona scelta per m è un numero primo non troppo vicino ad una potenza esatta di 2. Ad esempio $m = 701$ perché è un numero primo vicino a $2000/3$, ma non a una potenza di 2.

Metodo della moltiplicazione

Questo metodo si svolge in due passi, prima moltiplica la chiave k per una costante A (composta tra 0 e 1) e si estrae la parte frazionaria di $k \cdot A$. Dopodiché si moltiplica il valore ottenuto per m e prende la parte intera inferiore del risultato. La funzione di hash è

$$h(k) = \lfloor m(kA \mod 1) \rfloor$$

dove " $k \cdot A \mod 1$ " rappresenta la parte frazionaria di $k \cdot A$, cioè $k \cdot A - \lfloor k \cdot A \rfloor$. Un vantaggio del metodo della moltiplicazione è che il valore di m non è critico.

Tipicamente, si sceglie m pari ad una potenza di 2 ($m = 2^p$ per qualche intero p).

Supponiamo che ogni chiave k possa essere rappresentata da una sola parola di w bit.

Come A prendiamo una frazione della forma $q/2^w$, dove quando q è un intero nell'intervallo $0 < q < 2^w$. Prima si moltiplica k per l'intero di w bit $q = A \cdot 2^w$. Il risultato è un valore di $2 \cdot w$ bit $r_1 2^w + r_0$ dove r_1 è la parte più significativa del prodotto e r_0 è la parte meno significativa del prodotto. Il valore hash desiderato di p bit

è formato dai p bit più significativi di r_0 .

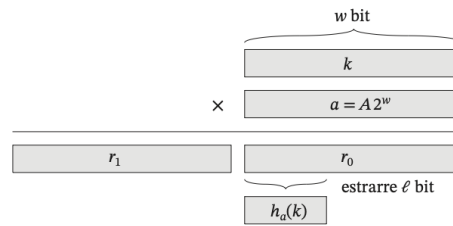


Figura 11.4 Il metodo moltiplica e scorri per calcolare una funzione di hash. Il valore della chiave k , rappresentato con w bit, viene moltiplicato per il valore $s = A \cdot 2^w$, anch'esso rappresentato con w bit. Gli ℓ bit più significativi della metà meno significativa del prodotto formano il valore di hash $h_a(k)$ desiderato.

Warning

Nessuna funzione di hash può evitare che un avversario malevolo inserisca nella tabella una sequenza di valori che vadano a finire tutti nella stessa lista

Più in generale, per ogni funzione di hash si possono trovare delle distribuzioni di probabilità degli input per le quali la funzione non ripartisce bene le chiavi tra le varie liste della tabella hash

Hashing Randomizzato

Possiamo usare la randomizzazione per rendere il comportamento della tabella hash indipendente dall'input. L'idea è quella di usare una funzione di hash scelta casualmente in un insieme "universale" di funzioni di hash. Questo approccio viene detto **hashing universale**.

Definizione Hashing Universale

Definizione 7.3.1. Un insieme H di funzioni di hash che associano un dato universo U di chiavi nell'insieme $0, 1, \dots, m-1$ degli indici della tabella hash si dice universale se, per ogni coppia di chiavi diverse j e k vi sono al più $|H|/m$ funzioni di hash in H tali che $h(j) = h(k)$

In altre parole, con una funzione di hash scelta a caso da H , la probabilità di collisione fra due chiavi distinte j e k non è maggiore della probabilità $1/m$ di una collisione nel caso in cui $h(k)$ e $h(j)$ fossero scelte in modo casuale e indipendente dall'insieme degli indici della tabella hash (hashing universale indipendente).

Progettare una famiglia universale di funzioni di hash - Metodo basato sulla teoria dei numeri

Iniziamo scegliendo un numero primo p sufficientemente grande in modo che sia maggiore di ogni possibile chiave k . Supponendo che la dimensione dell'universo delle chiavi sia maggiore del numero di celle nella tavola hash, abbiamo $p > m$. Adesso definiamo la funzione hash $h_{a,b}$ per ogni coppia di interi (a, b) tali che $1 \leq a < p$ e $0 \leq b < p$ come segue, ovvero utilizzando una trasformazione lineare seguita da riduzioni modulo p e poi modulo m :

$$h_{a,b}(k) = [(ak + b) \mod p] \mod m$$

La famiglia di tutte queste funzioni hash è

$$H_{pm} = h_{a,b} : 1 \leq a < p, 0 \leq b < p$$

con p numero primo, ed è universale.

Teorema

Teorema 7.3.3. La famiglia di funzioni $H = h_a : a$ è dispari, $1 < q < m$ dove $h_a(k) = (ka \mod 2^W)$ ottenuta tramite metodo moltiplica e scorri è $2/m$ - universale

7.4 Metodo dell'indirizzamento aperto

Nell'indirizzamento aperto, tutti gli elementi sono memorizzati nella tavola hash stessa, ovvero ogni cella della tavola contiene un elemento dell'insieme dinamico o la costante NIL . Non si utilizzano puntatori a zone esterne

di memoria, motivo per il quale un inserimento potrebbe non andare a buon fine a causa del fatto che la tabella potrebbe essere piena, perciò il fattore di carica α non supera mai 1. Per effettuare un inserimento si esaminano in successione le posizioni della tabella hash (ispezione), finché non si trova una cella vuota in cui inserire la chiave. Anziché seguire sempre lo stesso ordine $0, 1, \dots, m - 1$, la sequenza delle posizioni esaminate durante un'ispezione dipende dalla chiave da inserire. Per tal motivo, la funzione di hash è una funzione $h(k, i)$ che al variare di i tra 0 e $m - 1$ fornisce, per ciascuna chiave k , una sequenza di indici $h(k, 0), h(k, 1), \dots, h(k, m - 1)$ che rappresenta l'ordine di ispezione. Volendo poter ispezionare tutte le celle, la sequenza deve essere una permutazione dell'insieme degli indici della tabella.

Ricorda: nello pseudocodice di Insert si suppone che gli elementi della tavola hash T siano chiavi senza dati satelliti, ovvero la chiave k è identica all'elemento che contiene la chiave k . Ogni cella contiene una chiave o la costante NIL (se la cella è vuota).

Ricorda: quando cancelliamo una chiave della cella i , non possiamo semplicemente marcare questa cella come vuota inserendovi la costante NIL . Così facendo, potrebbero essere interrotte le sequenze di ispezione. Una soluzione consiste nel marcare la cella registrandovi il valore speciale **DELETED**, anziché NIL . Di conseguenza è necessario adattare la funzione Insert.

Pseudocodice

```

Insert(T, k){
    i = 0
    while i != m
        j = h(k,i)
        if T[j] == nil or T[j] == DELETED
            T[j] = K
            return j
        else
            i = i + 1
    return "tabella piena"
}

Search(T, k){
    i = 0
    while i != m and T[q] != nil
        q = h(k,i)
        if T[q] == K
            return q
        i = i + 1
    return nil
}

Delete(T, i){
    T[i] = DELETED
}

```

Con l'indirizzamento aperto, la funzione di hash fornisce una sequenza di ispezione, pertanto si modifica l'ipotesi di hashing indipendente e uniforme si parla di hashing con permutazioni indipendenti e uniformi. Per l'**hashing uniforme** si suppone che ogni chiave abbia la stessa probabilità $1/m!$ di generare una qualsiasi delle $m!$ possibili sequenze di ispezione.

Esistono tre tecniche comunemente utilizzate per calcolare le sequenze di ispezione richieste dall'indirizzamento aperto:

1. Ispezione lineare
2. Ispezione quadratica
3. Doppio hashing

Nessuna delle tre genera tutte le $m!$ sequenze di ispezione, le prime due ne generano soltanto m e l'ultima ne genera m^2

Ispezione lineare

La funzione di hash $h(k, i)$ si ottiene da una funzione di hash ordinaria $h'(k)$ ponendo

$$h(k, i) = [h'(k) + i] \mod m$$

L'esplorazione inizia dalla cella $h(k, 0) = h'(k)$ e continua con le celle $h'(k) + 1$, $h'(k) + 2$, ecc. fino ad arrivare alla cella $m - 1$, dopodiché si continua con le celle $0, 1$, ecc.. fino ad aver percorso circolarmente tutta la tabella. L'ispezione lineare è facile da implementare ma soffre del problema noto come **addensamento primario (clustering)**: si formano lunghe file di celle occupate, che aumentano il tempo medio di ricerca. Ciò significa che i nuovi elementi inseriti nella tabella tendono ad addensarsi attorno agli elementi già presenti.

Ispezione quadratica

La funzione di hash $h(k, i)$ si ottiene da una funzione di hash ordinaria $h'(k)$ ponendo

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \mod m$$

dove c_1 e c_2 sono due costanti con $c_2 \neq 0$

I valori di m , c_1 e c_2 non possono essere qualsiasi ma devono essere scelti opportunamente in modo che la sequenza di ispezione percorra tutta la tabella.

Ispezione con doppio hashing

La funzione di hash $h(k, i)$ si ottiene da due funzioni di hash ordinarie $h_1(k)$ ed $h_2(k)$ ponendo

$$h(k, i) = [h_1(k) + i h_2(k)] \mod m$$

Perché la sequenza di ispezione percorra tutta la tabella, il valore di $h_2(k)$ deve essere relativamente primo con m . E' possibile soddisfare questa condizione in diversi modi:

1. Scegliendo $m = 2^p$ e definendo $h_2(k) = 2h'(k) + 1$ in modo che produca sempre un numero dispari, con $h'(k)$ funzione di hash qualsiasi per una tabella di dimensione $m' = m/2 = 2^{p-1}$
2. Scegliendo m primo e definendo $h_2(k)$ in modo che generi sempre un numero intero positivo minore di m . Ad esempio, $h_1(k) = k \mod m$, $h_2(k) = 1 + (k \mod m')$, dove m' è minore di m , di solito $m' = m - 1$

Tabella hash Adattiva

Indipendentemente dalla strategia per la gestione delle collisioni, non conviene far crescere troppo α . Possiamo sostituire l'array con un «array dinamico»

- Sopra una certa soglia di carico, si raddoppia la dimensione dell'array
- Si reinseriscono tutte le chiavi
- Nel caso di indirizzamento aperto, in fase di nuovo inserimento si rimuovono anche tutti gli elementi marcati DELETED

Il costo ammortizzato è $O(1)$

Capitolo 8

Grafi

Un grafo è una generalizzazione del concetto di albero, infatti questo è un tipo speciale di grafo. In particolare, un albero è un grafo connesso e aciclico. Questo significa che in un albero non ci sono cicli (percorsi che iniziano e finiscono nello stesso nodo) e che tutti i nodi sono collegati da esattamente un percorso. Se nei grafi da ogni vertice possono partire e giungere più archi, negli alberi ogni nodo ha al massimo un padre. Un grafo viene definito come una struttura matematica in grado di rappresentare relazioni tra oggetti. È composto da un insieme di nodi (o vertici) e un insieme di archi che collegano coppie di nodi. I nodi rappresentano entità, mentre gli archi rappresentano le relazioni tra di esse.

Definizioni sui Grafi

Definizione 8.0.1. Un grafo $G = (V, E)$ è costituito da un insieme di vertici V ed un insieme di archi E ciascuno dei quali connette due vertici V detti estremi dell'arco.

Definizione 8.0.2. Un grafo è **orientato** quando esiste una relazione d'ordine tra gli estremi degli archi, ovvero gli archi hanno un verso e una direzione definita, espressi tramite una freccia. Immaginiamo un arco come una freccia che va dalla coda alla testa, infatti il primo estremo si dice coda ed il secondo testa.

Definizione 8.0.3. Un grafo si dice **denso** se il numero di archi presenti è molto vicino al numero totale di possibili archi.

Definizione 8.0.4. Un **cappio** è un arco i cui estremi coincidono.

Definizione 8.0.5. Un grafo orientato è **semplice** se non ci sono cappi e non ci sono due archi con gli stessi estremi. Se esistessero due archi con gli stessi estremi si definirebbe **multigrafo**.

Definizione 8.0.6. Se un grafo è semplice possiamo identificare un arco con la coppia dei suoi estremi: $e = uv \in E$, in questo caso diciamo che l'arco e è **incidente** in u e in v . Se il grafo è orientato, la coppia uv è ordinata. In questo caso diciamo che l'arco e **esce** da u ed **entra** in v . Se $e = uv \in E$ diciamo che il vertice v è **adiacente** al vertice u . Se il grafo non è orientato la relazione di adiacenza è simmetrica.

Definizione 8.0.7. Il **grado** $\delta(v)$ del vertice v è il numero di archi incidenti in v . Se il grafo è orientato abbiamo:

- grado entrante $\delta^-(v)$ che rappresenta il numero di archi entranti in v
- grado uscente $\delta^+(v)$ che rappresenta il numero di archi uscenti in v

Un nodo **pozzo** è un nodo che ha grado uscente pari a zero, $\delta^+(v) = 0$.

Definizione 8.0.8. Un **percorso** π di lunghezza k dal vertice u al vertice v in un grafo $G = (V, E)$ è una sequenza di $k + 1$ vertici $\pi = x_0, x_1, x_2, \dots, x_k$ tali che $x_0 = u, x_k = v$ e $x_{i-1}x_i \in E$ per $i = 1, \dots, k$. Il percorso $\pi = x_0$ ha lunghezza $k = 0$. Se $k > 0$ e $x_0 = x_k$ diciamo che il percorso è **chiuso**.

Definizione 8.0.9. Un cammino è un percorso i cui vertici $x_0, x_1, \dots, x_k$ sono tutti diversi con la possibile eccezione di $x_0 = x_k$, nel qual caso esso è un **ciclo**.

Un ciclo di lunghezza $k = 1$ è un **cappio**.

Un **grafo aciclico** è un grafo che non contiene cicli.

Quando esiste almeno un cammino dal vertice u al vertice v diciamo che il vertice v è **accessibile** o raggiungibile da u .

Definizione 8.0.10. Un grafo non orientato si dice **connesso** se esiste almeno un cammino tra ogni coppia di vertici.

Le componenti connesse di un grafo sono le classi di equivalenza dei suoi vertici rispetto alla relazione di accessibilità. Quando si parla di componente connessa di un grafo, si fa riferimento ad un gruppo di nodi del grafo in cui esiste un percorso da qualsiasi nodo del gruppo verso qualsiasi altro nodo del gruppo. La "relazione di accessibilità" a cui si fa riferimento è semplicemente la possibilità di raggiungere un nodo da un altro attraverso uno o più archi nel grafo. Le "classi di equivalenza" a cui si fa riferimento sono insiemi di nodi che sono considerati equivalenti in termini di accessibilità. Dunque, due nodi appartengono alla stessa classe di equivalenza se e solo se sono raggiungibili tra loro attraverso archi nel grafo. Nei grafi orientati la proprietà di raggiungibilità non è simmetrica, quindi va verificata in entrambi gli estremi del cammino.

Definizione 8.0.11. Un grafo orientato si dice **fortemente connesso** se esiste almeno un cammino da ogni vertice u ad ogni altro vertice v .

Definizione 8.0.12. Un **sottografo** del grafo $G = (V, E)$ è un grafo $G' = (V', E')$ tale che $V' \subseteq V$ e $E' \subseteq E$. Il sottografo di $G = (V, E)$ indotto da $V' \subseteq V$ è il grafo $G' = (V', E')$ tale che $E' = uv : uv \in E$ e $u, v \in V'$.

8.1 Rappresentazione di un grafo

Ci sono due modi per rappresentare un grafo $G = (V, E)$: con le liste di adiacenza oppure con la matrice di adiacenza. La rappresentazione di tale grafo mediante le liste delle adiacenza utilizza liste $Adj[u]$ con $u \in V$, in cui vengono memorizzati i vertici adiacenti al vertice u .

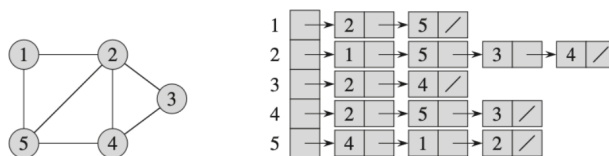


Figura 8.1: caption

Nella rappresentazione di $G = (V, E)$ mediante la matrice di adiacenza assumiamo che i vertici siano numerati da 1 fino a $|V|$, la rappresentazione è costituita da una matrice booleana A tale che:

$$\begin{cases} 1 & \text{se } ij \in E \\ 0 & \text{se } ij \notin E \end{cases}$$

Esempio con il grafo precedente:

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Figura 8.2: caption

Una matrice di adiacenza di un grafo non orientato è sempre simmetrica. Per quanto concerne l'efficienza di queste due rappresentazioni, bisogna considerare che la lista di adiacenza richiede spazio in memoria pari a $|V| + |E|$, dove $|V|$ indica la cardinalità di V (il numero di vertici = il numero di puntatori alle liste collegate),

mentre $|E|$ indica il numero degli elementi delle liste. La rappresentazione mediante matrice di adiacenza invece richiede spazio per una matrice di dimensione $|V|^2$ questo significa che se un grafo è denso converrà utilizzare la matrice delle adiacenze.

8.2 Visita in ampiezza BFS

Dato un grafo $G = (V, E)$ e un vertice distinto s , detto sorgente la visita in ampiezza BFS (breadth-first search) ispeziona gli archi di G per visitare tutti i vertici che sono raggiungibili da s .

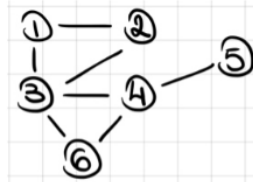


Figura 8.3: caption

Si parte dal nodo sorgente, la cui scelta è arbitraria, per poi propagarsi in ampiezza; dunque, preso 1 gli attribuisco la distanza da sè stesso, ovvero zero. Al secondo passo allargo l'area fino ad includere i nodi 2 e 3, che inserisco in una coda ed incremento di uno la distanza dalla sorgente. Dopodichè espando l'area ai vicini di questi ultimi nodi visitati, così via fino a visitare tutto il grafo.

Produce l'albero di visita in ampiezza i cui rami sono cammini di lunghezza minima.

Si espande uniformemente la frontiera tra i vertici scoperti e quelli non ancora scoperti, in altre parole, scopre tutti i vertici a distanza k da s prima di scoprire quelli a distanza $k + 1$

Per mantenere traccia del punto a cui si è arrivati nell'esecuzione dell'algoritmo, i vertici sono colorati:

- bianco (vertici non ancora raggiunti)
- grigio (vertici raggiunti che stanno sulla frontiera)
- nero (vertici raggiunti che non stanno sulla frontiera)

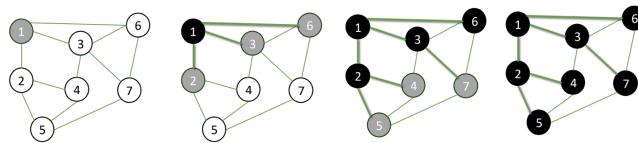


Figura 8.4: caption

Pseudocodice BFS

```

1  BFS (G, S)  // S = sorgente, G = grafo
2  foreach u in G.V = {S} // per ogni nodo appartenente al grafo, escludendo la sorgente
3      u.color = "BIANCO"
4      u.d = infinity
5      u.p = nil
6  end
7  s.color = "GRIGIO"
8  s.d = 0
9  s.p = nil // il padre della sorgente non e' nessuno
10 Q = 0
11 ENQUEUE (Q, S) // inserisco nella coda la sorgente
12 while Q != 0
```

```

13     u = DEQUEUE (Q) // visito il nodo u e lo tolgo dalla frontiera
14     foreach v in G.adj[u] // scansiono la lista delle adiacenze del vertice u
15         if v.color = "BIANCO"
16             v.color = "GRIGIO" // aggiungo alla frontiera ogni vertice v della lista di u
17             v.d = u.d + 1
18             v.p = u
19             ENQUEUE (Q, v) // metto v in coda
20         end
21     end
22     u.color = "NERO"
23 end

```

Analisi

La visita viene detta in ampiezza perchè l'algoritmo espande uniformemente la frontiera; infatti, scopre tutti i vertici a distanza k dalla sorgente s prima di scoprire quelli a distanza $k+1$. Per mantenere traccia dell'esecuzione dell'algoritmo i vertici vengono colorati di bianco (vertici non raggiunti), grigio (vertici raggiunti nella frontiera) e nero (vertici raggiunti che non sono nella frontiera). Inizialmente l'algoritmo effettua un ciclo preliminare su ogni vertice esclusa la sorgente del grafo, modificandone il colore in "BIANCO", ponendo la distanza di ogni vertice dalla sorgente ad infinity e assegnando come predecessore null. Dopodichè vengono modificati i campi della sorgente, ponendo il colore a "GRIGIO" in quanto dobbiamo visitarlo e quindi inserendolo nella frontiera e ponendo la sua distanza dalla sorgente pari a 0 ed infine aggiungo il vertice sorgente alla coda, perchè, come detto, è stato aggiunto alla frontiera. In seguito, viene svolto un ciclo while sulla coda Q dove si itera su ogni nodo. Inizialmente viene richiamata la funzione *DEQUEUE* che restituisce il nodo appena rimosso dalla coda, questo sta a significare che sto visitando il nodo appena rimosso. Dopodichè effettuo un ciclo su tutti i nodi della lista delle adiacenze del nodo u (ovvero tutti i nodi che sono collegati al nodo u) aggiungendoli in lista solo se essi sono colorati di "BIANCO" ovvero che non sono presenti in coda e quindi nella frontiera, incrementando la loro distanza dalla sorgente e ponendo il loro colore a "GRIGIO" in quanto li inserisco nella frontiera. Procedendo con questo ragionamento si vanno a visitare per ampiezza tutti i vertici del grafo. La complessità dell'algoritmo BFS è $O(|V| + |E|)$, dove $|V|$ è il numero di vertici del grafo e $|E|$ è il numero di archi del grafo.

Per dimostrare la correttezza di BFS ci occorre prima dimostrare 3 proprietà intermedie dell'algoritmo:

- proprietà delle distanze
- proprietà del limite superiore
- proprietà della coda

Proprietà delle distanze

Sia $G = (V, E)$, un grafo e $s \in V$ un vertice arbitrario. Allora per ogni arco $(u, v) \in E$ si ha che:

$$d(s, v) \leq d(s, u) + 1$$

Dimostrazione.

Per dimostrare questa proprietà dobbiamo considerare due casi:

1. u non è raggiungibile da s
2. u è raggiungibile da s

Caso 1

E' noto come nell'algoritmo BFS vengono inizializzate ad ∞ le distanze di tutti i nodi dalla sorgente. Se u non è raggiungibile da s , significa che non verrà mai aggiunto alla coda e la sua distanza $u.d$ da s rimarrà ∞ . Se u non è raggiungibile da s , non lo è neanche v allora $v.d = \infty$, che verifica la disequaglianza.

Caso 2

Se u è raggiungibile da s , significa che $u.d$ viene aggiornato con la distanza da u ad s che chiameremo $d(s, u)$, se questo cammino passa già per v , allora banalmente è verificata $d(s, v) < d(s, u)$, se invece il cammino non passa per v , allora aggiungendo l'arco (u, v) si ha che $d(s, v) = d(s, u) + 1$, da cui segue la disequaglianza $\square$

Proprietà del limite superiore

Sia $G = (V, E)$ un grafo e supponiamo che BFS venga eseguita da un nodo $s \in V$ arbitrario (sorgente). Allora per ogni vertice $v \in V$, la distanza $v.d$ calcolata da BFS è sempre tale che:

$$v.d \geq d(s, v)$$

Dimostrazione.

Per dimostrare questa proprietà dobbiamo farlo per induzione

Passo base:

Il passo base è rappresentato dall'inizializzazione della BFS nella quale tutte le distanze dei vertici del grafo dalla sorgente sono poste ad ∞ , quindi si ha banalmente che per ogni nodo u diverso da s :

$$u.d = \infty \geq d(s, u)$$

Per quanto riguarda la sorgente viene posta $s.d = 0$ che banalmente è uguale a $d(s, s)$. Quindi possiamo concludere dicendo che il passo base verifica la proprietà del limite superiore.

Passo induttivo:

Supponiamo che la disuguaglianza $u.d \geq d(s, u)$ sia vera fino a prima della riga

$$v.d = u.d + 1$$

dopo essere stata eseguita per ipotesi induttiva sappiamo che $u.d + 1 \geq d(s, u) + 1$, dalla proprietà delle distanze abbiamo che $d(s, u) + 1 \geq d(s, v)$, allora mettendo insieme tutte le disuguaglianze si ha: $v.d = u.d + 1 \geq d(s, u) + 1 \geq d(s, v)$, ottenendo così $v.d \geq d(s, v)$. $\square$

Proprietà della coda

Si considera l'esecuzione di BFS su un grafo $G = (V, E)$ e supponiamo che la coda Q contenga i vertici $\langle v_1, v_2, \dots, v_r \rangle$ (con v_1 inizio e v_r fine della coda), allora si ha che:

1. $v_r.d \leq v_1.d + 1$
2. $v_i.d \leq v_{i+1}.d$ per ogni i in $1, 2, \dots, r - 1$

Dimostrazione.

Per dimostrare questa proprietà dobbiamo farlo per induzione

Passo base:

Dopo l'inizializzazione la coda contiene solo s , allora la proprietà è verificata.

Passo induttivo:

1. Assumiamo che sia vera la seguente disuguaglianza:

$$v_r.d \leq v_1.d + 1 \leq v_2.d + 1$$

Quando v_1 , viene estratto dalla coda, v_2 diventa la nuova testa e si ha banalmente

$$v_r.d \leq v_2.d + 1$$

2. Sia v_r l'ultimo elemento in coda, prima di essere stato inserito in coda è stato posto $v.d = u.d + 1$, con u il primo elemento della coda ovvero v_1 , quindi $v_r.d = v_1.d + 1$. Ipotizziamo venga aggiunto un altro vertice tale per cui l'ultimo elemento è v_{r+1} . Si ha che $v_{r+1}.d = v_1.d + 1$. Per ipotesi induttiva, inoltre, si ha che $v_r.d \leq v_1.d + 1$ ma allora: $v_r.d \leq v_1.d + 1 = v_{r+1}.d$ da cui:

$$v_r.d \leq v_{r+1}.d$$

$\square$

Correttezza della BFS

L'algoritmo BFS visita tutti i vertici raggiungibili da s e quando termina $v.d = d(s, v)$ per ogni vertice v del grafo. Inoltre per ogni vertice $v \neq s$ raggiungibile da s :

- $u = v.p \neq \text{nil}$
- $(u, v) \in E$
- $d(s, v) = d(s, u) + 1$

Dimostrazione.

Se v non è raggiungibile da s allora dalla proprietà del limite superiore si ha che

$$v.d = d(s, v) = \infty$$

Questo valore rimarrà invariato per tutta l'esecuzione dell'algoritmo, questo significa che il vertice v non verrà mai aggiunto in coda e quindi i vertici non raggiungibili non vengono visitati da BFS.

Se v è raggiungibile da s , per ogni vertice $v \in V$ raggiungibile da s i seguenti punti vengono effettuati una ed una sola volta:

1. a $v.color$ viene assegnato il colore "GRIGIO"
2. a $v.d$ viene assegnato il valore $d(v, s)$
3. se v è diverso da s , a $v.p$ viene assegnato un vertice u tale che $d(s, v) = d(s, u) + 1$, se $v = s$ a $s.p$ è assegnato nil
4. il vertice v viene aggiunto alla coda Q

Dopo l'inizializzazione nessun vertice viene più colorato di *BIANCO*, un vertice può essere colorato di *GRIGIO* soltanto se era *BIANCO*

Se consideriamo la sorgente banalmente l'inizializzazione avviene una sola volta quindi:

- $s.color = \text{"GRIGIO"}$
- $s.d = 0$
- $s.p = \text{nil}$
- $ENQUEUE(Q, S)$

Si considera poi un insieme di vertici

$$V_k = \{v \in V : d(s, v) = k\}$$

ovvero l'insieme di vertici posti a distanza k da s . Sia $v \in V_k$, durante l'esecuzione del ciclo while la coda Q non è mai vuota e non appena il vertice v viene messo in coda i valori $v.d$ e $v.p$ non possono più cambiare in quanto a $v.color$ viene assegnato il colore "GRIGIO" e per entrare nell'if è richiesto che $v.color$ sia "BIANCO", di conseguenza se l'ordine di inserimento in coda è

$$s = v_0, v_1, \dots, v_t$$

banalmente si ha che

$$v_i.d \leq v_{i+1}.d$$

che corrisponde alla proprietà della coda. Il vertice v a distanza k non può essere scoperto prima, in quanto non è adiacente a nessun vertice a distanza minore $k - 1$, infatti a tale distanza è presente u distante da s $k - 1$, a cui s è adiacente. $\square$

8.3 DFS

L'algoritmo DFS (deep-first search) visita in profondità un grafo, ispezionando gli archi uscenti a partire dall'ultimo vertice v scoperto. Quando tutti gli archi di v sono stati ispezionati "torna indietro" per ispezionare i restanti archi del vertice da cui v era stato scoperto

Pseudocodice

```

DFS(G,s){
  foreach u in G.V
    u.color = "BIANCO"
    u.p = nil
    time = 0
  foreach u in G.v
    if u.color = "BIANCO"
      DFS_VISIT(G, u)
}
DFS_VISIT{
  time = time + 1
  u.d = time // istante di scoperta
  u.color = "GRIGIO"
  foreach v in G.adj[u]
    if v.color = "BIANCO"
      v.p = u
      DFS_VISIT(G, v) // chiamo ricorsivamente sul primo sul primo nodo non ancora scoperto
  time = time + 1
  u.f = time // istante di completamento
  u.color = "NERO"
}

```

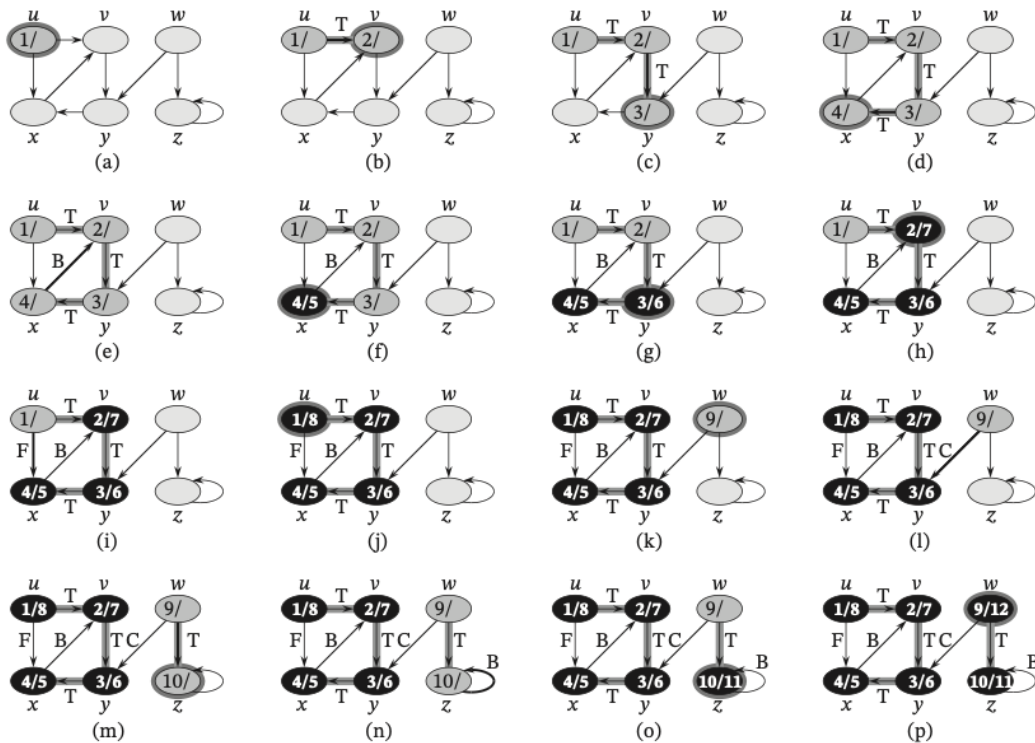


Figura 20.4 L'esecuzione dell'algoritmo DFS di visita in profondità su un grafo orientato. Gli archi vengono classificati mano a mano che vengono esplorati: gli archi d'albero sono etichettati T, quelli all'indietro B, gli archi in avanti F e quelli trasversali C. All'interno dei vertici sono riportati i marcaterno (tempo di scoperta/tempo di completamento). Gli archi d'albero sono evidenziati in grigio. I vertici i cui tempi di scoperta o di completamento sono cambiati in ciascun passo sono evidenziati contornandoli con un cerchio mentre gli archi che vengono esplorati in tale passo sono evidenziati con una linea ingrossata.

Analisi

DFS pone due marcatempi su ogni vertice u , in particolare il primo identifica il tempo di scoperta ($u.d$) e viene assegnato quando u diventa **grigio**, mentre il secondo marcatempo identifica il tempo di completamento ($u.f$) ossia quando il vertice è completato e colorato di **nero**. DFS inizializza il colore di tutti i vertici a **bianco** e inizializza una variabile globale che identifica il tempo. Successivamente chiama **DFS_VISIT** sui vertici u che non sono stati ancora visitati. All'interno di **DFS_VISIT** viene incrementata la variabile tempo e viene assegnato al primo marcatempo di u , indicandone l'istante in cui viene colorato di "GRIGIO". Dopodichè si prende la lista di adiacenza del nodo u e si scorre richiamando **ricorsivamente** **DFS_VISIT** sul nodo v facente parte della lista di u . Infine viene incrementata nuovamente la variabile tempo e viene assegnata al marcatempo di completamento di u . La complessità di DFS è $O(|V| + |E|)$, dove $|V|$ è il numero di vertici del grafo e $|E|$ è il numero di archi del grafo.

Proprietà delle parentesi

Se si rappresenta:

- la scoperta di ogni vertice u con una parentesi aperta (u e
- il completamento con una parentesi chiusa $_u$)

si ottiene una sequenza bilanciata di parentesi.

Ossia, per ogni coppia di vertici u e v , ci sono quattro possibilità:

1. $(u \dots u) \dots (v \dots v)$
2. $(v \dots v) \dots (u \dots u)$
3. $(u \dots (v \dots v) \dots u)$
4. $(v \dots (u \dots u) \dots v)$

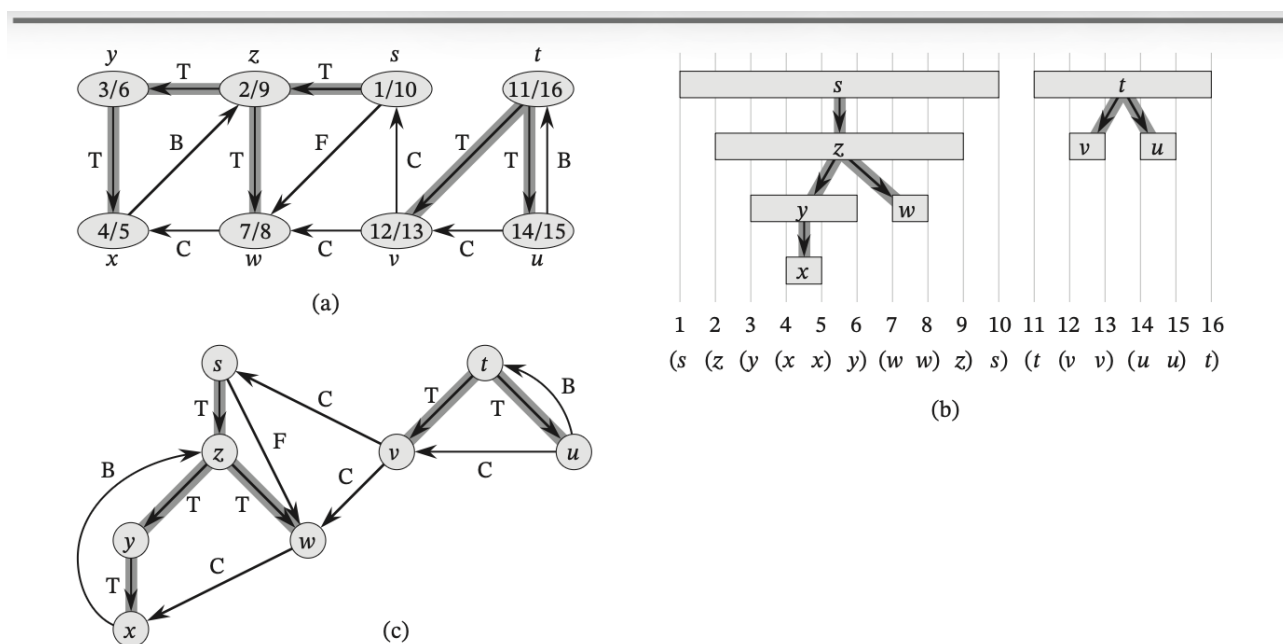


Figura 20.5 Proprietà della visita in profondità. **(a)** Il risultato di una visita in profondità di un grafo orientato. I vertici includono i marcatempi e i tipi degli archi sono indicati come nella Figura 20.4. **(b)** Gli intervalli fra il tempo di scoperta e il tempo di completamento di ciascun vertice corrispondono alla parentesizzazione illustrata. Ogni rettangolo si estende nell'intervallo definito dai tempi di scoperta e di completamento del corrispondente vertice. Sono mostrati soltanto gli archi d'albero. Se due intervalli si sovrappongono, allora uno viene annidato all'interno dell'altro e il vertice che corrisponde all'intervallo più piccolo è un discendente del vertice che corrisponde all'intervallo più grande. **(c)** Il grafo della parte (a) ridisegnato con tutti gli archi d'albero e in avanti che scendono in un albero di visita in profondità, e con tutti gli archi all'indietro che salgono da un discendente a un antenato.

Teorema delle Parentesi

Teorema 8.3.1. In una qualsiasi DFS di un grafo $G = (V, E)$ (orientato o non orientato), per ogni coppia di nodi u, v è soddisfatta una e una sola delle seguenti tre condizioni:

1. Gli intervalli $[u.d, u.f]$ e $[v.d, v.f]$ sono completamente disgiunti; u e v non sono discendenti uno dell'altro nella foresta della visita in profondità
2. L'intervallo $[u.d, u.f]$ è interamente contenuto nell'intervallo $[v.d, v.f]$ e u è un discendente di v in un albero di visita in profondità
3. L'intervallo $[v.d, v.f]$ è interamente contenuto nell'intervallo $[u.d, u.f]$ e v è un discendente di u in un albero di visita in profondità.

Dimostrazione. **Caso** $u.f < v.d$

Sapendo che $u.d < u.f < v.f$ ne segue che:

$$u.d < u.f < v.d < v.f$$

e si decide che i due intervalli non hanno intersezione e quindi sono disgiunti. Bisogna notare che essendo i due intervalli disgiunti si ha che il vertice v viene scoperto dopo che il nodo u è stato visitato, segue che non esiste un cammino che congiunge u e v .

Caso $u.f > v.d$

In questo caso si ha che il vertice v viene scoperto quando u è grigio (in esplorazione), quindi la lista delle adiacenze di v viene completamente esplorata prima di riprendere l'esplorazione di u e pertanto v viene completato prima di u . $\square$

Teorema del cammino bianco

Teorema 8.3.2. In una foresta di visita in profondità di un grafo $G = (V, E)$ (orientato o non orientato), il vertice v è discendente del vertice u se e solo se, al tempo $u.d$ quando u viene scoperto il vertice v può essere raggiunto da u lungo un cammino che è formato esclusivamente da vertici bianchi.

Dimostrazione. $\Rightarrow$) Sia v un discendente di u , sia $P = x_0, x_1, \dots, x_k$ (con $x_0 = u$ e $x_k = v$) la sequenza dei vertici da u a v . Sapendo che x_{i+1} viene scoperto dalla lista delle adiacenze di x_i , esiste l'arco (x_i, x_{i+1}) e banalmente x_i viene visitato prima di x_{i+1} . Quindi nel cammino P quando $x_0 = u$ viene scoperto, tutti i nodi successivi ancora non sono stati scoperti e quindi sono bianchi.

$\Leftarrow$) Supponiamo che quando u viene scoperto esista un cammino bianco da u a v , allora essendo che v viene scoperto dopo u , dalla proprietà delle parentesi si ha che

$$(u \dots u) \dots (v \dots v)$$

quindi sono disgiunti, oppure

$$(u \dots (v \dots v) \dots u)$$

quindi sono discendenti. Supponiamo per assurdo che

$$(u \dots u) \dots (v \dots v)$$

e che quindi v non sia discendente di u . Si può assumere che v sia il primo vertice del cammino bianco che non è discendente di u . Sia $w = x_{k-1}$, il vertice che precede v nel cammino bianco. Siccome w è discendente di u per la proprietà delle parentesi si ha che

$$(u \dots (w \dots w) \dots u) \dots (v \dots v)$$

ma questo è assurdo in quanto v è nella lista delle adiacenze di w e quindi deve essere scoperto prima che w sia completato. Quindi si conclude che v è discendente di u . $\square$

Tipi di archi DFS

Sia $uv \in E$ un arco, questo può essere classificato in:

- **Archivi d'albero:** v è scoperto dalle adiacenze di u quindi ($v.status = UNEXPLORED$)
- **Archivi all'indietro:** $u = v$, oppure v è ascendente di u ($v.status = EXPLORING$)
- **Archivi in avanti:** v discendente di u ($v.status = EXPLORED$ AND $v.timestamp > u.timestamp$)
- **Archivi trasversali:** u e v appartengono a rami diversi ($v.status = EXPLORED$ AND $v.timestamp < u.timestamp$)

Il timestamp a cui fanno riferimento è quello di scoperta dei nodi.

Teorema 8.3.3. In una DFS di un grafo $G = (V, E)$ non orientato, gli archi di G possono essere solo archi dell'albero (T) o archi all'indietro.

Ordinamento Topologico

Un Ordinamento Topologico di un DAG (Directed Acyclic Graph) $G = (V, E)$ è un ordinamento totale di tutti i suoi vertici tale che, se G contiene un arco (u, v) , allora u appare prima di v nell'ordinamento.

Topological-Sort(G)

1. Chiama DFS(G) per calcolare i tempi di completamento $v.f$ per ciascun vertice v
2. Una volta completata la visita di un vertice, inserisce il vertice in testa a una lista concatenata
3. return la lista concatenata dei vertici

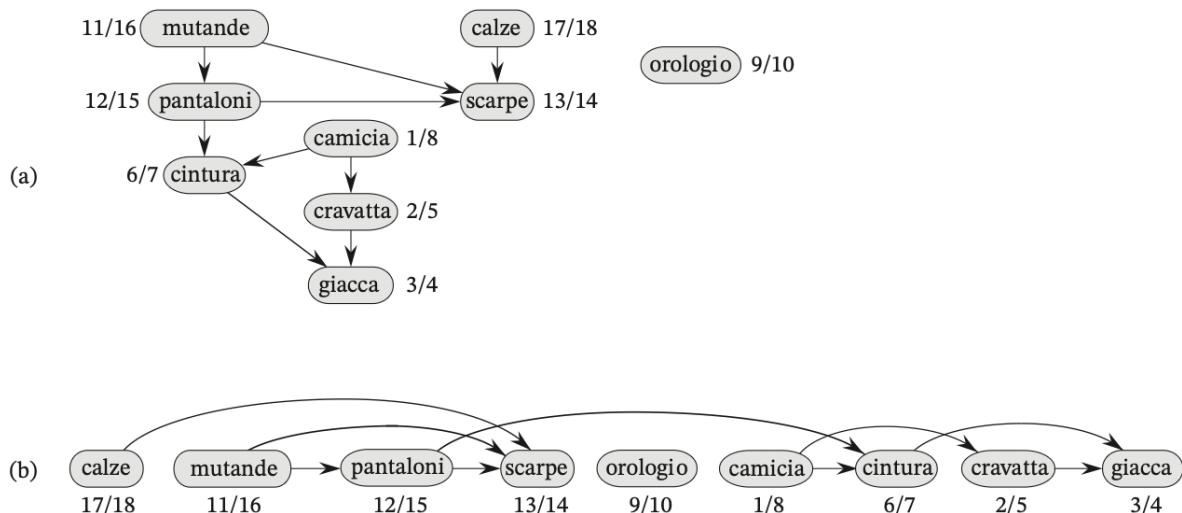


Figura 20.7 (a) Il professor Bumstead ordina topologicamente i suoi indumenti quando si veste. Ogni arco orientato (u, v) significa che l'indumento u deve essere indossato prima dell'indumento v . I tempi di scoperta e di completamento di una visita in profondità sono illustrati accanto a ciascun vertice. (b) Lo stesso grafo rappresentato secondo l'ordinamento topologico, con i suoi vertici ordinati da sinistra a destra in ordine di tempi di completamento decrescenti. Tutti gli archi orientati sono diretti da sinistra a destra.

Teorema 8.3.4. Un grafo orientato è aciclico (un DAG) se, e solo se, nella visita in profondità non si trova nessun arco all'indietro

Dimostrazione. $\Rightarrow$) Se in una visita in profondità si trova un arco all'indietro vu , allora tale arco aggiunto al cammino da u a v (che esiste in quanto v è discendente di u) forma un ciclo.

$\Leftarrow$) Viceversa, supponiamo che il grafo abbia un ciclo. Sia v il primo vertice del ciclo ad essere scoperto e sia uv l'arco del ciclo che entra in v . Quando v viene scoperto, esiste un cammino bianco da v ad u e quindi, per la proprietà del cammino bianco, u è discendente di v . Di conseguenza uv è un arco all'indietro. $\square$

Teorema 8.3.5. TOPOLOGICAL-SORT produce un ordinamento topologico del DAG che gli viene fornito in input

Dimostrazione. Basta dimostrare che per ogni arco uv il vertice v viene finito prima del vertice u . Quando l'arco uv viene esplorato, il vertice u è grigio mentre il vertice v non può essere grigio, altrimenti uv sarebbe un arco all'indietro.

Se v è nero, allora è già stato completato, mentre u non lo è ancora.

Se v è bianco, allora è discendente di u per la proprietà del cammino bianco e viene completato prima di u per la proprietà delle parentesi. $\square$

Componenti fortemente connesse

Definizione 8.3.1. Una componente fortemente connessa (CFC) di un grafo orientato $G = (V, E)$ è un insieme massimale di vertici $U \subseteq V$ tale che, $\forall u, v \in U$ esiste un cammino da u a v ed un cammino da v ad u .

Per trovare le componenti fortemente connesse di un grafo è sufficiente seguire questi passi:

1. Chiamare DFS sul grafo e calcolare i tempi di completamento per ciascun vertice;
2. Creare il grafo trasposto di G , ottenuto invertendo il verso di tutti gli archi;
3. Chiamare DFS sul grafo trasposto considerando i vertici in ordine decrescente di tempo di completamento calcolato al punto 1.;
4. Restituire o stampare le componenti fortemente connesse del grafo.

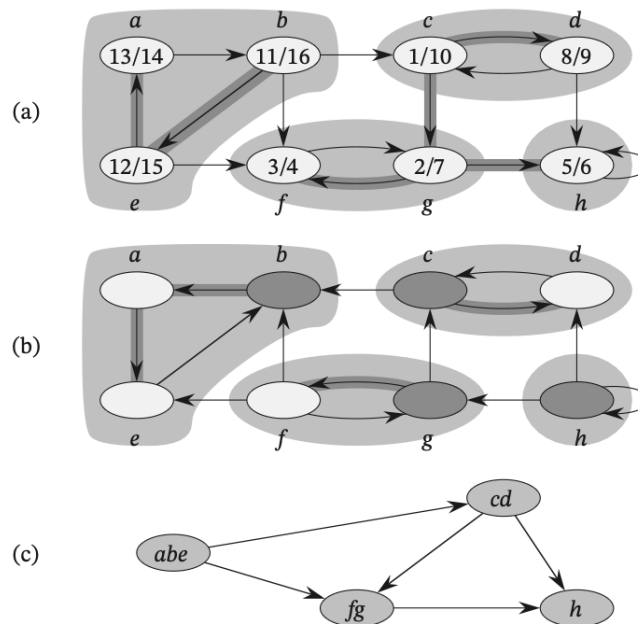
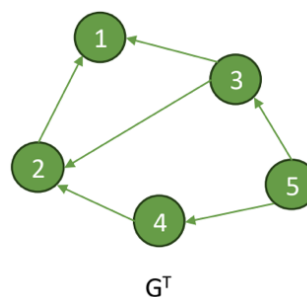
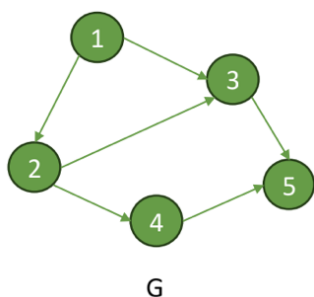


Figura 20.9 (a) Un grafo orientato G . Le componenti fortemente connesse di G sono illustrate come aree ombreggiate. Ogni vertice è etichettato con i suoi tempi di scoperta e di completamento. Gli archi d'albero hanno lo sfondo grigio scuro. (b) Il grafo G^T , il trasposto di G , con la foresta di visita in profondità calcolata nella riga 3 della procedura STRONGLY-CONNECTED-COMPONENTS, evidenziata con gli archi d'albero su sfondo grigio scuro. Ciascuna componente fortemente connessa corrisponde a un albero di visita in profondità. I vertici b , c , g e h , rappresentati con colore grigio scuro, sono le radici degli alberi di visita in profondità, prodotti dalla visita in profondità di G^T . (c) Il grafo aciclico delle componenti G^{SCC} , ottenuto contraendo tutti gli archi all'interno di ciascuna componente fortemente connessa di G , in modo che resti un solo vertice in ciascuna componente.

Grafo Trasposto G^T

Dato un grafo $G = (V, E)$, il grafo trasposto $G^T = (V, E^T)$ è il grafo avente gli stessi vertici di G e un insieme di archi definiti come

$$E^T = \{(u, v) : (v, u) \in E\}$$



8.4 Shortest Path (Percorsi minimi)

Il problema dei cammini minimi prende in input un grafo orientato pesato $G(V, E)$ con una funzione peso $\omega : E \rightarrow \mathbb{R}$ che associa ad ogni arco un numero reale che rappresenta il suo peso. V è l'insieme dei vertici $v_1, v_2, \dots, v_n$. Il problema restituisce il cammino minimo (sequenza di nodi) partendo da un nodo sorgente fino ad un nodo destinazione di un grafo. Nel caso di grafi non pesati il cammino minimo è rappresentato dal cammino più corto, mentre nel caso di grafi pesati il cammino minimo è quello che ha peso totale minimo.

Esistono numerose **varianti** ma queste sono le più importanti:

- **SSPS (Single Pair Shortest Path)**: Viene fornito in input una coppia di sorgente e destinazione, e viene restituito il percorso minimo tra sorgente e destinazione.
- **SSPS (Single Source Shortest Path)**: Viene fornito in input solo la sorgente e viene restituito il percorso minimo per visitare tutti i vertici del grafo.
- **SDSP (Single Destination Shortest Path)**: Viene fornito in input solo la destinazione e vengono restituiti i percorsi minimi da ogni vertice del grafo.
- **APSP (All Pairs Shortest Path)**: Vengono restituiti tutti percorsi minimi per ogni coppia di vertici (sorgente, destinazione) del grafo.

In generale, indichiamo il **peso di un cammino minimo** dal vertice u al vertice v con questa notazione:

$$\omega(p) = \delta(u, v)$$

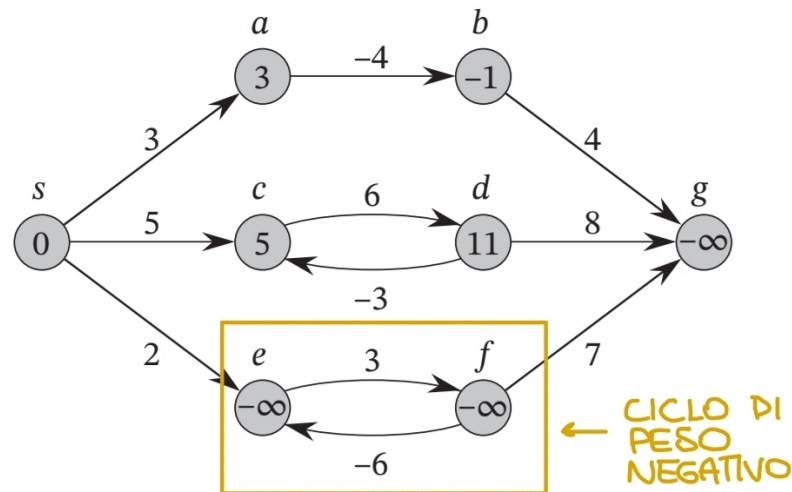
Se un nodo **non è raggiungibile** in alcun modo allora il peso sarà:

$$\delta(u, v) = +\infty$$

Proprietà

Gli algoritmi per cammini minimi godono delle seguenti proprietà:

- **Sottostruttura ottima**: ogni sotto-percorso del cammino minimo è a sua volta minimo (ottimo). Questa proprietà è molto importante in quanto consente l'utilizzo di algoritmi **greedy** (vd. 5).
- **Cicli**: nel caso in cui non ci sono cicli di peso negativo all'interno del grafo il peso del cammino minimo è ben definito. Invece nel caso in cui ci sono cicli di peso negativo il peso del cammino minimo non è definito e diciamo che il peso di tale cammino è $\delta(s, v) = -\infty$. Vediamo un esempio:



- **(Non) Unicità:** I cammini minimi non sono necessariamente unici, nè lo sono gli alberi dei cammini minimi.

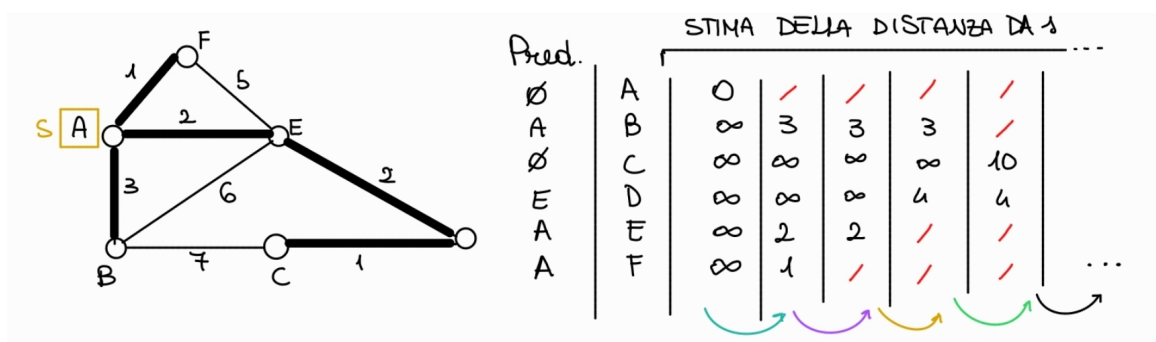
Rappresentazione dei cammini minimi

Gli algoritmi per i cammini minimi non solo calcolano il peso del cammino minimo, ma permettono di sapere i vertici che compongono tale cammino. Dato un grafo $G = (V, E)$ per ogni vertice $v \in V$ manteniamo un **predecessore** $v.\pi$ che può essere un altro vertice o il valore NIL. Gli algoritmi per i cammini minimi impostano π in modo che la catena di predecessori che inizia da un vertice v percorra all'indietro un cammino minimo da s a v . Quindi, per ogni vertice $v \neq \text{NIL}$, possiamo stampare un cammino minimo da s a v .

Rilassamento

Gli algoritmi che vedremo usano la tecnica del **rilassamento**. Per ogni vertice $v \in V$, gli algoritmi per i cammini minimi da sorgente unica (single source) mantengono un attributo $v.d$ che rappresenta la **stima del cammino minimo**, ovvero il percorso provvisorio minimo che va dalla sorgente fino al vertice che stiamo analizzando. Il processo di rilassamento di un arco (u, v) consiste nel verificare se passando per u , è possibile migliorare il cammino minimo per v precedentemente trovato. Arrivati alla fine di questo processo quindi avremo un percorso minimo che parte da s e visita tutti i vertici del grafo.

Vediamo un esempio:



Creiamo una tabella in costante aggiornamento dove teniamo traccia dei predecessori e delle stime delle distanze. All'inizio le stime delle distanze sono impostate tutte a ∞ tranne la sorgente la cui distanza da se stessa è ovviamente 0. Successivamente procediamo in questa maniera:

1. Seleziono la distanza stimata minima, ovvero seleziono il vertice la cui distanza stimata è minima rispetto a tutte quelle nella colonna, nel primo caso la distanza minima è 0, quindi selezioniamo il vertice A.
2. Una volta selezionato il vertice, verifichiamo quali vertici posso raggiungere a partire dal vertice selezionato e con che peso avviene tale raggiungimento. Nel nostro caso, abbiamo selezionato il vertice A, da A possiamo arrivare a F con costo 1, possiamo arrivare a B con costo 3 e possiamo arrivare a E con costo 2.

- Infine dobbiamo aggiornare la tabella con i valori che abbiamo trovato, impostando A come predecessore di A, B, E e aggiornando la stima delle distanze nella prossima colonna con i costi del raggiungimento di ogni vertice. Nella colonna successiva, quindi, ci saranno le distanze aggiornate e continueremo il processo ripartendo il con lo step (1.) (selezionando la distanza minima ecc...), mentre il nodo che avevamo in precedenza selezionato lo abbiamo segnato con una barra rossa. Quando aggiorniamo i dati, seguiamo il criterio principale di trovare il percorso minimo, quindi aggiorneremo un valore se e solo se è minore di quello attuale.

Il processo di rilassamento sarà concluso quando tutta la colonna delle distanza stimate sarà composta da barre rosse, questo vorrà dire che abbiamo visitato tutti i vertici e che abbiamo delineato il cammino minimo.

Algoritmo di Bellman-Ford

L'algoritmo di **Bellman-Ford** risolve il problema dei cammini minimi da sorgente unica nel caso generale in cui i pesi degli archi **possono essere negativi**. Dato un grafo orientato pesato $G = (V, E)$ con vertice sorgente s e funzione peso $w : E \rightarrow \mathbb{R}$ (ovvero che a ogni arco viene assegnato un numero reale rappresentate il suo peso), l'algoritmo di Bellman-Ford restituisce un valore booleano che indica se esiste un ciclo di peso negativo che è raggiungibile dalla sorgente. Se un tale ciclo esiste, l'algoritmo indica che il problema non ha soluzione. Se un tale ciclo non esiste, l'algoritmo fornisce i cammini minimi e i loro pesi. La procedura **BELLMAN-FORD rilassa** gli archi, riducendo progressivamente il valore stimato $v.d$ per il peso di un cammino minimo dalla sorgente s a ciascun vertice $v \in V$, fino a raggiungere il peso effettivo $\delta(s, v)$ di un cammino minimo.

Ecco lo **pseudocodice**:

```

BELLMAN-FORD( $G, w, s$ )
INITIALIZE-SINGLE-SOURCE( $G, s$ )
for  $i = 1$  to  $|G.V| - 1$ 
    for ogni arco  $(u, v)$  in  $G.E$ 
        RELAX( $u, v, w$ )    // applica il rilassamento
for ogni arco  $(u, v)$  in  $G.E$ 
    if  $v.d > u.d + w(u, v)$ 
        return FALSE
return TRUE

```

L'algoritmo restituisce TRUE se e soltanto se il grafo non contiene cicli di peso negativo che sono raggiungibili dalla sorgente.

Vediamo un esempio:

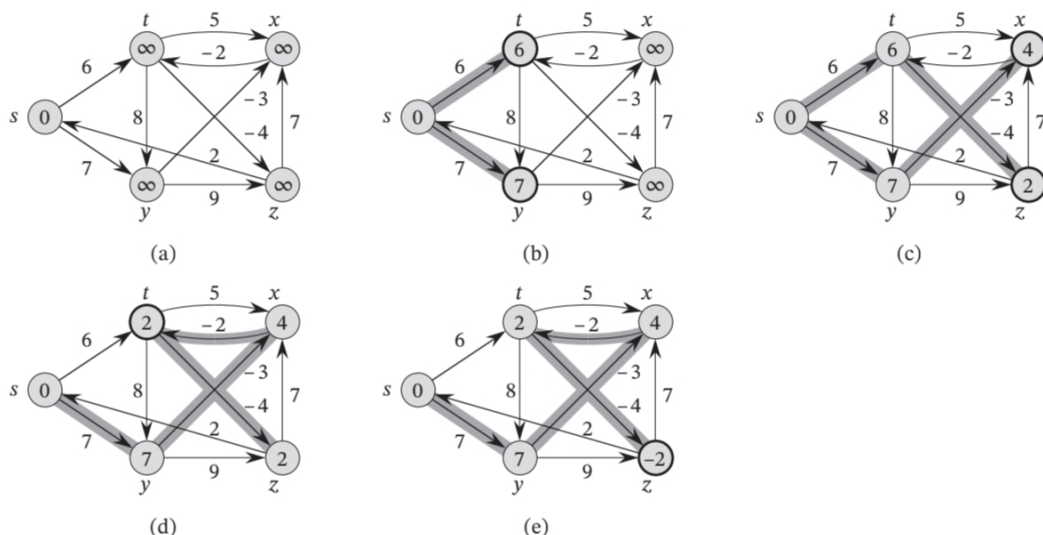


Figura 22.4 L'esecuzione dell'algoritmo di Bellman-Ford. La sorgente è il vertice s . I valori d sono annotati all'interno dei vertici e gli archi ombreggiati indicano i valori dei predecessori: se l'arco (u, v) è ombreggiato, allora $v.\pi = u$. In questo esempio specifico, ogni passaggio rilassa gli archi nell'ordine $(t, x), (t, y), (t, z), (x, t), (y, x), (y, z), (z, x), (z, s), (s, t), (s, y)$. **(a)** La situazione precedente al primo passaggio sugli archi. **(b)-(e)** La situazione dopo successivi passaggi sugli archi. I vertici che, durante un passaggio, hanno cambiato stima del peso di cammino minimo e predecessore sono evidenziati con il contorno ingrossato. I valori d e π nella parte (e) sono i valori finali. L'algoritmo di Bellman-Ford restituisce TRUE in questo esempio.

Invece, per quanto riguarda problemi di cammino minimo da un'unica sorgente con archi che **non sono mai positivi**, esiste l'**algoritmo di Dijkstra**, anch'esso basato sulla tecnica del rilassamento ma che non abbiamo approfondito.

Algoritmo di Bellman Ford versione distribuita*

L'algoritmo di Bellman Ford **distribuito** si differenzia da quello che abbiamo visto in precedenza per queste tre caratteristiche:

1. **Conoscenza locale**: ogni nodo conosce solo i per raggiungere i nodi adiacenti
2. **Messaggistica**: i nodi si scambiano messaggi informativi contenenti i propri **distance vector**, ovvero il vettore che contiene le distanze dagli altri nodi.
3. **Asincronicità**: calcolo del risultato finale alla ricezione di un aggiornamento

Questa è l'equazione con cui aggiorniamo le distanze del cammino minimo, siano x e y due nodi di un grafo, l'equazione è la seguente:

$$d_x(y) = \min_{v \in N(x)} \{c(x, v) + d_v(y)\}$$

dove:

- $N(x)$ è l'insieme dei vicini del nodo x , ovvero tutti i nodi che hanno un collegamento (arco) diretto a x .
- v è un singolo nodo vicino.
- y è il nodo di destinazione finale.
- $d_x(y)$ è il costo stimato da x a y .
- $c(x, y)$ è il costo del link diretto verso il vicino v .
- $d_v(y)$ è la distanza da v a y dichiarata dal vicino.

L'equazione con cui aggiorniamo il risultato quindi funziona così: ad ogni nodo visitato vengono analizzati tutti i suoi vicini, e viene scelto il cammino che minimizza la somma per arrivare a y .

Routing Loop

Se uno dei nodi "fallisce", ovvero per qualche motivo il nodo non è più utilizzabile o raggiungibile, possono accadere quelli che si chiamano i **routing loop**, che accadono quando dei nodi si scambiano informazioni obsolete creando un ciclo infinito. Per evitare questo problema possiamo utilizzare 3 soluzioni:

- **Split Horizon**: Se X raggiunge Z tramite Y , allora X non deve inviare a Y alcun aggiornamento relativo a Z
- **Poison Reverse**: Se X raggiunge Z tramite Y , allora X dichiara a Y che la sua distanza per Z è infinito (∞)
- **Soluzione Pratica (RIP)**: Viene utilizzato il valore 16 come infinito per terminare i loop circolari.

Algoritmo di Floyd-Warshall (APSP con pesi arbitrari)

L'algoritmo di **Floyd-Warshall** risolve il problema dell'All-Pairs Shortest Path (APSP) su grafi orientati in cui gli archi possono avere pesi arbitrari (anche negativi), a patto che **non esistano cicli di peso totale negativo**. Se ci fosse un ciclo negativo, infatti, l'algoritmo continuerebbe a percorrerlo all'infinito per abbassare il costo totale, rendendo impossibile definire un cammino "minimo". Ecco lo pseudocodice:

```
FloydWarshall(Graph G)
    dist = new n x n matrix of distances           // (all infinite)
    next = new n x n matrix of vertex indices      // (all zeros)
    forall (u,v) in E(G)
        dist[u][v]=w(u,v)
        next[u][v]=v
    for k from 1 to n                               // primo ciclo
```

```

for i from 1 to n      //secondo ciclo
  for j from 1 to n    //terzo ciclo
    if(dist[i][j]>dist[i][k]+dist[k][j])
      dist[i][j]=dist[i][k]+dist[k][j]
      next[i][j]=next[i][k]

```

L'algoritmo si basa sul paradigma della **programmazione dinamica** e ha un costo computazionale temporale pari a $\Theta(n^3)$, dove n è il numero totale dei nodi nel grafo.

Output dell'algoritmo

L'algoritmo costruisce e restituisce due matrici fondamentali:

- **La matrice dist:** Al termine dell'esecuzione, la cella `dist[i][j]` conterrà il **costo totale minimo** definitivo per viaggiare dal nodo i al nodo j .
- **La matrice next:** Viene utilizzata per ricostruire passo-passo il percorso fisico. La cella `next[i][j]` contiene l'indice del nodo immediatamente successivo a i nel cammino ottimale verso j .

Analisi dello Pseudocodice

- **Fase di Inizializzazione:** La matrice `dist` viene popolata inizialmente con i pesi diretti degli archi ($w(u, v)$), oppure con infinito (∞) se non esiste un collegamento diretto tra i nodi. La matrice `next` viene inizializzata con il nodo di destinazione se l'arco diretto esiste, altrimenti a 0.
- **I tre cicli annidati (k, i, j):** Rappresentano il nucleo logico dell'algoritmo. L'intuizione alla base è chiedersi: *"Per andare dal nodo i al nodo j , mi conviene utilizzare il percorso diretto (o il migliore trovato finora), oppure risulta più economico fare una deviazione passando per un nodo intermedio k ?"*
 - La condizione `if (dist[i][j] > dist[i][k] + dist[k][j])` implementa esattamente questa logica. Se la deviazione passando per k risulta meno costosa, si aggiorna la matrice `dist` con il nuovo costo ridotto. Parallelamente, si aggiorna `next[i][j]` affinché punti allo stesso nodo successivo previsto per il percorso $i \rightarrow k$.

Chiusura Transitiva e Riflessiva - Variante Booleana di Floyd-Warshall

Esiste una variante semplificata, di natura booleana, dell'algoritmo di Floyd-Warshall. In questo contesto, non interessa quantificare il costo del cammino minimo o tracciarne le tappe; l'obiettivo è rispondere a una semplice domanda binaria (Vero/Falso): **"Esiste un qualsiasi percorso (diretto o indiretto) per raggiungere il nodo v partendo dal nodo u ?"**

Questo concetto è noto come **chiusura transitiva**. L'attributo "riflessiva" indica l'assunzione di base secondo cui ogni nodo è sempre in grado di raggiungere se stesso. Ecco lo pseudocodice:

```

FloydWarshall(Graph G)
  T[] [] = new n*n adjacency matrix 1-based
  forall i: T[i][i] = 1 reflexive
  for k from 1 to n
    for i from 1 to n
      for j from 1 to n
        T[i][j] = T[i][j] || (T[i][k] && T[k][j])
  return T[] []

```

Analisi dello Pseudocodice Booleano

- Al posto dei costi, si opera su una **matrice booleana** `T` composta da 0 e 1. Nello stato iniziale, `T` coincide con la matrice di adiacenza del grafo (1 se esiste un arco diretto, 0 altrimenti), alla quale si aggiunge un 1 su tutta la diagonale principale (`T[i][i] = 1`) per la proprietà riflessiva.
- **Ciclo di aggiornamento:** L'istruzione chiave è `T[i][j] = T[i][j] || (T[i][k] && T[k][j])`. Questa espressione traduce la seguente logica: il nodo i è in grado di raggiungere il nodo j se:

1. Aveva già modo di raggiungerlo in precedenza (`T[i][j]` è vero), oppure
2. Il nodo i può raggiungere il nodo intermedio k , e a sua volta il nodo k può raggiungere j .

Al termine dell'esecuzione, la matrice di adiacenza iniziale è stata trasformata in una matrice finale in cui la presenza di un 1 certifica l'esistenza di un percorso tra i due nodi.

8.5 Alberi di connessione minimi MST

Il problema degli alberi di connessione minimi

Sia un grafo non orientato $G = (V, E)$ connesso e i cui archi hanno associato un peso $\omega(e)$. Il problema di definire un albero di connessione minimo è quello di trovare un sottoinsieme T di archi tali che:

1. T rappresenta un albero
2. il peso $\omega(T) = \sum_{e \in T} \omega(e)$ sia minimo

Un qualsiasi albero di connessione ha esattamente $|V| - 1$ archi.

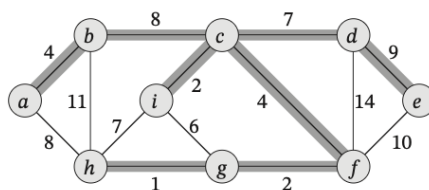
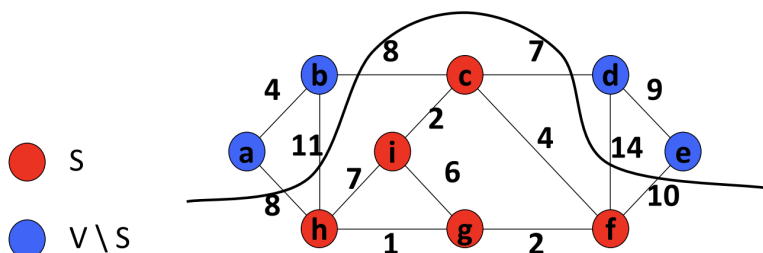


Figura 21.1 Un albero di connessione minimo per un grafo connesso. Sono indicati i pesi degli archi. Gli archi di un albero di connessione minimo sono rappresentati su sfondo grigio. Il peso totale dell'albero è 37. Questo albero non è unico: eliminando l'arco (b, c) e sostituendolo con quello (a, h) si ottiene un altro albero di connessione con peso 37.

Definizioni preliminari



Definizione 8.5.1. Un **taglio** in un grafo non orientato $G = (V, E)$ è una partizione dell'insieme di vertici V in due sottoinsiemi non vuoti $(S, V \setminus S)$

Definizione 8.5.2. Un arco **interseca** il taglio $(S, V \setminus S)$ se uno dei suoi estremi è in S e l'altro in $V \setminus S$

Definizione 8.5.3. Un taglio $(S, V \setminus S)$ rispetta un insieme di archi A se nessun arco in A interseca il taglio

Definizione 8.5.4. Un arco **leggero** per il taglio è un arco di costo minimo tra tutti quelli che intersecano il taglio

Algoritmo greedy per generare un albero di connessione minimo

Gli algoritmi avidi sono una particolare classe di algoritmi che risolvono i problemi di ottimizzazione adottando ad ogni passo la strategia migliore, ossia ragionano su un ottimo locale. Un metodo per generare un albero di connessione minimo da un grafo è il seguente:

```

GENERIC-MST(G){
  A=0
  while "A non forma un albero di connessione"
    trova un arco (u, v) che è sicuro per A
    A = A U (u, v)
  return A
}

```

Questo metodo fa crescere l'albero di connessione un arco alla volta. Si nota come vengono aggiunti solo **archi sicuri** per A , ovvero quegli archi che consentono al metodo di conservare la seguente invariante di ciclo: **di ogni iterazione, A è un sottoinsieme di qualche albero di connessione minimo.**

Teorema: Caratterizzazione degli archi scuri

Teorema 8.5.1. Sia $G = (V, E)$ un grafo connesso non orientato con una funzione ω a valori reali definita in E . Sia A un sottoinsieme di E , che è contenuto in qualche albero di connessione minimo per G , sia $(S, V \setminus S)$ un taglio qualsiasi di G che rispetta A e sia $a = (u, v)$ un arco leggero per il taglio $(S, V \setminus S)$. Allora l'arco a è sicuro.

Dimostrazione. Sia T un albero di connessione minimo che contiene A . Se T contiene anche l'arco a , allora il teorema è dimostrato in quanto a è un arco sicuro per A . Sia quindi T l'albero di connessione minima in figura, se T non contiene l'arco $a = (u, v)$, aggiungendo tale arco si genera un ciclo (banalmente in un albero non può esserci un ciclo). Essendo T un albero, deve esistere necessariamente un percorso p che congiunga i vertici u e v . In aggiunta dato che u e v si trovano su lati opposti del taglio $(S, V \setminus S)$ c'è almeno un altro arco $b = (x, y)$ tale che intersechi il taglio. L'arco b non appartiene ad A , in quanto come ipotesi del Teorema deve essere che il taglio $(S, V \setminus S)$ rispetta A . Sia T' l'albero di connessione minimo ottenuto da T togliendo l'arco b e aggiungendo l'arco a . Dato che a come ipotesi è un arco leggero, si ha che il costo di T' è minore o uguale di T . Per definizione T è un albero di connessione minimo, ma allora anche T' lo è. Infine essendo che T' contiene sia A che a , si ottiene che a rispetta A , in quanto aggiungendo questo arco ad A , questo è ancora un sottoinsieme di albero di connessione minimo $\square$

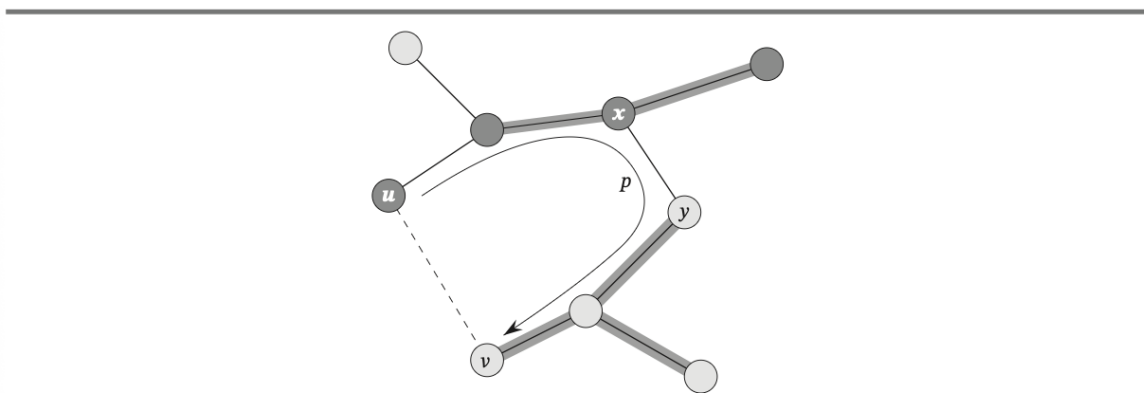


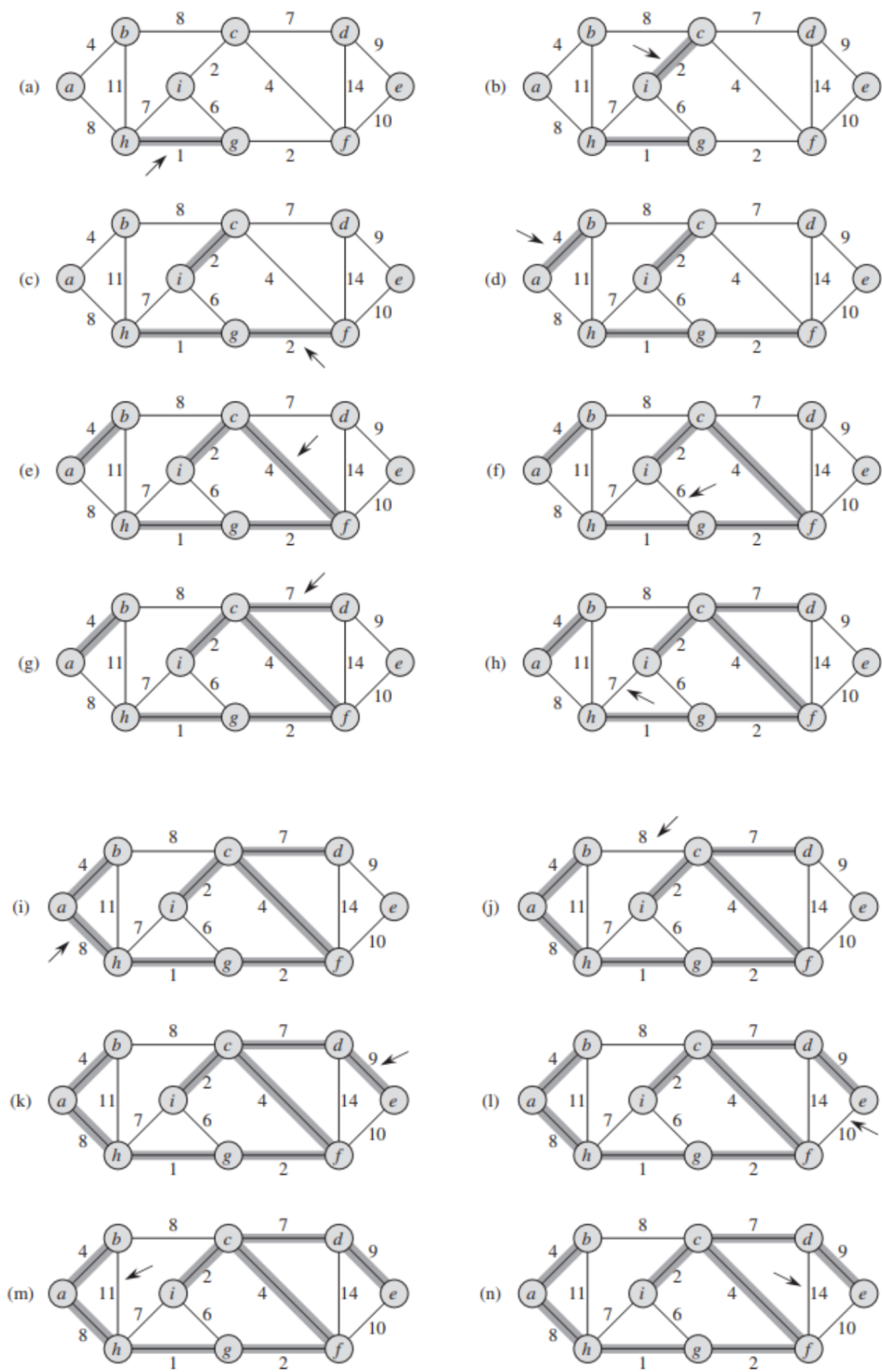
Figura 21.3 La dimostrazione del Teorema 21.1. I vertici in S sono scuri e quelli in $V - S$ sono chiari. Sono rappresentati soltanto gli archi nell'albero di connessione minimo T assieme all'arco (u, v) , che non appartiene a T . Gli archi in A hanno lo sfondo grigio e (u, v) è un arco leggero che attraversa il taglio $(S, V - S)$. L'arco (x, y) si trova nell'unico cammino semplice p che va da u a v in T . Un albero di connessione minimo T' che contiene (u, v) si ottiene eliminando l'arco (x, y) da T e aggiungendo quello (u, v) .

Algoritmo di Kruskal

L'algoritmo di Kruskal trova un arco sicuro da aggiungere alla foresta in costruzione scegliendo, fra tutti gli archi che collegano due alberi qualsiasi nella foresta, un arco (u, v) con il minimo peso.

```
MST-KRUSKAL(G){
  A=0
  foreach v in G.V
    Make-Set(v)
  l = converti_in_lista (G, E) //crea una lista di archi di G
  sort_decreasing(l, w)        //ordina la lista in ordine monotono crescente rispetto al peso
  foreach (u, v) in l          //per ogni arco (u,v) preso nell'ordine dalla lista
    if Find-Set(u) != Find-Set(v)
      A = A U (u, v)
      Union(u, v)
  return A
}
```

Esempio:



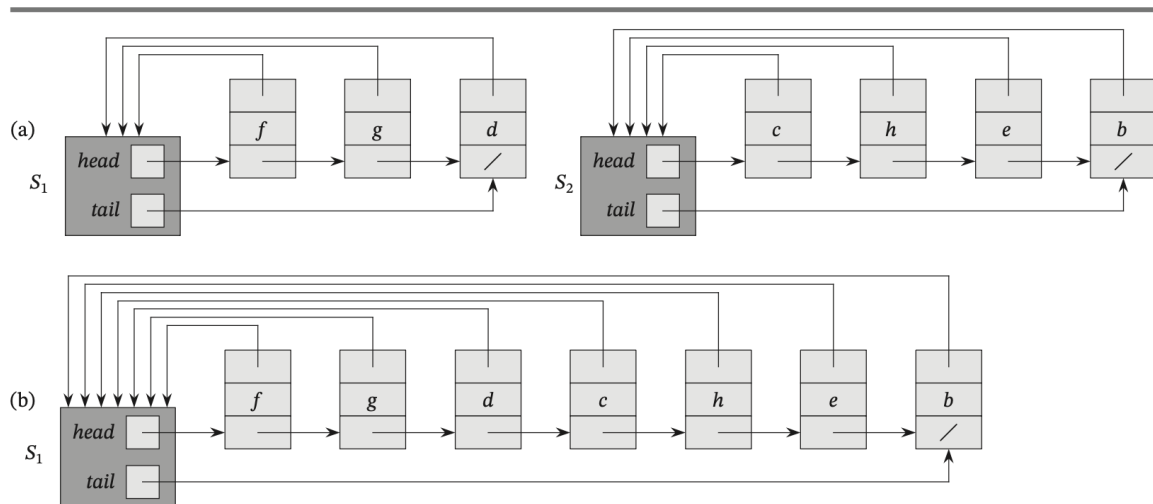


Figura 19.2 (a) Rappresentazioni con liste concatenate di due insiemi. L'insieme S_1 contiene gli elementi d, f e g , con f come rappresentante. L'insieme S_2 contiene gli elementi b, c, e e h , con c come rappresentante. Ogni oggetto nella lista contiene un elemento dell'insieme, un puntatore al successivo oggetto nella lista e un puntatore che permette di tornare all'oggetto dell'insieme. Ogni oggetto dell'insieme ha i puntatori *head* e *tail* rispettivamente al primo e all'ultimo oggetto. (b) Il risultato dell'operazione $UNION(g, e)$, che concatena la lista concatenata che contiene e alla lista concatenata che contiene g . Il rappresentante dell'insieme risultante è f . L'oggetto dell'insieme per la lista S_2 di e viene eliminato.

Per procedere con questo algoritmo è necessario definire una struttura dati che definisce una collezione di insiemi disgiunti (l'intersezione tra i vari insiemi è vuota), in cui ogni insieme è individuato da un rappresentante che è uno degli elementi dell'insieme. In questo algoritmo si è fatto uso di tre funzioni principali:

- **Make-Set(x):** aggiunge alla struttura dati un nuovo insieme contenente solo l'elemento x (si richiede x non compaia in nessun altro insieme della struttura). Ha costo $O(1)$.
- **Find-Set(x):** restituisce il rappresentante dell'insieme che contiene x . Ha costo $O(1)$.
- **Union(x, y):** unisce i due insiemi contenuti x ed y . Ha costo che dipende dalla dimensione di y e tende ad n .

Il costo dell'algoritmo di Kruskal è $O(n^2)$, il costo dipende principalmente dall'andamento della funzione **Union**. Per migliorare le performance, modifichiamo **UNION** attraverso l'euristica dell'unione pesata, scegliendo sempre la lista più corta per aggiornare i puntatori al rappresentante, memorizzando la lunghezza della lista in un nuovo campo L del rappresentante.

Teorema 8.5.2. Utilizzando la rappresentazione degli insiemi disgiunti tramite liste concatenate e l'euristica dell'unione pesata, una sequenza di m operazioni *MAKE-SET*, *UNION* e *FIND-SET*, n delle quali sono operazioni *MAKE-SET*, impiega un tempo $O(m + n \log n)$

Per migliorare il costo di questo algoritmo è necessario modificare l'implementazione della funzione. Questo algoritmo può essere implementato con un'altra struttura dati per rappresentare gli insiemi disgiunti: **le foreste di insiemi disgiunti**, in cui ogni insieme è rappresentato da un albero i cui nodi, oltre al campo *data* che contiene l'informazione, hanno un campo p che punta al predecessore.

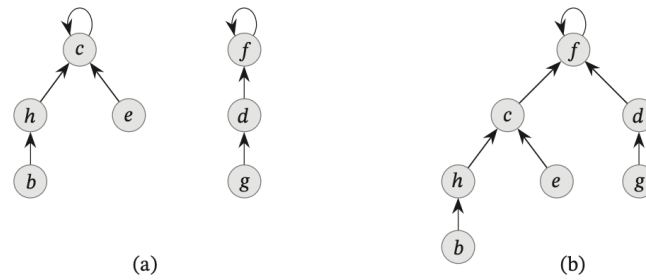


Figura 19.4 Una foresta di insiemi disgiunti. **(a)** Alberi che rappresentano i due insiemi della Figura 19.2. L'albero a sinistra rappresenta l'insieme $\{b, c, e, h\}$, con c come rappresentante; l'albero a destra rappresenta l'insieme $\{d, f, g\}$, con f come rappresentante. **(b)** Dopo l'operazione $UNION(e, g)$.

In questo caso possiamo implementare le tre funzioni in questo modo:

```

Make-Set(x){
    x.p = x
}

Find-Set(x){
    if x.p != x
        Find-Set(x.p)
    return x.p
}

Union(x,y){
    x = Find-Set(x)
    y = Find-Set(y)
    x.p = y
}

```

Si può notare che il costo della **Union** è uguale al costo della **Find-Set**, ovvero $O(\log n)$, in quanto devo scorrere da x alla radice dell'albero. Anche in questo caso il costo dell'algoritmo di Kruskal implementando quest'altro tipo di struttura dati è $O(n^2)$.

Possiamo migliorare notevolmente l'efficienza usando due euristiche.

La prima euristica, detta **unione per rango**, l'idea intuitiva consiste nel fare in modo che la radice dell'albero con meno nodi punti a quella con più nodi. Per ogni nodo manteniamo un **rango**, che è un limite superiore per l'altezza del nodo. L'unione per rango fa puntare la radice con rango minore alla radice con rango maggiore durante un'operazione $UNION$.

```

Make-Set(x)
    x.p = x
    x.rank = 0

```

La seconda euristica, detta **compressione del cammino**, in cui l'operazione $Find-Set$ viene usata per fare in modo che ciascun nodo nel cammino di ricerca punti direttamente alla radice.

```

Find-Set(x)
    if x.p != x                //non è la radice ?
        x.p = Find-Set(x.p)    //la radice diventa padre
    return x.p                 //restituisce la radice

```

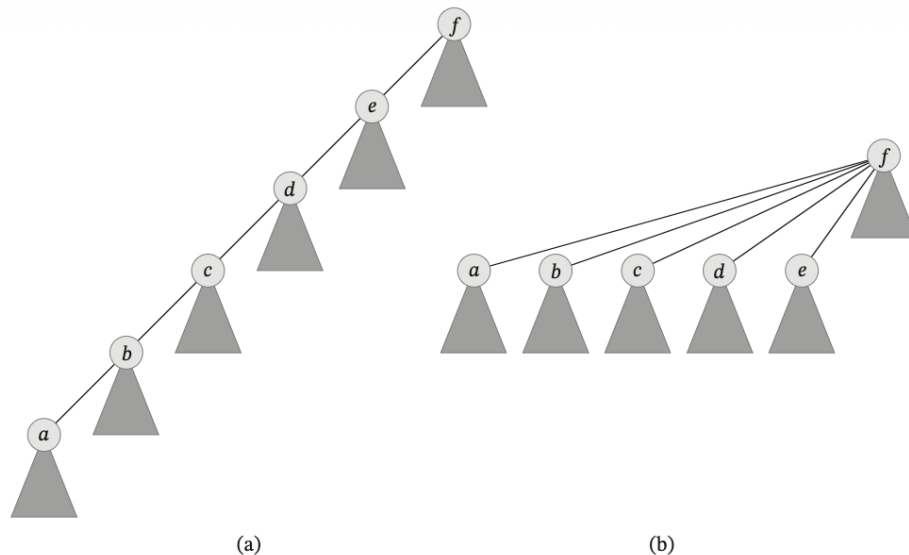


Figura 19.5 La compressione del cammino durante l'operazione FIND-SET. Le frecce e i cappi sulle radici sono stati omessi. **(a)** Un albero che rappresenta un insieme prima di eseguire FIND-SET(*a*). I triangoli rappresentano i sottoalberi le cui radici sono i nodi indicati nella figura. Ogni nodo ha un puntatore a suo padre. **(b)** Lo stesso insieme dopo l'esecuzione di FIND-SET(*a*). Ogni nodo lungo il cammino di ricerca adesso punta direttamente alla radice.

L'union invece diventa:

```

Union(x,y)
    x = Find-Set(x)
    y = Find-Set(y)
    Link(x,y)

Link(x,y)
    if x.rank > y.rank
        y.p = x
    else
        x.p = y
        if x.rank == y.rank
            y.rank = y.rank + 1

```

Foreste di Insiemi Disgiunti: complessità

Euristica del rango

Una sequenza di m operazioni delle quali n sono *Make-Set* richiede un tempo $O(m \log n)$ **Euristica di compressione dei cammini**

Una sequenza di m operazioni delle quali n sono *Make-Set* e k sono *Find-Set* richiede un tempo $\Theta(k \log_{(1+k/n)} n)$ se $k \geq n$

Le migliori prestazioni in assoluto si ottengono usando entrambe le euristiche.

Una sequenza di m operazioni delle quali n sono *make-Set* richiede un tempo $O(m\alpha(n))$, dove $\alpha(n)$ è una funzione che cresce molto lentamente, $\alpha(n) \leq 4$ in ogni concepibile uso della struttura dati.

La complessità ammortizzata di una singola operazione risulta quindi $O(\alpha(n))$: praticamente costante.

Algoritmo di Prim

A differenza dell'algoritmo di Kruskal che iterava sul costo degli archi in ordine crescente, l'algoritmo di Prim, per generare un albero di connessione minima, ragiona con una coda di min proprietà (come può essere ad esempio un min-Heap) le cui chiavi dei nodi indicano il peso minimo di un arco qualsiasi che collega il nodo ad un altro vertice nel grafo.

Pseudocodice algoritmo di Prim

```

MST-Prim(G,s){
    foreach u in. G.v

```

```

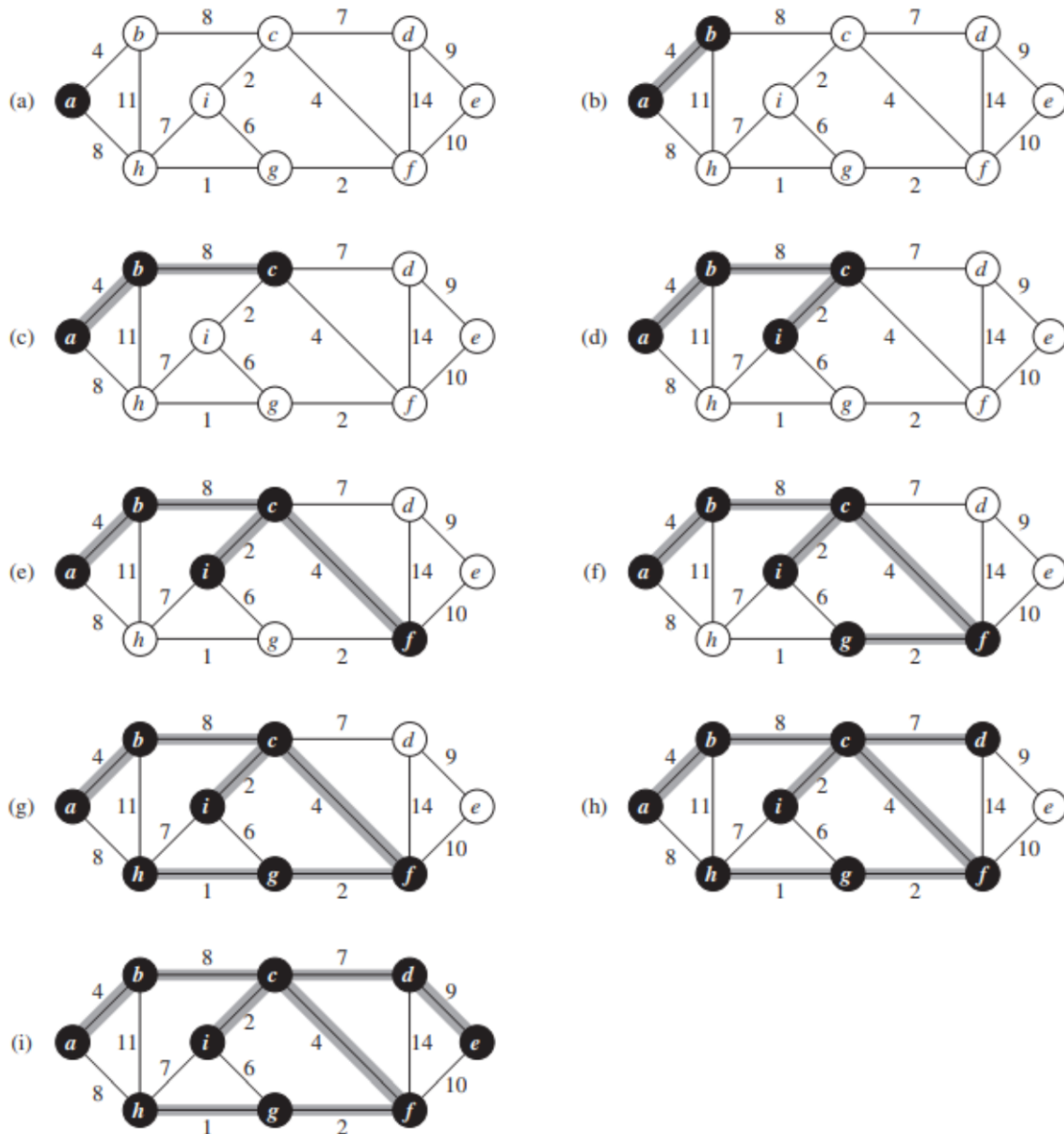
    u.key = infinity
    u.p = nil
s.key = 0
Q = 0

foreach u in G.V
    Insert(Q,u)

while Q != 0
    u = Extract_Min(Q)
    foreach v in G.Adj[u]
        if v in Q and w(u, v) < v.key
            v.p = u
            v.key = w(u, v)
            Decrease_key(Q,v,w(u, v))
}

```

Esempio:



Innanzitutto si sceglie un nodo da cui partire che fungerà da sorgente, in questo caso il nodo *a*. Successivamente l'algoritmo inizializza tutte le chiavi dei nodi ad infinito e i predecessori a *nil*, mentre per quanto riguarda la

sorgente pone la chiave a 0 e aggiunge tutti i nodi al *min-heap*. Quindi la coda adesso è formata da:

$$Q = [(a, 0), (b, \infty), (c, \infty), (d, \infty), (e, \infty), (f, \infty), (g, \infty), (h, \infty), (i, \infty)]$$

Successivamente l'esecuzione entra nel *while*, che scansione l'intera coda svuotando man mano il *min-heap*. L'algoritmo prende il primo nodo (in questo caso il vertice a) e scansiona la sua lista di adiacenza (contenente i vertici b ed h). Si valuta quindi se il costo dell'arco (prima (a, b) e poi (a, h)) è minore della chiave del nodo e successivamente se la condizione è verificata va ad aggiornare la chiave del vertice nel *min-heap*. Quindi la coda adesso diventa:

$$Q = [(b, 4), (h, 8), (c, \infty), (d, \infty), (e, \infty), (f, \infty), (g, \infty), (i, \infty)]$$

Si procede con il nodo b e vengono valutati gli archi (b, c) , (b, h) , aggiornando il valore della chiave del nodo c , ma non quello del nodo h in quanto $h.key < \omega(b, h)$, quindi la condizione non è verificata. In questo caso nel *min-heap*, dato che c ed h hanno lo stesso valore di chiave, si può decidere se visitare prima il nodo c o prima il nodo h , in entrambi i casi avremo un albero di connessione a costo minimo.

$$Q = [(c, 8), (h, 8), (d, \infty), (e, \infty), (f, \infty), (g, \infty), (i, \infty)]$$

Si procede in seguito con il nodo c e si valutano gli archi (c, i) , (c, f) , (c, d) e la coda diventa:

$$Q = [(i, 2), (f, 4), (d, 7), (h, 8), (e, \infty), (g, \infty)]$$

Dopodiché si procede con il vertice i e vengono valutati gli archi (i, h) , (i, g) e la coda diventa:

$$Q = [(f, 4), (g, 6), (d, 7), (h, 7), (e, \infty)]$$

Dopo è il turno del nodo f e vengono valutati gli archi (f, g) , (f, d) , (f, e) , la coda diventa:

$$Q = [(g, 2), (d, 7), (h, 7), (e, 10)]$$

Si estrae il nodo g e viene valutato l'arco (g, h) :

$$Q = [(h, 1), (d, 7), (e, 10)]$$

Si procede con il nodo h e non vengono valutati archi in quanto nessun vertice a cui è connesso h è nella coda:

$$Q = [(d, 7), (e, 10)]$$

Si rimuove d dalla coda e viene valutato l'arco (d, e) :

$$Q = [(e, 9)]$$

Infine, non ci sono più archi da valutare e si svuota la coda.

Analisi della complessità

Il tempo totale dell'algoritmo di Prim è $O(|V| \log |V| + |E| \log |V|) \rightarrow O(|E| \log |V|)$.

Se usiamo un Heap di Fibonacci ottengo un incremento di performance nel caso medio con una complessità di $O(|E| + |V| \log |V|)$ in quanto *Extrac_Min* ha un costo ammortizzato di $O(\log |V|)$ e ogni operazione *Insert* e *Decrease_Key* ha un costo ammortizzato di $O(1)$.

Capitolo 9

Alberi

Un albero è un grafo connesso senza cicli, ovvero un DAG connesso. Una delle caratteristiche fondamentali degli alberi è che essi hanno esattamente $n - 1$ archi, proprietà che può essere proposta anche per un grafo connesso senza cicli ricordando che il numero n di nodi che costituiscono un albero può essere inteso come $|V|$, dove V è l'insieme dei vertici del grafo G . Dunque per verificare che un grafo non abbia cicli è sufficiente stabilire che abbia un numero di archi pari a $n - 1$. In questo caso, essendo fornito un grafo semplice e connesso con n nodi ed m archi basta implementare un algoritmo che verifica che $m = n - 1$, ovvero effettuare un confronto semplice con costo costante $O(1)$.

9.1 Tipi di albero

Albero libero: Un albero libero è un tipo di grafo non orientato connesso e senza cicli, ovvero un DAG (Directed Acyclic Graph) connesso. Non è caratterizzato da uno specifico nodo che assume il ruolo di radice, bensì qualsiasi nodo può esserlo.

Albero radicato: Un albero radicato è una tipologia di albero libero in cui un vertice è stato scelto come radice. La radice è il punto di partenza da cui si può raggiungere ogni altro nodo attraverso un percorso unico.

Albero ordinato: Un albero radicato si dice **ordinato** se i figli di ogni nodo hanno un ordine specifico. Inoltre l'ordine può essere necessario per la visita pre-ordine, in-ordine e post-ordine.

Albero posizionale: Un albero ordinato si dice posizionale se ad ogni figlio di un nodo è associata una posizione specifica, ad esempio quando si parla di figlio "destro" e "sinistro" per gli alberi binari. Ricorda: le posizioni sono etichettate e differenziano i figli di un nodo. Quando non sono occupate da un nodo sono posizioni vuote.

Albero k -ario: Un albero k -ario è un albero posizionale in cui ogni posizione maggiore di k è vuota, ciò si traduce nel dire che ogni nodo ha al massimo k figli. Viene utilizzato per rappresentare strutture con una quantità fissa di sottoelementi.

Albero binario: Un albero binario è un esempio di albero k -ario in cui ogni nodo ha al massimo due figli, pertanto $k = 2$. I due figli sono tipicamente chiamati "destro" e "sinistro". Questa tipologia di albero è fondamentale per molte strutture dati e algoritmi, come gli alberi binari di ricerca e gli heap.

9.2 Alberi binari

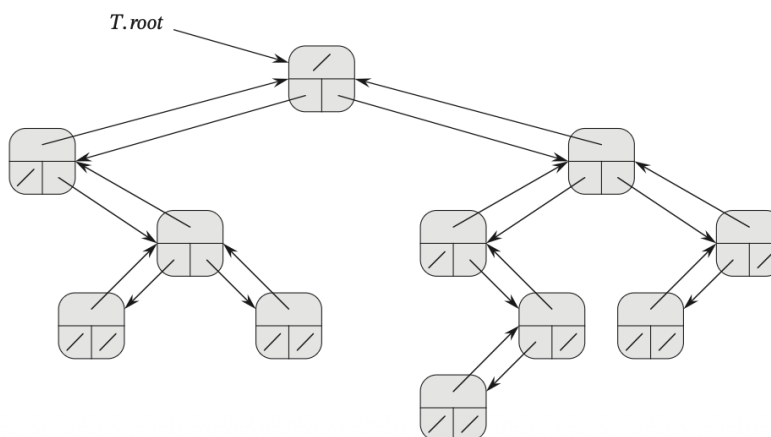
Rappresentazione degli alberi binari

Ogni nodo dell'albero è composto dai seguenti attributi:

1. Puntatore al nodo padre: **p**
2. Puntatore al figlio sinistro: **l**
3. Puntatore al figlio destro: **r**
4. Chiave del nodo: **key**, ovvero l'attributo in cui viene memorizzata l'etichetta del nodo

Figura 10.6

La rappresentazione di un albero binario T . Ogni nodo x ha gli attributi $x.p$ (in alto), $x.left$ (in basso a sinistra) e $x.right$ (in basso a destra). Gli attributi key non sono mostrati.



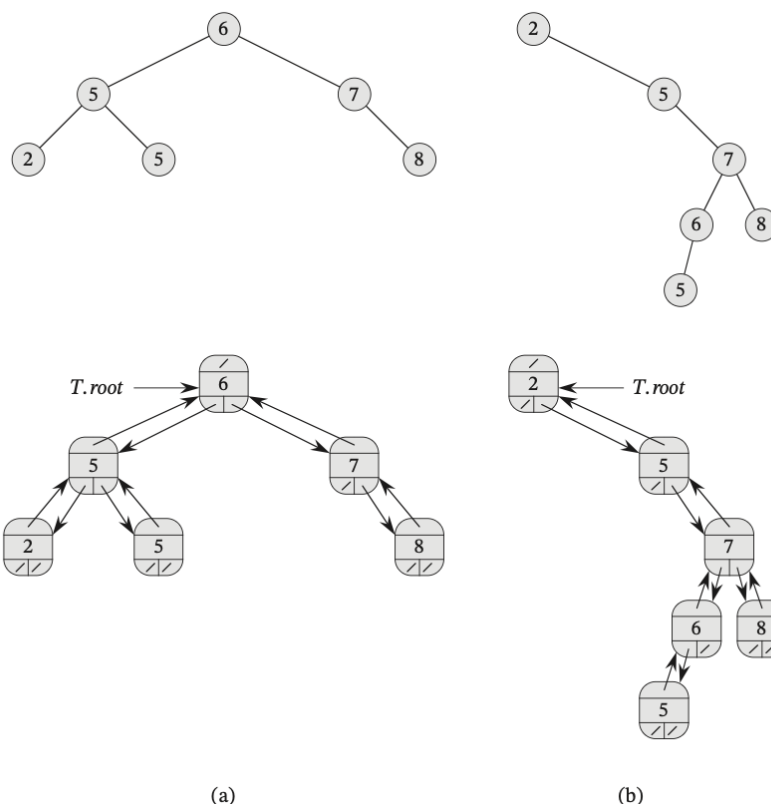
Albero binario di ricerca BST

Un albero di ricerca definito con la sigla BST, è un albero binario che soddisfa la seguente proprietà:

Definizione 9.2.1. Sia x un nodo di un albero di ricerca, se y è un nodo del sottoalbero sinistro di x allora $y.key \leq x.key$, mentre se y è un nodo del sottoalbero destro di x allora $y.key \geq x.key$

Figura 12.1

Alberi binari di ricerca. Per ogni nodo x , le chiavi nel sottoalbero sinistro di x valgono al più $x.key$ e le chiavi nel sottoalbero destro di x valgono almeno $x.key$. Alberi binari di ricerca differenti possono rappresentare lo stesso insieme di valori. Il tempo di esecuzione nel caso peggiore per la maggior parte delle operazioni di ricerca in un albero è proporzionale all'altezza dell'albero. (a) Un albero binario di ricerca con 6 nodi e altezza 2. La figura in alto mostra come visualizzare l'albero concettualmente, mentre la figura in basso mostra gli attributi $left$, $right$ e p di ogni nodo, come nella Figura 10.6 a pagina 238. (b) Un albero binario di ricerca meno efficiente di altezza 4 che contiene le stesse chiavi.



Visita di un albero binario

Diversamente dal caso delle strutture lineari, dove è possibile visitare tutti gli elementi in sequenza nell'ordine in cui sono rappresentati, nel caso di alberi binari vi sono modi diversi di esplorare i nodi. In particolare, le tre operazioni caratteristiche della visita in profondità (visita della radice, visita del sottoalbero sinistro, visita del sottoalbero destro) possono essere realizzate in qualunque ordine. Esistono tre versioni di visita in profondità:

1. **Attraversamento simmetrico o visita in ordine (inorder tree walk):** sottoalbero sinistro, radice, sottoalbero destro. Ciò implica che i nodi vengono visitati in ordine crescente di chiave.
2. **Attraversamento anticipato (preorder tree walk):** radice, sottoalbero sinistro, sottoalbero destro. Ciò implica che la radice venga visitata prima dei suoi sottoalberi.

3. **Attraversamento posticipato (postorder tree walk):** sottoalbero sinistro, sottoalbero destro, radice.
Ciò implica che la radice venga visitata dopo dei suoi sottoalberi.

Pseudocodice

```
Pre-order(x){
    if x != nil
        print x.key
        Preorder(x.left)
        Preorder(x.right)
}

Post-order(x){
    if x != nil
        Postorder(x.left)
        Postorder(x.right)
        print x.key
}

In-order(x){
    if x != nil
        Inorder(x.left)
        print x.key
        Inorder(x.right)
}
```

Analisi della complessità

Teorema 9.2.1. Se x è la radice di un sottoalbero con n nodi allora **Inorder(x)** richiede un tempo $\Theta(n)$

Dimostrazione. Sia $T(n)$ il tempo richiesto da Inorder(x) per visitare un sottoalbero di n nodi con radice x , è necessario dimostrare che $T(n)$ è $\Omega(n)$ e allo stesso tempo $O(n)$.

Dire che $T(n)$ è $\Omega(n)$ significa che $T(n)$ è almeno pari a $c \cdot n$ per qualche costante c e per n sufficientemente grande. Durante l'attraversamento in-ordine dobbiamo visitare tutti i nodi, ogni nodo deve essere visitato almeno una volta, ecco spiegato il motivo per il quale il tempo richiesto per visitare tutti i nodi è almeno proporzionale ad n .

Dire che $T(n)$ è $O(n)$ significa che $T(n)$ è al massimo $c \cdot n$ per qualche costante c e per n sufficientemente grande. Sapendo che l'algoritmo di attraversamento in-ordine segue questi passi:

1. Visita il sottoalbero sinistro
2. Visita la radice x
3. Visita il sottoalbero destro

Supponiamo che $n = 0$, per visitare un nodo serve una piccola quantità di tempo, basta verificare che $x = nil$, ecco perchè $T(n = 0) = c$.

Supponendo che $n > 0$ e che il sottoalbero sinistro abbia k nodi e il sottoalbero destro ne abbia $n - k - 1$, il tempo totale $T(n)$ è quindi minore o uguale alla somma dei tempi richiesti per visitare il sottoalbero sinistro, la radice e il destro:

$$T(n) \leq T(k) + T(n - k - 1) + d$$

dove d è il tempo costante per visitare la radice x (che è un'operazione $O(1)$). Applichiamo il metodo di sostituzione per dimostrare che:

$$T(n) = (c + d)n + c \text{ per ogni } n \geq 0$$

Caso base ($n=0$): $T(0) = c = (c + d) \cdot 0 + c$

Per $n > 0$:

$$\begin{aligned} T(n) &\leq T(k) + T(n - k - 1) + d \\ &= ((c + d)k + c) + ((c + d)(n - k - 1) + c) + d \\ &= (c + d)k + c + (c + d)n - (c + d)k - (c + d) + c + d \\ &= c - c - d + d + c + (c + d)n = (c + d)n + c \end{aligned}$$

□

Interrogazioni di un albero binario di ricerca

I BST sono strutture dati sulle quali vengono realizzate molte delle operazioni definite su insiemi dinamici, anche chiamate "interrogazioni"

1. **Search(x, key):** data la radice x e una chiave key , restituisce il puntatore a un nodo con chiave key , se esiste, o nil
2. **Minimum(x) o Maximum(x):** dato un albero con radice x , restituisce l'elemento la cui chiave è minima o massima
3. **Successor(x) o Predecessor(x):** dato un albero con radice x , restituisce il nodo successivo o precedente a x rispetto all'attraversamento simmetrico

Pseudocodice

```
Search(x, k){
    if x == nil or k == x.key
        return x
    if x < x.key
        return Search(x.left, k)
    else
        return Search(x.right, k)
}

Minimum(x){
    while x.right != nil
        x = x.right
    return x
}

Maximum(x){
    while x.right != nil
        x = x.right
    return x
}

Successor(x){
    if x.right != nil
        return Minimum(x.right)
    else
        y = x.p
        while y != nil and x == y.right
            x = y
            y = x.p
        return y
}

Predecessor(x){
    if x.left != nil
        return Maximum(x.left)
    else
        y = x.p
        while y != nil and x == y.left
            x = y
            y = x.p
        return y
}
```

Tutte queste funzioni hanno costo $O(h)$, dove h è l'altezza del nodo x nell'albero.

Tra queste vi è una categoria "a parte" che riguarda le operazioni che agendo sull'albero ne modificano il contenuto. Nel caso di alberi binari di ricerca queste operazioni devono essere realizzate preservando la proprietà dell'ordinamento delle chiavi. Ciò significa che è necessario verificare se un inserimento o cancellazione interferiscono con tale proprietà, in tal caso è necessario aggiornare l'albero affinché sia possibile ripristinarla.

1. **Insert(T, z):** l'idea è quella di inserire nell'albero un nuovo nodo z sottoforma di foglia. Affinchè ciò sia possibile, è necessario trovare una foglia che possa diventare per essa il padre senza compromettere le proprietà dell'albero

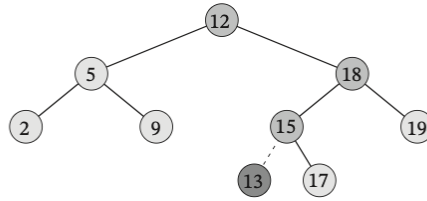
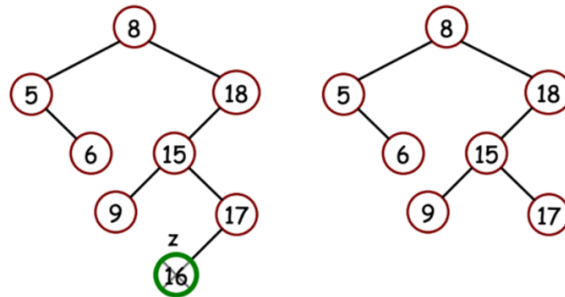


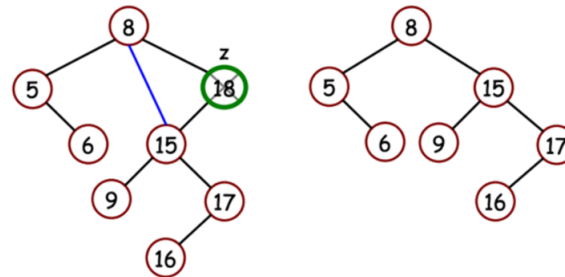
Figura 12.3 L'elemento con la chiave 13 viene inserito in un albero binario di ricerca. Il cammino semplice, dalla radice verso la posizione dove l'elemento viene inserito, è indicato dai nodi in grigio intermedio. Il nuovo nodo è in grigio scuro e il collegamento con il nodo padre è tratteggiato.

2. **Delete(T, z):** il ragionamento per questo tipo di operazione non è unico, bensì ci si può trovare in tre situazioni diverse:

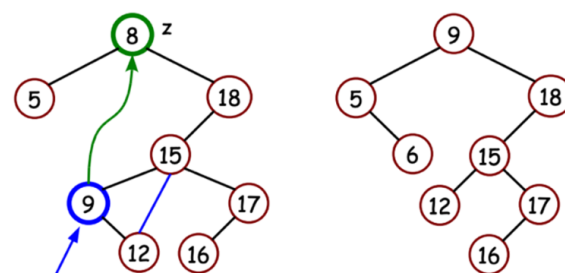
- **z è una foglia**, ovvero non ha figli: si elimina il nodo modificando a nil il puntatore del padre



- **z ha un figlio, w :** in tal caso è come se il nodo w venisse "promosso" a prendere il posto di z nell'albero, aggiornando il puntatore del padre di z in modo che punti a w



- **z ha due figli:** bisogna trovare il successore y di z nel sottoalbero destro, spostare y nella posizione precedentemente coperta da z , infine ricollegare i sottoalberi. Per spostare y bisogna tener conto di una condizione, ovvero se y è figlio destro di z semplicemente lo si fa risalire per poi "attaccargli" il vecchio sottoalbero sinistro di z , altrimenti si scambia y con il suo figlio destro per poi portare verso l'alto y fino a sostituire z



Tree-Successor(T, z) è il nodo più a sinistra del sottoalbero destro di z , ha al più un figlio (destro)

Pseudocodice

```
Insert(T, z){
    x = T.root
    y = nil
    while x != NULL //trovo nodo dove attaccare z
        y = x
        if z.key < y.key
            x = y.left
        else
            x = y.right
    z.p = y

    if y == nil // l' albero e' vuoto
        T.root = z // z diventa la radice
    else if z.key < y.key // inserisco z nell ' albero
        y.left = z
    else
        y.right = z
}

Transplant(T, u, v){
    if u.p == nil
        T.root = v
    else if u == u.p.left
        u.p.left = v
    else
        u.p.right = v
    if v != nil
        v.p = u.p
}

Delete(T, z){
    if z.left == nil
        // se il sottoalbero sinisto di z e' vuoto, devo attaccare z. right a z.p
        Transplant(T, z, z.right)
    else if z.right == nil
        // se il sottoalbero destro di z e' vuoto, devo attaccare z. left a z.p
        Transplant(T, z, z.left)
    // cosi facendo ho trattato i casi se z ha un solo figlio oppure e' una foglia
    else
        y = Minimum(z.right) // trovo y, il successore di z
        // spostato y nella posizione di z, tenendo conto dei sottoalberi di y
        if y != z.right
            Transplant(T, y, y.right)
            y.right = z.right
            y.right.p = y
        Transplant(T, z, y)
        y.left = z.left
        y.left.p = y
}
```

Costo $O(h)$, dove h è l'altezza dell'albero.

9.3 Alberi Rosso-Neri

Un albero rosso-nero è un albero binario di ricerca con un bit in più di memoria per ogni nodo necessario per il colore, che può essere RED o BLACK. L'albero si dice **approssimativamente bilanciato** poichè, grazie a specifici vincoli per assegnare i colori ai nodi, garantisce che nessun cammino che va dalla radice a una foglia sia lungo più del doppio di qualsiasi altro. Ogni nodo ha i seguenti attributi: color, key, left, right e p. Se manca

un figlio o il padre di un nodo, il corrispondente attributo puntatore del nodo contiene il valore NIL, il quale viene trattato come un puntatore a una foglia.

Proprietà di un albero rosso-nero

1. Ogni nodo è rosso o nero
2. La radice è nera
3. Ogni foglia (NIL) è nera
4. Se un nodo è rosso, allora entrambi i suoi figli sono neri
5. Per ogni nodo, tutti i cammini semplici che vanno dal nodo alle foglie sue discendenti contengono lo stesso numero di nodi neri

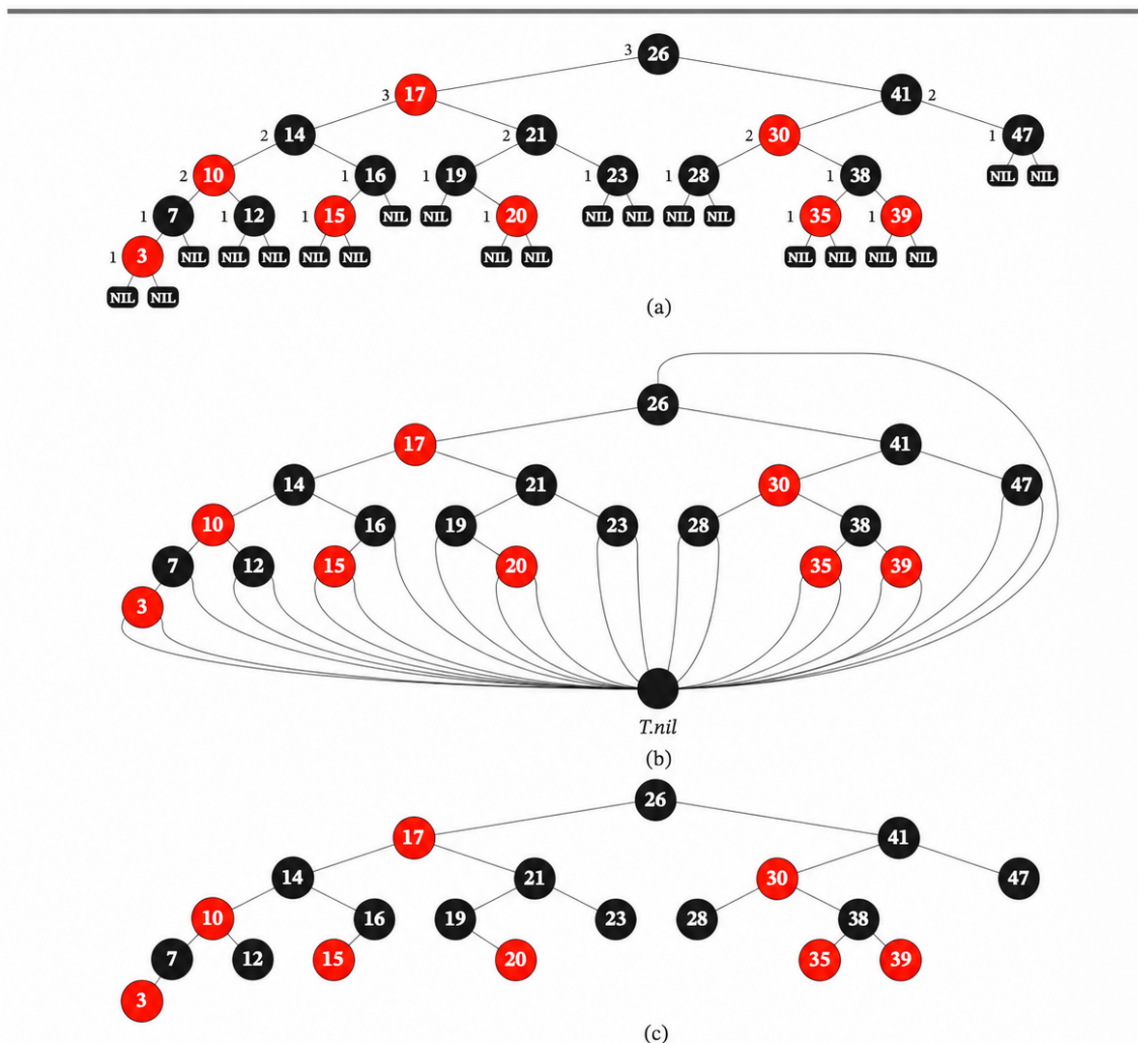


Figura 13.1 Un albero rosso-nero. Ogni nodo di un albero rosso-nero è rosso o nero (qui i nodi neri sono su sfondo nero, quelli rossi su sfondo grigio); i figli di un nodo rosso sono entrambi neri e ogni cammino semplice che va da un nodo a una foglia sua discendente contiene lo stesso numero di nodi neri. (a) Ogni foglia, rappresentata da NIL, è nera. Ogni nodo che non è NIL ha indicata a fianco la sua altezza nera; i nodi NIL hanno altezza nera 0. (b) Lo stesso albero rosso-nero dove tutti i nodi NIL sono stati sostituiti dall'unica sentinella $T.nil$, che è sempre nera (le altezze nere non sono indicate). La sentinella è anche il padre della radice. (c) Lo stesso albero rosso-nero; qui non sono indicate le foglie, così come il padre della radice. Nel resto del capitolo si usa questo ultimo schema di rappresentazione.

Per un albero rosso-nero si definisce sentinella, l'oggetto $T.nil$ con gli stessi attributi di un nodo ordinario dell'albero, il suo colore è "BLACK" e gli attributi p , $left$, $right$ e key possono assumere valori arbitrari. Bisogna notare che tutti i puntatori a NIL sono sostituiti con puntatori alla sentinella. L'uso di quest'ultima permette di trattare un figlio NIL di un nodo x come un nodo ordinario il cui padre è x .

Definizione 9.3.1. Si definisce altezza nera di un nodo x , indicata da $\text{bh}(x)$, il numero di nodi neri lungo un cammino semplice che inizia dal nodo x (ma non lo include) e termina in una foglia.

Definizione 9.3.2. L'altezza massima di un albero rosso-nero con n nodi interni è $2 \log(n + 1)$

Dimostrazione. Dimostriamo prima (per induzione) la seguente proprietà:

il sottoalbero radicato in un nodo x qualunque contiene almeno $2^{\text{bh}(x)} - 1$ nodi interni

Passo base:

$h = 0$ (x è una foglia)

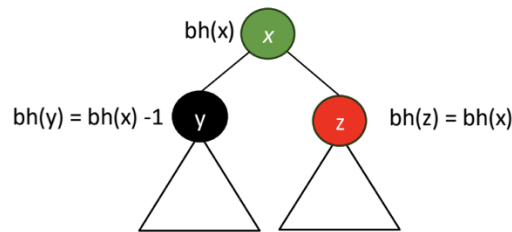
se x è una foglia $\text{bh}(x) = 0$

$2^0 - 1 = 1 - 1 = 0$

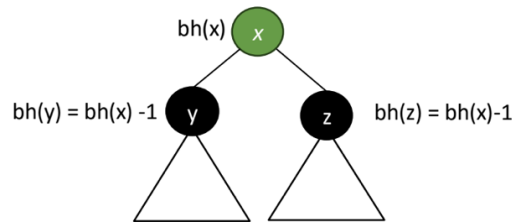
Passo induttivo:

$h > 0$ (x è un nodo interno)

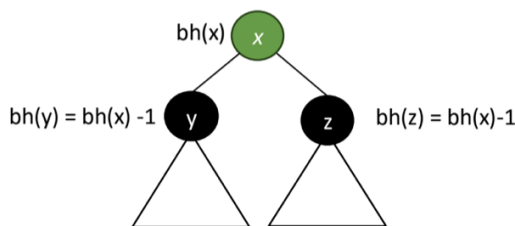
x ha due figli, uno dei quali (o entrambi) potrebbe essere una foglia



Per l'ipotesi induttiva, poichè l'altezza dei figli di x è inferiore all'altezza h di x , possiamo affermare che ciascun figlio ha almeno $2^{\text{bh}(x)-1} - 1$ nodi interni.



Quindi, il sottoalbero radicato in x ha almeno $(2^{\text{bh}(x)-1} - 1) + (2^{\text{bh}(x)-1} - 1) + 1 = 2^{\text{bh}(x)} - 1$ nodi interni da cui segue la proprietà



Per concludere la dimostrazione, ragioniamo ora sull'altezza dell'albero h

- per la proprietà 4 degli alberi rosso-neri, almeno la metà dei nodi su un cammino verso le foglie sono neri
- ne segue che $\text{bh}(\text{root}) \geq h/2$, e quindi
- $n \geq 2^{h/2} - 1$
- $(n + 1) \geq 2^{h/2}$
- $\log(n + 1) \geq h/2$
- $h \leq 2 \log(n + 1)$

□

Un'immediata conseguenza di questa definizione è che su di un albero rosso-nero con n nodi interni le operazioni Search, Minimum, Maximum, Successor e Predecessor richiedono tutte un tempo $O(\log n)$.

Definizione 9.3.3. Negli alberi rosso-neri le operazioni **Insert** e **Delete** sono opportunamente modificate inserendo una fase di ribilanciamento per garantire un'altezza dell'albero

$$h = O(\log n)$$

Capitolo 10

Esercizi

10.1 Esercizi venerdì 13/03/2026

Esercizio 1

Si consideri una lista di interi A (non vuota) di dimensione n contenente tutti gli interi compresi tra 1 e $n+1$ in cui vi è un unico buco (ossia in cui manca un unico elemento). Assumendo che A sia ordinato in modo crescente

1. scrivere lo pseudo-codice di un algoritmo efficiente che restituisce il valore mancante
 - es. se $A=[1, 2, 3, 5, 6, 7]$ l'algoritmo deve restituire 4
2. implementare l'algoritmo proposto in JAVA
3. determinare il costo computazionale dell'algoritmo proposto al punto 1

1.

```
Trova_buco(A,i,j)
    if(A[i] != i e |A| > 1)
        return i
    if(A[j] = j e |A| > 1)
        return j+1
    if(|A|=1)
        if(A[i] != i)
            return 1
        else return 2
    q = punto medio tra i e j
    if q = A[q]
        Trova_buco(q+1,j)
    else
        Trova_buco(i,q)
```

All'inizio mi trovo con $i=1$ e $j=7$, nel primo if controllo se c'è il buco nel primo elemento e lo restituisco. Nel secondo if controllo se c'è il buco nell'ultimo elemento.

Esercizio 2

Consideriamo due liste A e B contenenti rispettivamente n e m interi. Le due liste sono ordinate in senso crescente e ciascuna lista contiene elementi distinti. Tuttavia, lo stesso valore v può comparire sia in A che in B

1. Scrivere lo pseudo-codice di un algoritmo efficiente che sia in grado di stampare tutti i valori di A che **NON** sono presenti in B
 - es. Se $A = [1, 3, 4, 7, 8]$ e $B = [2, 3, 5, 7]$ l'algoritmo deve stampare 1, 4, 8
2. implementare la soluzione proposta in JAVA
3. Determinare il costo computazionale dell'algoritmo presentato al punto 1

1.

$A = [1, 3, 4, 7, 8]$ con $|A| = n$

$B = [2, 3, 5, 7]$ con $|B| = m$

Procedura merge-sort, uso indice i per A e j per B :

```
A = [1, 3, 4, 7, 8]
      ^
      |
      i

B = [2, 3, 5, 7]
      ^
      |
      j
```

Questo è lo pseudocodice:

```
i = 1
j = 1
while (i <= n AND j <= m)
    if(A[i] < B[j])
        stampo(A[i])
        i = i + 1
    else if (A[i] > B[j])
        j = j + 1
    else
        i = i + 1
        j = j + 1
while(i <= n)
    stampo A[i]
    i = i + 1
```

da correggere per quando ci sono i duplicati in A

Costo algoritmo $n + m$

Se non era ordinato comunque aveva un vantaggio prima ordinare le due liste e poi applicare il merge invece di fare la ricerca binaria

Esercizio 3

Consideriamo un array A di numeri reali di dimensione n.

1. Scrivere lo pseudo-codice di un algoritmo che permuti A in modo tale che tutti i valori negativi compaiano prima di tutti quelli positivi. Non è necessario che i valori negati e positivi siano a loro volta ordinati.
2. Implementare l'algoritmo proposto in JAVA
3. determinare il costo computazionale dell'algoritmo proposta al punto 1.

1.

In questo caso usiamo Quick sort con pivot=0.

```
A = [-1, 7, 15, -3, 2, 8, -10]
      ^ ^
      | |
      i j
```

```
A = [-1, 7, 15, -3, 2, 8, -10]
      ^
      |
      j,i
```

```
A = [-1, 7, 15, -3, 2, 8, -10]
      ^           ^
      |           |
      i           j
```

pseudocodice

```
while(j <= |A|)
  if(A[j] < 0)
    swap(A[i+1], A[j])
    i++
  j++
```

10.2 Esercizi venerdì 17/04/2026 (Tabelle Hash)

Esercizio 1

Consideriamo una funzione hash $h: U \rightarrow \{0, 1, \dots, m-1\}$. h si dice *perfetta* se per ogni coppia di chiavi distinte x e y , si ha $h(x) \neq h(y)$.

1. Scrivere un algoritmo per il calcolo di una funzione hash perfetta in cui l'insieme universo sia costituito da sequenze di cinque caratteri scelti tra i seguenti: 'A', 'C', 'T', 'G' e il valore di m sia minimo possibile.
2. Determinare il costo computazionale dell'algoritmo di cui al punto precedente.

Abbiamo che S è:

$$S = \{\text{stringhe di lunghezza 5 con i soli caratteri 'A', 'C', 'T', 'G'}\}$$

In S ci sono stringhe del tipo: "AAAAA", "AAAAC", "AAACA", ..., quindi $|S|$ (cardinalità di S) è calcolata come tutte le permutazioni di 4 caratteri in 5 blocchi, quindi $|S|$ è:

$$|S| = 4^5$$

Di conseguenza anche la dimensione della tabella hash è $m = 4^5$.

Per calcolare la funzione hash perfetta associamo ad ogni carattere un numero da 0 a 4:

A	C	T	G
0	1	2	3

L'algoritmo per la funzione hash perfetta è questo, $\forall s \in S$:

1. converti s in base 4.
2. converti s da base 4 in base 10.
3. restituisci la tabella.

Qual è il costo di questo algoritmo?

- il passaggio (1) ha costo costante.
- il passaggio (2) ha costo costante.

In totale l'algoritmo ha costo costante.

Esercizio 2

Sia dato l'insieme di chiavi $K = \{25, 83, 57, 26, 13, 60, 93, 4\}$, e sia $m=10$.

Si consideri una tabella hash (inizialmente vuota) di dimensione m .

Mostrare come cambia la struttura della tabella ogni volta che si inserisce una delle chiavi di K , nell'ordine indicato

- La funzione hash utilizzata è $h(k) = k \bmod m$.
- La tabella utilizza il metodo delle liste collegate per gestire le collisioni
 - assumere che gli elementi vengano sempre inseriti in fondo alle liste.

Abbiamo una tabella hash da riempire di dimensioni $m = 10$, e delle chiavi:

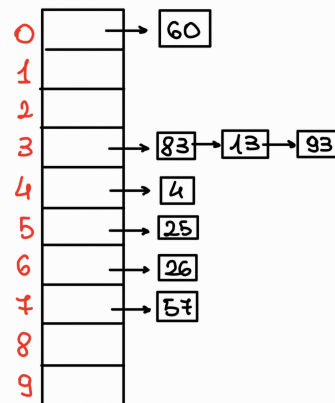
$$K = \{25, 83, 57, 26, 13, 60, 93, 4\}$$

Per gestire le collisioni dobbiamo usare il metodo delle liste collegate, quindi quando troviamo che in una cella corrispondente ad una chiave c'è già un valore, "agganciamo" a quel valore il nuovo valore che dobbiamo inserire. La funzione hash è:

$$h(k) = k \bmod m = \boxed{k \bmod 10}$$

- $h(25) = 25 \bmod 10 = 5$
- $h(83) = 83 \bmod 10 = 3$
- $h(57) = 57 \bmod 10 = 7$
- $h(26) = 26 \bmod 10 = 6$
- $h(13) = 13 \bmod 10 = 3$, collego a 83
- $h(60) = 60 \bmod 10 = 0$
- $h(93) = 93 \bmod 10 = 3$, collego a 13
- $h(4) = 4 \bmod 10 = 4$

→



Esercizio 3

Si consideri una tabella hash memorizzata in un vettore $A[0..m-1]$ con $m = 10$ slot in cui le collisioni sono gestite mediante indirizzamento aperto.

L'insieme delle chiavi è costituito da interi non negativi.

La funzione hash adottata è $h(k, i) = ((k \bmod m) + i) \bmod m$, essendo k il valore della chiave e i il contatore del "tentativo di inserimento".

Mostrare il contenuto del vettore A dopo ciascuna delle operazioni seguenti:

1. INSERT(7)
2. INSERT(19)
3. INSERT(37)
4. DELETE(7)
5. INSERT(21)
6. DELETE(37)
7. INSERT(17)
8. INSERT(11)
9. DELETE(21)

La tabella hash ha dimensione $m = 10$, e la funzione di hash è la seguente:

$$h(k, i) = ((k \bmod m) + i) \bmod m = \boxed{((k \bmod 10) + i)}$$

dove:

- k è la chiave
- i è il contatore per i tentativi di inserimento

Le collisioni verranno gestite con indirizzamento aperto, quindi se una cella è già occupata, semplicemente il valore non verrà inserito e incrementiamo il contatore i , fino a quando non troviamo una cella libera.

- INSERT(7): $h(7, 0) = 7 \bmod 10 + 0 = 7$
- INSERT(19): $h(19, 0) = 19 \bmod 10 + 0 = 9$
- INSERT(37): $h(37, 0) = 37 \bmod 10 + 0 = 7$,
occupato da 7

$$- h(37, 1) = 37 \bmod 10 + 1 = 8$$

- DELETE(7): 7 diventa NULL
- INSERT(21): $h(21, 0) = 21 \bmod 10 + 0 = 1$
- DELETE(37): 37 diventa NULL
- INSERT(17): $h(17, 0) = 17 \bmod 10 + 0 = 7$,
occupato da NULL
- $h(17, 1) = 17 \bmod 10 + 1 = 8$,
occupato da NULL
- $h(17, 2) = 17 \bmod 10 + 2 = 9$,
occupato da 19
- $h(17, 3) = 17 \bmod 10 + 3 = 0$

- INSERT(11): $h(11, 0) = 11 \bmod 10 + 0 = 1$,
occupato da 21

$$- h(11, 1) = 11 \bmod 10 + 1 = 2$$

- DELETE(21): 21 diventa NULL

0	17
1	NULL
2	11
3	
4	
5	
6	
7	NULL
8	NULL
9	19